



Designing Efficient, Practical and Secure Post-quantum Cryptosystems

Anaëlle Le Dévéhat

► To cite this version:

Anaëlle Le Dévéhat. Designing Efficient, Practical and Secure Post-quantum Cryptosystems. Cryptography and Security [cs.CR]. Institut Polytechnique de Paris, 2025. English. ⟨NNT : 2025IPPAX114⟩. ⟨tel-05466211v2⟩

HAL Id: tel-05466211

<https://hal.science/tel-05466211v2>

Submitted on 5 Feb 2026

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



HAL Authorization

Designing Efficient, Practical and Secure Post-quantum Cryptosystems

Thèse de doctorat de l'Institut Polytechnique de Paris
préparée à l'École polytechnique

École doctorale n°626 École doctorale de l'Institut Polytechnique de Paris (EDIPP)
Spécialité de doctorat : Informatique

Thèse présentée et soutenue à Palaiseau, le 16 décembre 2025, par

ANAËLLE LE DÉVÉHAT

Composition du Jury :

Emmanuel Haucourt Professeur, École polytechnique (Cosynus)	Président
Fabien Laguillaumie Professeur, Université of Montpellier (ECO-LIRMM)	Rapporteur
Pierre-Jean Spaenlehauer Chargé de recherche, Inria Nancy – Grand Est (CARAMBA)	Rapporteur
Éric Schost Professeur, University of Waterloo	Examineur
Monika Trimoska Professeure assistante, Eindhoven University of Technology (TU/e)	Examineur
Benjamin Smith Directeur de recherche, École polytechnique (GRACE)	Directeur de thèse
François Morain Professeur, École polytechnique (GRACE)	Co-directeur de thèse

*À Maman et Papa,
pour tout.*

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à mon encadrant Ben Smith, sans qui cette thèse n'aurait jamais pu avoir lieu. Ses conseils, sa disponibilité et sa confiance m'ont accompagnée tout au long de ces trois dernières années. Je le remercie tout particulièrement pour sa grande compréhension et son soutien, qui ont énormément compté pour moi à certains moments de cette thèse.

Je tiens aussi à remercier François Morain pour ses nombreux conseils, ses encouragements et son suivi.

Durant cette thèse, j'ai fait la connaissance de nombreux chercheurs exceptionnels. J'ai notamment eu la chance de pouvoir collaborer avec Gustavo Banegas et Valérie Gilchrist. Mes travaux avec eux font partie, pour moi, des meilleurs moments de cette aventure. Je remercie également tous mes collègues de l'équipe GRACE à Inria Saclay pour leur bonne humeur et leur soutien durant ces trois dernières années.

I want to especially thank my friends from research that I met at work or during conferences. Meeting all of you is what I am most thankful. Special thanks to the isogeny community for making everything so much fun. Merci Abel, je n'ai jamais eu autant de chance que de m'asseoir à côté de toi lors de mon tout premier jour aux Isogeny Days en 2022. Thank you Valérie, Lucas, Axel, Laurane, Gioella, Maria, Ilinca, Bruno, Sana and many others.

Au cours de ces trois dernières années, j'ai réalisé ma thèse majoritairement au Campus Cyber aux côtés d'Éléonore Ndiaye. Chaque moment passé ensemble, nos pauses café, nos confidences, nos rires et ton amitié me sont si précieux, merci.

While being at LIX, I quickly met Bernardo, another Ph.D. student a few offices away along the corridor. Thank you for saving me countless times with research, paperwork,

and everything in between. You will forever be my Ph.D. brother.

Merci à tous mes amis pour leur présence et ces moments partagés après le travail, qui m'ont aidée à retrouver de l'énergie et à garder l'équilibre. Si j'ai pu continuer à faire cette thèse pendant trois ans, c'est grâce à vous. Merci de m'avoir soutenue dans les meilleurs moments comme dans les plus difficiles. Merci Mathias, Charles, Béatrice, Matéo, Jérémy, Lise, Pierre, Francesca, Johanna, Yassine, Bérénice, Jeanne, Marion et bien d'autres.

I want to address a special thank-you to Jonathan Komada Eriksen for not only his support during these last years but also for reading this manuscript. His valuable comments helped me shape this manuscript into what it is now.

Je remercie chaleureusement ma sœur Nina, pour tout son amour. Merci Thomas, Odei et Eneko pour leurs sourires et les bouffées de joie qui ont rythmé cette thèse.

Enfin, il m'est impossible de formuler avec de simples mots l'immensité de ma gratitude à mes parents, Nathalie et Éric. Leur soutien et leur amour inconditionnels me sont inestimables. Merci pour tout.



This work was supported by the France 2030 program under grant agreement ANR-22-PETQ-0008 PQ-TLS.

Résumé en français

Cette thèse explore des stratégies d'optimisation pour la cryptographie post-quantique. Par systèmes cryptographiques post-quantiques, nous entendons des schémas classiques (non quantiques) dont la sécurité est destinée à résister à des attaques provenant d'ordinateurs quantiques. L'optimisation en cryptographie peut signifier beaucoup de choses différentes. Elle prend souvent la forme d'un compromis entre efficacité, praticité, simplicité et sécurité. Quand on parle de concevoir des cryptosystèmes efficaces, pratiques et sûrs, il est important de garder à l'esprit leurs applications et les caractéristiques visées. Ainsi, dans certains cas, par exemple dans l'IoT, on peut viser des tailles de clés et des calculs plus faibles plutôt qu'une sécurité plus élevée. Cette thèse est divisée en six chapitres.

Le **Chapitre 2** fournit le cadre mathématique nécessaire pour comprendre les chapitres suivants. Nous introduisons d'abord le concept d'action de groupe et de cryptographie basée sur les actions de groupe. Nous définissons ensuite les courbes elliptiques et les isogénies pour comprendre comment construire une action de groupe à partir de ces objets. Nous donnons aussi une courte introduction aux réseaux et aux codes en cryptographie. À mesure que nous introduisons ces concepts, nous présentons une vue d'ensemble de différents cryptosystèmes pertinents pour le reste de notre travail.

Notre première contribution, dans le **Chapitre 3**, consiste en un article publié à CANS 2025 : *Compressed verification for post-quantum signatures with long-term public keys* (travail conjoint avec Gustavo Banegas et Benjamin Smith). Elle s'applique aux signatures dans le cadre GPV avec des réseaux ou des codes non structurés. De nombreuses applications de signatures — telles que les certificats racine, les mises à jour logicielles sécurisées et les protocoles d'authentification — impliquent des clés publiques

de longue durée qui sont transférées ou installées une seule fois puis utilisées pour de nombreuses vérifications. Cette longévité des clés rend les schémas de signatures post-quantiques avec des hypothèses conservatrices (par ex., des réseaux sans structure) attrayants pour une sécurité à long terme. Cependant beaucoup de ces schémas, en particulier ceux avec des signatures courtes, souffrent de clés publiques extrêmement volumineuses. Même dans des scénarios où la bande passante n'est pas une préoccupation majeure, de grandes clés augmentent les coûts de stockage et ralentissent la vérification. Ainsi, même si la cryptographie basée sur les réseaux est aujourd'hui au centre de la cryptographie post-quantique, elle se concentre principalement sur des cryptosystèmes reposant sur des réseaux structurés. Garder une attention sur des alternatives telles que le cadre GPV général reste nécessaire. La cryptographie doit être polyvalente au cas où de nouvelles attaques rendraient certaines familles de cryptosystèmes compromises. Nous abordons cela avec une méthode visant à remplacer de grandes clés publiques dans des signatures de style GPV par des clés de vérification privées plus petites. Cela réduit significativement le stockage et le temps d'exécution du vérificateur tout en préservant la sécurité. Appliquée aux schémas conservateurs à signatures courtes Wave et Squirrels, notre méthode compresse les clés Squirrels-I de 665 kB à 20.7 kB et les clés Wave822 de 3.5 MB à 207.97 kB.

Notre deuxième contribution, dans le **Chapitre 4**, consiste en un article publié à Latincrypt 2023 : *Fast and Frobenius: Rational Isogeny Evaluation over Finite Fields* (travail conjoint avec Gustavo Banegas, Valerie Gilchrist et Benjamin Smith). Nous apportons une accélération du calcul d'isogénies. Considérons le problème d'évaluer efficacement des isogénies $\varphi : E \rightarrow E/H$ de courbes elliptiques sur un corps fini $\mathbb{F}_q$, où le noyau $H = \langle G \rangle$ est un groupe cyclique d'ordre impair (premier) : étant donnés E , G , et un point (ou plusieurs points) P sur E , nous voulons calculer $\varphi(P)$. Ce problème est au cœur des implémentations de cryptosystèmes post-quantiques basés sur les actions de groupe et sur les isogénies tels que CSIDH. Les algorithmes basés sur les formulæ de Vêlu donnent une solution efficace lorsque le générateur du noyau G est défini sur $\mathbb{F}_q$. Cependant pour des isogénies générales, G n'est défini que sur une extension $\mathbb{F}_{q^k}$, même si $\langle G \rangle$ dans son ensemble (et donc φ) est défini sur le corps de base $\mathbb{F}_q$. Les performances des algorithmes d'évaluation d'isogénies se dégradent rapidement à mesure que k grandit. Dans ce chapitre, nous réexaminons cela avec un focus particulier sur le cas où $1 \leq k \leq 12$. Nos techniques accélèrent des schémas commutatifs basés sur les isogénies, tels que CSIDH [20] et CSI-FiSh [13]. Elles accélèrent aussi l'évaluation d'isogénies dans le cryptosystème de Couveignes-Rostovtsev-Stolbunov (CRS), ce qui constitue une première étape vers une mise en pratique plus réaliste afin qu'il puisse être

une solution de repli utilisant des courbes ordinaires pour CSIDH.

Notre troisième contribution, dans les **chapitres 5 et 6**, trouve sa racine dans des algorithmes plus classiques de recherche parallèle de collisions. Nous proposons des algorithmes classiques pour des logarithmes discrets en plusieurs dimensions. Notre travail constitue une première étape vers l’algorithme de Gaudry–Schost dans des dimensions supérieures à 4. Cela a une application en cryptographie basée sur les actions de groupe, plus précisément, cela s’applique aux *Restricted Effective Group Actions (REGA)*. Lorsqu’ils étudient la sécurité des cryptosystèmes basés sur REGA, les cryptographes considèrent la complexité d’une recherche parallèle de collisions [118] sur l’espace entier pour établir leurs estimations. Cependant, selon le choix des paramètres, les cryptosystèmes basés sur REGA peuvent être vulnérables à des recherches parallèles de collisions ciblant des sous-ensembles spécifiques. Pour éviter ce risque, nous devons choisir des ensembles de paramètres appropriés. Le principal facteur limitant est un manque de compréhension et de recherche sur les attaques tirant parti des bornes dans une REGA. Les seuls algorithmes connus pour traiter ce problème sont une attaque récente à faible mémoire pour de petites bornes [26] ainsi que l’algorithme de Gaudry–Schost et ses dérivés [56, 59], que nous étendons aux hautes dimensions. Nous analysons la complexité de notre algorithme de deux manières. La première pose les bases d’une analyse générale (poursuivie dans le **Chapitre 6**) en s’appuyant sur les techniques de Gaudry et Schost. La seconde est menée comme une application de l’algorithme de recherche parallèle de collisions de van Oorschot–Wiener. Enfin, nous comparons nos résultats avec une implémentation de preuve de concept.

Contents

1	Introduction	11
1.1	Outline of the thesis	13
2	Background	17
2.1	Group actions	17
2.2	Cryptographic group actions	21
2.3	Elliptic Curves	23
2.4	Isogenies	32
2.5	Class Group Action	38
2.6	Signature protocol	42
2.7	Lattices	43
2.8	Codes	47
3	Eval-idation: Compressed verification for post-quantum signatures with long-term public keys	49
3.1	Introduction	49
3.2	Warm-up: Bernstein’s trick for Rabin–Williams	54
3.3	The general approach	57
3.4	Compressed verification for SQUIRRELS	61
3.5	Compressed verification for WAVE	70
4	Fast and Frobenius: Rational Isogeny Evaluation over Finite Fields	73
4.1	Introduction	73

4.2	Background	74
4.3	Evaluating isogenies	76
4.4	Accelerating Vélu: faster iteration over the kernel	77
4.5	Irrational kernel points: exploiting Frobenius	84
4.6	Applications to key exchange	90
5	Kangaroos: Classical algorithms for restricted hidden shifts and high-dimensional discrete logarithms	93
5.1	Introduction	93
5.2	Group actions and group operations	97
5.3	Solving the MDLP in dimensions ≤ 4	98
5.4	Special points	101
5.5	Solving hidden shifts by random sampling	102
5.6	Tame and Wild shift sets	106
5.7	“Two-thirds” shift sets	108
5.8	Low-memory algorithms	111
5.9	An instance of van Oorschot–Wiener	118
5.10	Experimental evaluation	120
5.11	Comparisons with existing methods	126
5.12	Classical security of CSIDH	128
6	Kangaroos and special points	131
6.1	An adequate new search space	132
6.2	Parallel algorithm layout	134
6.3	Theoretical results	135
6.4	Discussion	139
	Bibliography	143

Chapter 1

Introduction

This document is a compilation of various techniques for optimisations aimed at improving post-quantum cryptographic schemes. Cryptography is the art of secrecy, it answers the need for safe communication over unsafe channels. Historically, cryptography usually arose from a military need to send orders to soldiers without those orders being compromised if the message was intercepted. As a matter of fact, the Spartan governing council and military commanders are considered the first ones to have systematised a cryptographic tool. The Spartan Scytale (c. 5th century BCE) is a transposition cipher which consists of a cylinder with a strip of parchment. The message is written on the parchment wound around the cylinder. As such, the message is only readable on a cylinder of the same diameter. In the context of cryptography, the diameter of the cylinder is the secret key which enables one to decipher. Moving further in history, Caesar's cipher is one of the most famous and simplest encryption schemes. It is a substitution cipher, it encrypts each letter of the plaintext by another letter, a fixed number k of positions down the alphabet. It was famously attributed to Caesar who used it for his private correspondence with a military purpose.

Cryptography has evolved a lot since those times and underwent a revolution provoked by the development of computers in the second half of the 20th century. A lot of attention was paid to symmetric cryptography and more specifically block ciphers such as DES (Data Encryption Standard) [83, 84] and then AES [37]. Later on, asymmetric cryptography has sparked interest [47, 103] to answer a need to exchange and decide on a private key

for symmetric cryptography usage over insecure communication channels. Building on hard computational problems, several such schemes are used every day.

For over a decade now, cryptography has been taking a drastic turn. This was caused by several works. At first, in 1982, Feynman introduced a theoretical way quantum mechanics could be harnessed for computation [52]. Quantum computers and quantum algorithms have been researched since then. The impact of quantum computers on cryptography was shown to be devastating by Peter Shor’s algorithm published in 1994 [108]. This is a quantum algorithm for finding the prime factors of an integer on which current asymmetric cryptography relies. Following this discovery, post-quantum cryptography (i.e., resisting attacks from quantum computers) emerged as a new challenge for research. To catalyse research on post-quantum cryptography, the National Institute of Standards and Technology (NIST) launched a competition in 2016 to identify and select candidates for future standardization [89]. Building on different quantumly-hard problems, different lines of research in post-quantum public-key cryptography are explored.

In this work, we will focus on three different types of post-quantum cryptography: lattice-based cryptography, code-based cryptography and group-action-based cryptography which we will explore the most. In particular, we will shed a lot of light on group actions instantiated through the use of isogenies.

In this document, one of the works focuses on the GPV framework. This framework was introduced as a way to build digital signatures from unstructured lattices by Gentry, Peikert and Vaikuntanathan in 2008 [61]. This groundbreaking work can be seen as one of the first direct constructions of lattice-based signatures. However, this construction lacks efficiency and practicality and has been an active area of research. Notably, in 2012, Micciancio and Peikert improved it and gave a more satisfactory solution to some of its shortages [77]. However, there is still a lot of room for improvement, especially regarding overly large key sizes.

Setting the GPV framework aside, the post-quantum cryptography schemes we work on in this thesis are group-action-based using isogenies. We are particularly interested in a family of cryptosystems that originated in the work of Couveignes [33] and Rostovtsev and Stolbunov [105], often referred to as “CRS”. The hard problem underlying this line of research is: given two isogenous elliptic curves, find an isogeny between them. The CRS cryptosystem, relying on the isogeny problem on ordinary curves, was shown to be too slow. The use of supersingular curves in isogeny-based cryptography was first introduced by Charles, Goren and Lauter constructing hash functions from expander graphs [22]. Following these works, SIDH (Supersingular Isogeny Diffie–Hellman), an

isogeny-based key exchange, came to light. It was submitted to the NIST competition and advanced quite far as one of the finalists. In 2023, it got completely broken by some ground-breaking techniques [19, 75, 104]. These new techniques are now opening the way to a new line of research featuring higher-dimensional isogenies and various cryptosystems building upon the SQISign digital signature [6, 39, 44, 82]. These techniques are out of the scope of this work. A few years after SIDH, a key exchange named CSIDH appeared building upon CRS. It uses the idea of CRS in the specific setting of supersingular curves over $\mathbb{F}_p$. In both cases, CRS and CSIDH are instances of cryptographic group actions.

1.1 Outline of the thesis

The main focus of this thesis is around different optimisations for post-quantum cryptosystems. Optimisation in cryptography can mean a lot of different things. It often comes in the form of a trade-off between efficiency, practicality, simplicity and security. When we talk about designing efficient, practical and secure cryptosystems, it is important to keep in mind their applications and targeted features. As such, for some cases, for example in IoT, one might aim at lower key sizes and computations rather than higher security. This thesis is divided into six chapters.

Chapter 2 provides the mathematical background to understand the following chapters. We first introduce the concept of group action and group-action-based cryptography. We then define elliptic curves and isogenies to understand how to build a group action from these objects. We also give a short introduction to lattices and codes for cryptography. As we introduce these concepts, we make an overview of different cryptosystems relevant for the rest of our work.

Our first contribution, in **Chapter 3**, consists of an article published at CANS 2025: *Compressed verification for post-quantum signatures with long-term public keys* (joint work with Gustavo Banegas and Benjamin Smith). It applies to signatures in the GPV framework with unstructured lattices or codes. Many signature applications—such as root certificates, secure software updates, and authentication protocols—involve long-lived public keys that are transferred or installed once and then used for many verifications. This key longevity makes post-quantum signature schemes with conservative assumptions (e.g., structure-free lattices) attractive for long-term security. But many such schemes, especially those with short signatures, suffer from extremely large public keys. Even in scenarios where bandwidth is not a major concern, large keys increase storage costs and slow down verification. As such, even though lattice-based cryptography is nowadays at the center of post-quantum cryptography, it mainly focuses on cryptosystems relying on

structured lattices. Keeping a focus on alternatives such as the GPV framework remains necessary. Cryptography needs to be versatile in case new attacks make some families of cryptosystems compromised. We address this with a method to replace large public keys in GPV-style signatures with smaller, private verification keys. This significantly reduces verifier storage and runtime while preserving security. Applied to the conservative, short-signature schemes Wave and Squirrels, our method compresses Squirrels-I keys from 665 kB to 20.7 kB and Wave822 keys from 3.5 MB to 207.97 kB.

Our second contribution, in **Chapter 4**, consists of an article published at Latincrypt 2023: *Fast and Frobenius: Rational Isogeny Evaluation over Finite Fields* (joint work with Gustavo Banegas, Valerie Gilchrist and Benjamin Smith). We provide a speedup in isogeny computation. Consider the problem of efficiently evaluating isogenies $\varphi : E \rightarrow E/H$ of elliptic curves over a finite field $\mathbb{F}_q$, where the kernel $H = \langle G \rangle$ is a cyclic group of odd (prime) order: given E , G , and a point (or several points) P on E , we want to compute $\varphi(P)$. This problem is at the heart of implementations of group-action- and isogeny-based post-quantum cryptosystems such as CSIDH. Algorithms based on Vélú’s formulæ give an efficient solution when the kernel generator G is defined over $\mathbb{F}_q$. But for general isogenies, G is only defined over some extension $\mathbb{F}_{q^k}$, even though $\langle G \rangle$ as a whole (and thus φ) is defined over the base field $\mathbb{F}_q$. The performance of isogeny evaluation algorithms degrades rapidly as k grows. In this chapter, we revisit this with a special focus on the case where $1 \leq k \leq 12$. Our techniques speed up commutative isogeny-based schemes such as CSIDH [20] and CSI-FiSh [13]. They also speed up isogenies evaluation in the Couveignes-Rostovtsev-Stolbunov cryptosystem (CRS), which is a first step towards making it more practical so that it could be a fallback using ordinary curves for CSIDH.

Our third contribution, in **Chapters 5 and 6**, takes its root in more classical parallel collision search algorithms. We propose classical algorithms for multi-dimensional restricted discrete logarithms and hidden shifts. Our work consists of a first step towards the Gaudry–Schost algorithm in dimensions greater than 4. This has an application in group-action-based cryptography, more specifically, it applies to *Restricted Effective Group Actions (REGA)*. When looking at the security of REGA-based cryptosystems, cryptographers look at the complexity of a parallel collision search [118] on the whole space to make their estimations. However, depending on the choice of parameters, REGA-based cryptosystems can fall prey to some parallel collision search targeted on specific subsets. To avoid this risk, we need to choose appropriate parameter sets. The main bottleneck is a lack of understanding and research on attacks taking advantage of the bounds in a REGA. The only known algorithms to tackle this issue are a recent low-memory attack for small bounds [26] as well as the Gaudry–Schost algorithm and

its derivatives [56, 59] which we extend to high dimensions. We analyse the complexity of our algorithm in two ways. The first one lays the groundwork for a general analysis (continued in **Chapter 6**) drawing on techniques of Gaudry and Schost. The second one is conducted as an application of the van Oorschot–Wiener parallel collision search algorithm. Finally, we compare our results with a proof-of-concept implementation.

Chapter 2

Background

2.1 Group actions

In this section, we introduce the concept of group actions. This concept is core to most of the work covered in Chapters 4, 5 and 6. Group-action is a general theoretical framework used for constructing cryptosystems. Most of this section content takes root in the work of Couveignes on Hard Homogeneous Spaces [33], and the work of [3] introducing cryptographic group actions. It is motivated by modern cryptography using group actions to build several applications.

Definition 2.1.1 (Group action) *A group action of a group G on a set X is a map of the form $\star : G \times X \rightarrow X$ satisfying the following properties:*

- $g_1 \star (g_2 \star x) = (g_1 g_2) \star x$, for all $g_1, g_2 \in G, x \in X$, and
- $1 \star x = x$, for all $x \in X$.

When there are no confusions possible with another operation (like the group operation for example), we might loosely write $g \star x$ by gx . Let G a group acting on X . We can define, for each fixed $g \in G$, the map σ_g by

$$\begin{aligned}\sigma_g : X &\rightarrow X \\ \sigma_g(x) &= g \star x.\end{aligned}$$

The map σ_g is actually a permutation of X . Moreover for any two elements $g_1, g_2 \in G$ we have $\sigma_{g_1.g_2} = \sigma_{g_1} \circ \sigma_{g_2}$.

Example 2.1.2 (Group acting on itself) *A group G acts on itself through left multiplication. We set $X = G$ and the action to be $g_1 \star g = g_1 g$ for $g_1, g \in G$. The two axioms of a group action are fulfilled with 1 being the identity element in G and because $g_1(g_2 g) = (g_1 g_2)g$ for $g_1, g_2, g \in G$.*

Example 2.1.3 (Conjugation action) *A group G acts on itself through conjugation. We define the group action $\star$ by $g \star x = gxg^{-1}$ for $g, x \in G$.*

Example 2.1.4 (Matrix action) *Let $G = GL_n(\mathbb{F})$, the group of $n \times n$ invertible matrices over some field $\mathbb{F}$. This group acts on the vector space $\mathbb{F}^n$ through matrix-vector left multiplication. Matrix action on a vector space represents how the whole space transforms under linear operations.*

Group actions can have various hardness properties in the cryptographic sense, i.e., some problems are *hard* while some others are *easy*. We mean by hardness that a problem can not be solved efficiently in polynomial time. Hardness properties help us to build different cryptographic frameworks for constructing protocols.

Let X be a set and G a group. Let's first cover the problems that are required to be easy or hard to have what we would call an *Efficient Group Action (EGA, see Section 2.2.1)*. The following should be easy i.e., there exist polynomial algorithms that solve them.

2.1.5 (Membership) *Given an object x , decide if $x \in X$.*

2.1.6 (Equality) *Given two elements x_1 and x_2 in X , decide whether $x_1 = x_2$.*

2.1.7 (Random sampling) *Find uniformly at random an element in X .*

2.1.8 (Group operation) *Given $g_1, g_2 \in G$, compute $g_1 g_2$.*

2.1.9 (Group operation inverse) *Given $g \in G$, compute g^{-1} .*

2.1.10 (Action) *Given $g \in G$ and $x \in X$, compute $g \star x$.*

The following hard problems are what will provide security for cryptographic constructions based on group action.

Problem 2.1.11 (Vectorization) *Given $x_1, x_2 \in X$, compute some $g \in G$ (if it exists) such that $g \star x_1 = x_2$.*

Problem 2.1.12 (Parallelization) *Given $x, g_0 \star x, g_1 \star x \in X$ for some $g_0, g_1 \in G$, compute $(g_0 g_1) \star x$.*

Remark 2.1.13 *In cryptographic applications, the vectorization problem is an analogue of the Discrete Logarithm Problem. In the same way, the parallelization problem corresponds to the Computational Diffie–Hellman Problem.*

Special case of group action: Hard Homogeneous Spaces

Historically, attention was given to Hard Homogeneous Spaces which is a subcase of group action. It was brought up by Couveignes [33] to get a hard instance of the vectorization problem. It has later been considered as a framework to build post-quantum secure protocols.

Definition 2.1.14 (Homogeneous Space) *Let G be a finite commutative group. Let H be homogeneous space for G . A homogeneous space H for G is a non-empty finite set acted on simply transitively by G . I.e $\forall x, y \in H$, there is a unique $g \in G$ such that $g \star x = y$.*

This means that for any two elements in H , there is a unique element in G that maps them together. Moreover, we get that $\#H = \#G$. Hard homogeneous spaces are a subcase of homogeneous spaces where some problems are *hard* while some others are *easy*.

2.1.1 Hardness

In this section we will dive into the hardness of *Vectorization*, *Parallelization* and *Hidden shift* problems. These problems are cryptographically relevant because, as we will see in this section, they offer quantum resistance.

Classical hardness of vectorization

Classical hardness of the vectorization problem relies on Pollard’s Rho algorithm [96] (probabilistic) or Baby-Step Giant-Step (BSGS) algorithm (space-time trade-off) [107]. Note that these two algorithms were designed to tackle the discrete logarithm problem. However they can be easily adapted to group action [79]. This gives immediate square-root upper bounds on hardness. And no better generic algorithm is currently known.

Quantum hardness of vectorization

Breaking the abelian hidden shift problem reduces to breaking the vectorization problem which means vectorization is solved by hidden shifts algorithms. Note that the other way around is not proven.

Problem 2.1.15 (Abelian Hidden Shift) *Let $(G, +)$ be a finite abelian group and X a set. Given an oracle access to two injective functions $f_1, f_2 : G \rightarrow X$, for which there exist an element $s \in G$ such that $f_1(g) = f_2(g + s)$ for all $g \in G$. We also require that all of the $|G|$ outputs of f_1 and f_2 are distinct. Find s .*

A famous result is that Kuperberg’s algorithm [68, 69] solves Problem 2.1.15 in quantum sub-exponential time, using $2^{O(\sqrt{\log |G|})}$ operations for any finite abelian group G . Kuperberg’s algorithm is essentially a black-box quantum algorithm: it presupposes access to a quantum circuit implementing the group action and invokes it a subexponential number of times. To obtain a concrete subexponential attack for a specific group action, one must design that circuit explicitly. If implementing the action itself is costly—e.g., exponential—then this dominates and the overall attack is no longer subexponential.

In the case of isogeny-based group action, Bernstein, Lange, Martindale and Panny have explored the cost of every query made when attacking CSIDH with Kuperberg’s algorithm [11].

Quantum equivalence of vectorization and parallelization

In a classical setting, Problems 2.1.11 and 2.1.12 are not equivalent. If the *parallelization* problem straightforwardly reduces to the *vectorization* problem, the other way do not apply in the absence of quantum adversaries.

However, in the quantum setting it has been shown by Galbraith, Panny, Smith and Vercauteren [55] and by Montgomery and Zhandry [79] that there is a full quantum equivalence between those two problems.

Shor’s algorithm

Shor’s algorithm [109] is a famous algorithm that solves the integer factorization problem and the discrete logarithm problem in abelian groups. However, this algorithm does not apply to the group action setting as it does not solve general instances of the hidden shift problem. It only applies to the case where $f_1 = f_2$.

2.1.2 Some examples

Example 2.1.16 Let V be a vector space over a field $\mathbb{F}$, and A the underlying affine space of V . The action of V on A is given by:

$$v \star a = a + v \quad \text{for } v \in V \text{ and } a \in A.$$

The vectorization problem in this case is to find $v \in V$ such that $v \star a = b$, which simplifies to:

$$v = b - a.$$

Vectorization here is trivial as it is just group operations that were assumed to be easy. This is a special case of a group acting on itself.

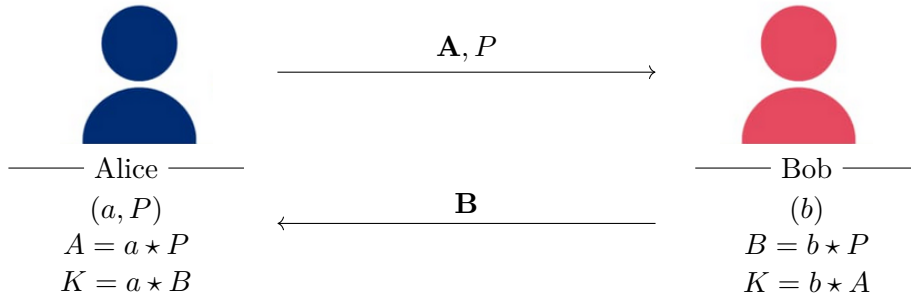
Example 2.1.17 Let $G = \mathbb{F}_q^*$ be a multiplicative prime-order sub-group of a finite field $\mathbb{F}_q$, and let $a \in G$ be an element. The group action is defined by exponentiation, where the action of an exponent $x \in (\mathbb{Z}/N\mathbb{Z})^\times$ where $N = \#G$ on a is:

$$x \star a = a^x.$$

Vectorization in this setting is the Discrete Logarithm Problem in $\langle g \rangle \subseteq \mathbb{F}_q^\times$. The DLP in this setting is subexponential in $\log q$ [62]. However, quantum algorithms like Shor's Algorithm can solve the DLP in polynomial time, making it quantum easy.

2.2 Cryptographic group actions

We first focus on the Diffie–Hellman protocol. In cryptography, this protocol answers a need for key distribution. A key-exchange allows two parties (usually named Alice and Bob in the literature) to agree on a secret key over an insecure communication channel without having communicated before. We consider a set X equipped with a group action of an **abelian** group G . Alice and Bob both have an ephemeral secret key $a, b \in G$. $P \in X$ is a system parameter.



Since G is an abelian group, we have $K = a \star (b \star P) = b \star (a \star P) = (ab) \star P$. This protocol shows incredible simplicity and flexibility which has led to it being a foundation for public-key cryptography. The security of the public key depends on the hardness of the vectorization problem. The security of the shared secret relies on the hardness of the parallelization problem. In Subsection 2.5, we investigate choices of group actions so that the Diffie–Hellman protocol is quantum resistant.

Example 2.2.1 (X25519) *The state of the art for key exchange is X25519, a highly efficient version of elliptic-curve Diffie–Hellman. But Shor’s algorithm also breaks this, which makes it unsuitable for post-quantum cryptography.*

2.2.1 EGA and REGA

In Section 2.1, we have introduced group actions. Depending on the hardness of the introduced problems, some group actions can be cryptographically relevant. The presentation in this section largely follows [3]; definitions are adapted with minor changes to notation and ordering. The standard notion for cryptographic group action is *effective group action* (EGA).

Definition 2.2.2 (EGA) *An effective group action $(G, X, \star)$ has the following properties:*

- *The group G is finite and abelian. The following problems in G are easy i.e., there exist polynomial-time algorithms that solves them:*
 - *Membership, 2.1.5.*
 - *Equality, 2.1.6.*
 - *Random sampling, 2.1.7.*
 - *Group operation, 2.1.8.*
 - *Group inversion, 2.1.9.*
- *The set X is finite. The following problem in X is easy i.e., there exist a polynomial-time algorithm that solves:*
 - *Membership, 2.1.5.*
- *There exist an element $P \in X$ which representation is known.*
- *There exist an efficient algorithm to compute the group action, 2.1.10.*

In all applications that will follow, we fit in the work by Couveignes and are interested in homogeneous spaces (see 2.1) which are *hard*. Find below a reformulation of HHS, already introduced in subsection 2.1, using EGA. In reality, it can be complicated to build such an abstract notion. That is why we introduce a weakened version of EGA: Restricted Effective Group Action (REGA).

Definition 2.2.3 (Hard Homogeneous Spaces) *Hard homogeneous spaces are homogeneous spaces which are equipped with EGA. Furthermore, the following two problems are difficult i.e., there doesn't exist probabilistic polynomial-time algorithms to solve them:*

- Vectorization, Problem 2.1.11,
- Parallelization, Problem 2.1.12.

Definition 2.2.4 (Restricted Effective Group Action) *Let $(G, X, \star)$ be a group action and let $g = (g_1, \dots, g_n)$ be a (not necessarily minimal) generating set for G . The action is said to be g -restricted effective, if the following properties are satisfied:*

- G is finite and $n = \text{poly}(\log(|G|))$.
- The set X is finite. The following problem in X is easy i.e., there exist a polynomial-time algorithm that solves:
 - Membership, 2.1.5.
- There exists an efficient algorithm that given any $i \in [n]$ and any bit string representation of $P \in X$, outputs $g_i \star P$ and $g_i^{-1} \star P$.

2.3 Elliptic Curves

The presentation in this section largely follows the book published by Joseph H. Silverman on *the arithmetic of elliptic curves* [110], *the handbook of finite fields* by Gary Mullen and Daniel Panario [81] and [48]; definitions are adapted with minor changes to notation and ordering.

2.3.1 Finite fields and Galois theory

Theorem 2.3.1 ([81, Theorem 2.1.12]) *Let $\mathbb{F}$ be a finite field. The cardinality of $\mathbb{F}$ is p^n where p is the characteristic of $\mathbb{F}$ and n is the dimension of $\mathbb{F}$ over its prime subfield.*

A finite field of a given order is unique up to field isomorphism, as such we can talk about *the* finite field of a particular order q . When the order p of a finite field $\mathbb{F}$ is prime, then $\mathbb{F}$ is the same as (isomorphic to) the ring $\mathbb{Z}/p\mathbb{Z}$ of integers modulo p . However, that is not the case for non-prime orders.

Every finite field is a vector space over each of its subfields. Let's consider a finite field $\mathbb{F}_{p^n}$, where p is prime and $n > 1$. Then $\mathbb{F}_{p^n}$ is a field extension and a vector space over $\mathbb{F}_p$ of dimension n . We know there are many bases for this vector space. One common way of constructing $\mathbb{F}_{p^n}$ is to define the field using an irreducible polynomial of degree n over $\mathbb{F}_p$. Let $f(X)$ be such a polynomial. We can construct $\mathbb{F}_{p^n}$ by taking the quotient ring

$$\mathbb{F}_{p^n} \cong \mathbb{F}_p[X]/(f(X))$$

In Chapter 4, we use Galois orbits. As such we introduce the basics needed to introduce this definition. Let K be a field. We use the notation $\text{Aut}(K)$ to refer to the collection of automorphisms of K . We now consider the case where K is an extension field of a field $\mathbb{F}$.

Definition 2.3.2 (Galois group of $K/\mathbb{F}$: $\text{Gal}(K/\mathbb{F})$) *Let $K/\mathbb{F}$ be an extension field. We note $\text{Aut}(K/\mathbb{F})$ to be the collection of automorphisms of K which fix $\mathbb{F}$ i.e., the image of every element in $\mathbb{F}$ is itself. We have $\text{Gal}(K/\mathbb{F}) = \text{Aut}(K/\mathbb{F})$.*

Definition 2.3.3 *Let $K/\mathbb{F}$ be a Galois extension. We call the conjugates (or Galois conjugates) of an element $\alpha \in K$ over $\mathbb{F}$ all the elements $\sigma(\alpha)$ for σ in $\text{Gal}(K/\mathbb{F})$.*

Definition 2.3.4 (Frobenius automorphism π_p) *Let p be a prime and n a positive integer. We define the Frobenius automorphism by*

$$\begin{aligned} \pi_p : \mathbb{F}_{p^n} &\rightarrow \mathbb{F}_{p^n} \\ \alpha &\mapsto \alpha^p \end{aligned}$$

Let's consider the case where $K = \mathbb{F}_{p^n}$ for a prime p . Then $\mathbb{F}_{p^n}$ is Galois over $\mathbb{F}_p$. In this case, the Galois group is cyclic of order n generated by the Frobenius automorphism,

$$\text{Gal}(\mathbb{F}_{p^n}/\mathbb{F}_p) = \langle \pi_p \rangle \cong \mathbb{Z}/n\mathbb{Z}.$$

For every divisor d of n , there is precisely one subfield of $\mathbb{F}_{p^n}$ of degree d over $\mathbb{F}_p$. That subfield is isomorphic to $\mathbb{F}_d$ and is the fixed field given by π_p^d of order n/d . The Galois group $\text{Gal}(\mathbb{F}_{p^d}/\mathbb{F}_p)$ is generated by the image of π_p in the quotient group $\text{Gal}(\mathbb{F}_{p^n}/\mathbb{F}_p)/\langle \pi_p^d \rangle$.

Definition 2.3.5 (Galois orbit) Let q be an integer. The Galois orbit of an element $\alpha \in \text{Gal}(\mathbb{F}_{q^n}/\mathbb{F}_q)$ corresponds to the conjugates of α .

Example 2.3.6 Let's consider $\mathbb{F}_4$. It has $4 = 2^2$ elements. Here we have $p = 2$ and $n = 2$. We take $f(X) = X^2 + X + 1$. Elements of $\mathbb{F}_4$ are all polynomials in $\mathbb{F}_2[X]$ modulo $f(X)$. We get $\mathbb{F}_4 = \{0, 1, X, X + 1\}$. All calculations in the finite field $\mathbb{F}_4$ are then performed modulo $f(X)$.

Now, the Galois group $\text{Gal}(\mathbb{F}_4/\mathbb{F}_2)$ is generated by the Frobenius automorphism π_2 , where $\pi_2(\alpha) = \alpha^2$ for any element α in $\mathbb{F}_4$. For any element $\alpha \in \mathbb{F}_4$ not in $\mathbb{F}_2$, the Galois orbit of α by Frobenius is: $\{\alpha, \alpha^2\}$. More precisely, the set of elements of $\mathbb{F}_2(\alpha)$ decomposes into the disjoint union of Galois orbits:

$$\{0\} \cup \{1\} \cup \{\alpha, \alpha^2\}$$

Example 2.3.7 Let's consider the polynomial $f(X) = (X - 3)(X^2 + 1)(X^3 - 2) \in \mathbb{F}_7[X]$. Let $i \in \mathbb{F}_{7^2}$ such that $i^2 = -1$ and $\alpha \in \mathbb{F}_{7^3}$ such that $\alpha^3 = 2$. Collecting the roots of the polynomial f gives us Galois orbits in the Galois group $\text{Gal}(\mathbb{F}_{7^6}/\mathbb{F}_7)$: $\{3\} \cup \{i, -i\} \cup \{\alpha, \alpha^7, \alpha^{7^2}\}$.

2.3.2 Elliptic Curves

We now focus on elliptic curves. These objects set the stage for our later isogeny constructions. Elliptic curves have been studied for a long time and are of interest from various mathematical perspectives. We refer the reader to *The Arithmetic of Elliptic Curves* by J. Silverman [110] for more information than the concise presentation provided here. We want to briefly introduce the arithmetic properties of these curves. As such, we will analyse them when defined over arithmetically interesting fields such as a finite field K .

Definition and representations

Definition 2.3.8 (Elliptic curve) An elliptic curve is a pair $(\mathcal{E}, O)$, where $\mathcal{E}$ is a nonsingular algebraic curve of genus one and $O \in \mathcal{E}$. The elliptic curve $\mathcal{E}$ is defined over K , written $\mathcal{E}/K$, if $\mathcal{E}$ is defined over K as a curve and $O \in \mathcal{E}(K)$.

Definition 2.3.9 (Affine space) The affine n -space over $\overline{K}$ is the set of n -tuples

$$\mathbb{A}^n = \{P = (x_1, \dots, x_n) : x_i \in \overline{K}\}.$$

Definition 2.3.10 (Projective Space $\mathbb{P}$) A projective n -space (over $\bar{K}$), denoted as $\mathbb{P}^n$ is the set of all $(n+1)$ -tuples

$$(x_0, \dots, x_n) \in \mathbb{A}^{n+1}$$

such that at least one x_i is nonzero, modulo the equivalence relation:

$$(x_0, \dots, x_n) \sim (y_0, \dots, y_n)$$

if there exists a $\lambda \in \bar{K}^*$ such that $x_i = \lambda y_i$ for all i . For an equivalence class $[(x_0, \dots, x_n)]$, the individuals $x_0, \dots, x_n$ are called homogeneous coordinates.

The Riemann–Roch Theorem [110] show that every elliptic curve can be written as a plane cubic, and conversely, every smooth projective plane cubic curve is an elliptic curve.

Definition 2.3.11 (Simplified Weierstrass equation) Let $\mathcal{E}$ be an elliptic curve defined over K . We introduce the Weierstrass equation, it is a standard theorem [110] that all elliptic curves are isomorphic to a plane curve of the following form. When $\text{char}(\bar{K}) \neq 2, 3$, the Weierstrass equation can be simplified. Let's first introduce this case.

$$\mathcal{E} : y^2 = x^3 + ax + b.$$

In this setting, we get the discriminant and j -invariant of the elliptic curve to be

$$\Delta = -16(4a^3 + 27b^2)$$

$$\text{and } j = -1728 \frac{(4a)^3}{\Delta}.$$

We will mostly focus on simplified Weierstrass equations. However, for correctness, we also define here the definition of the Weierstrass equation for any characteristic.

Definition 2.3.12 (Weierstrass equation) There exist functions $x, y \in K(\mathcal{E})$ (field of rational functions on $\mathcal{E}$ over K) such that the Zariski closure of the image of the map

$$\phi : \mathcal{E} \rightarrow \mathbb{P}^2, \quad \phi = [(x, y, 1)],$$

is defined by an equation of the form

$$y^2 + a_1xy + a_3y = x^3 + a_2x^2 + a_4x + a_6 \quad \text{with } a_1, a_2, a_3, a_4, a_6 \in K.$$

If the characteristic of $\bar{K}$ is not equal to 2, then we can simplify the equation of $\mathcal{E}$ to

$$\mathcal{E} : y^2 = 4x^3 + b_2x^2 + 2b_4x + b_6,$$

where

$$b_2 = a_1^2 + 4a_4, \quad b_4 = 2a_4 + a_1a_3, \quad b_6 = a_3^2 + 4a_6.$$

We define

$$\begin{aligned} b_8 &= a_1^2a_6 + 4a_2a_6 - a_1a_3a_4 + a_2a_3^2 - a_4^2, \\ c_4 &= b_2^2 - 24b_4, \\ c_6 &= -b_2^3 + 36b_2b_4 - 216b_6. \end{aligned}$$

Definition 2.3.13 (Discriminant) *The discriminant of the Weierstrass equation is given by*

$$\Delta = -b_2^2b_8 - 8b_4^3 - 27b_6^2 + 9b_2b_4b_6.$$

Definition 2.3.14 (j -invariant) *The j -invariant of the elliptic curve $\mathcal{E}$ is given by*

$$j = c_4^3/\Delta.$$

The group law

Let $\mathcal{E}/K$ be an elliptic curve defined by a simplified Weierstrass equation over a field K , and let L/K be an extension field. The set of points $\mathcal{E}(L)$ forms an abelian group using the following rules:

- The point O is the identity element. We use the convention that O is on every vertical line.
- The negative point of $P = (x, y)$ is the point $-P = (x, -y - a_1x - a_3)$. If $\mathcal{E}$ is given by a simpler Weierstrass equation then $-P = (x, -y)$ which is the reflection P about the x -axis.
- The sum of distinct points P and Q is obtained by intersecting the line through P and Q with $\mathcal{E}$. This yields three points P, Q and R . Then $P + Q$ equals $-R$.
- The sum of P with itself is obtained similarly, using the tangent line to $\mathcal{E}$ at P .

Using this group law, we can define *scalar multiplication* (multiplication-by- k) endomorphisms on $\mathcal{E}$ defined by:

$$\begin{aligned} [k] : \mathcal{E} &\rightarrow \mathcal{E} \\ P &\mapsto \underbrace{P + P + \cdots + P}_{k \text{ times}}. \end{aligned}$$

We also set $[-k] = [k](-P)$. We call $\mathbb{P}^1$ the x -line of $\mathcal{E}$. $x(P) = x(Q)$ if and only if $P = Q$ or $P = -Q$. For the sake of different applications, we are interested in defining a class of curves $\mathcal{E}$ which provides efficient algorithms to compute $x(P) \mapsto x([k]P)$ on $\mathbb{P}^1$ (see Section 2.3.3).

Theorem 2.3.15 (Hasse's Theorem) *Let $\mathcal{E}/\mathbb{F}_q$ be an elliptic curve defined over a finite field. Then*

$$|\#\mathcal{E}(\mathbb{F}_q) - q - 1| \leq 2\sqrt{q}.$$

Definition 2.3.16 (Torsion subgroup) *Let $\mathcal{E}/K$ be an elliptic curve and let $m \in \mathbb{Z}$ with $m \geq 1$. The m -torsion subgroup of $\mathcal{E}$, denoted by $\mathcal{E}[m]$, is the set of points of $\mathcal{E}$ whose order divides m ,*

$$\mathcal{E}[m] = \ker[m] = \{ P \in \mathcal{E}(\overline{K}) : [m]P = \mathcal{O} \}.$$

ECDLP

The subcase of the vectorization problem applied to the group $G = \mathbb{Z}$ and $X = \mathcal{E}(\mathbb{F}_q)$ is also called *Elliptic Curve Discrete Logarithm Problem*. The key is that scalar multiplication is a group action (see Section 2.1). Note that this problem is not quantum resistant. It falls prey to Shor's algorithm.

Problem 2.3.17 (ECDLP) *Given an elliptic curve $\mathcal{E}$ and two points $P, Q \in \mathcal{E}(\mathbb{F}_q)$, find an integer m such that*

$$[m]P = Q.$$

2.3.3 Montgomery curves and Montgomery ladder

The presentation in this section largely follows [32]; definitions are adapted with minor changes to notation and ordering. Here we study a sub-case of the general Weierstrass

equation. The Montgomery model is of interest primarily for its efficient arithmetic operations. It provides an efficient way to perform elliptic-curve computations, in particular point addition and doubling.

Definition 2.3.18 (Montgomery curve) *A Montgomery curve over $\mathbb{F}_q$ is an elliptic curve defined by an affine equation*

$$\mathcal{E}_{(A,B)} : By^2 = x(x^2 + Ax + 1),$$

where A and B are parameters in $\mathbb{F}_q$ satisfying $B \neq 0$ and $A^2 \neq 4$.

The parameter A controls the geometry of the curve as the j -invariant of $\mathcal{E}_{(A,B)}$ is

$$j(\mathcal{E}_{(A,B)}) = \frac{256(A^2 - 3)^3}{A^2 - 4}.$$

Remark 2.3.19 *Not all elliptic curves can be written in the Montgomery model. However, we note that an $\mathcal{E}$ in the short Weierstrass model has a Montgomery model if and only if there exist α and β in $\mathbb{F}_q$ such that $\alpha^3 + a\alpha + b = 0$ and $3\alpha^2 + a = \beta^2$. [90]*

Example 2.3.20 (Curve25519) *The most widely-known Montgomery curve in contemporary cryptography is the curve used in Bernstein's Curve25519 software for Diffie-Hellman key exchange. This curve is defined by*

$$\mathcal{E}/\mathbb{F}_p : y^2 = x(x^2 + 486662x + 1) \quad \text{where } p = 2^{255} - 19.$$

$\#\mathcal{E}(\mathbb{F}_p) = 8r$, where r is a 253-bit prime.

The group law

The additive group law is given by the following formulae. We consider the addition of two points $P = (x_P, y_P)$ and $Q = (x_Q, y_Q)$ on $\mathcal{E}_{(A,B)}$.

$$\begin{aligned} x_{(P+Q)} &= B\lambda^2 - (x_P + x_Q) - A \\ \text{and } y_{(P+Q)} &= \lambda(x_P - x_{(P+Q)}) - y_P, \\ \text{where} \quad \lambda &= \begin{cases} (y_Q - y_P)/(x_Q - x_P) & \text{if } P \neq Q; -Q, \\ (3x_P^2 + 2Ax_P + 1)/(2By_P) & \text{if } P = Q. \end{cases} \end{aligned}$$

Optimized $xADD$ and $xDBL$

To simplify computations, we use projective coordinates $(X : Y : Z)$, which are related to the affine coordinates (x, y) by:

$$X = xZ, \quad Y = yZ, \quad Z = Z.$$

The Montgomery curve equation in projective coordinates becomes:

$$BY^2Z = X^3 + AX^2Z + XZ^2. \quad (2.1)$$

The point at infinity (the identity element) has projective coordinates $O = (0 : 1 : 0)$. In the following algorithms we use projective coordinates in a compact form where $x((X : Y : Z)) = (X : Z)$ ($Z \neq 0$). We use $x(O) = (1 : 0)$. The last Y coordinate can be derived from the Montgomery curve equation in projective coordinates. However $x(P)$ determines P only up to sign, that is why we need three of the values $\{x(P), x(Q), x(P+Q), x(P-Q)\}$ to determine the fourth. We define *pseudo-addition* on $\mathbb{P}$:

$$xADD : (x(P), x(Q), x(P-Q)) \mapsto x(P+Q),$$

and *point-doubling*

$$xDBL : x(P) \mapsto x([2]P).$$

The Montgomery model gives simple identities between the x -coordinates of those different elements. As such, $xADD$ ($P \neq Q$) is obtained by using the simultaneous relations:

$$\begin{aligned} X_{(P+Q)} &= Z_{(P-Q)}[(X_P - Z_P)(X_Q + Z_Q) + (X_P + Z_P)(X_Q - Z_Q)]^2, \\ Z_{(P+Q)} &= X_{(P-Q)}[(X_P - Z_P)(X_Q + Z_Q) - (X_P + Z_P)(X_Q - Z_Q)]^2. \end{aligned}$$

Similarly, we get relations for the case $Q = P$ and use those to compute $xDBL$:

$$\begin{aligned} X_{[2]P} &= (X_P + Z_P)^2(X_P - Z_P)^2, \\ Z_{[2]P} &= (4X_P Z_P)((X_P - Z_P)^2 + ((A+2)/4)(4X_P Z_P)). \end{aligned}$$

These operations are much more efficient than classic x -line arithmetic in Weierstrass model and are thus widely used in cryptographic applications.

Special torsion and group structures

From the Montgomery curve equation (2.1) in projective coordinates, we note immediately that $\mathcal{E}_{(A,B)}$ always has a rational point of order two,

$$T := (0 : 0 : 1) \in \mathcal{E}_{(A,B)}[2](\mathbb{F}_q)$$

This point is quite special on Montgomery curves because the translation by T map induces $(X : Y) \mapsto (Z : X)$ on $\mathbb{P}^1$. Furthermore, a Montgomery curve never has prime order. On the contrary, we always have $\#\mathcal{E}_{(A,B)}(\mathbb{F}_q)$ divisible by 4.

The Montgomery ladder

The Montgomery ladder is about calculating scalar multiplication in terms of the x -line coordinate i.e., $x(P) \mapsto x([k]P)$. One possible way of doing this is to actually compute the whole scalar multiplication on $\mathcal{E}_{(A,B)}$. Algorithm 1, known as the Montgomery ladder [32, Section 4.2], does this using only $xADD$ and $xDBL$. Note that we can recover the y -coordinates from $(P, x([k]P), x([k+1]P))$ [74].

Algorithm 1 LADDER: The Montgomery ladder

- 1: **Input:** $k = \sum_{i=0}^{\ell-1} k_i 2^i$ with $k_{\ell-1} = 1$, and $(X_P, Z_P) \in \mathbb{F}_q^2$ s.t. $(X_P : Z_P) = x(P)$
 - 2: **Output:** $(X_k, Z_k) \in \mathbb{F}_q^2$ s.t. $(X_k : Z_k) = x([k]P)$ if $P \notin \{O, T\}$
 - 3: Also optionally returns $(X_{k+1}, Z_{k+1}) \in \mathbb{F}_q^2$ s.t. $(X_{k+1} : Z_{k+1}) = x([k+1]P)$ if $P \notin \{O, T\}$
 - 4: **Cost:** $\ell - 1$ calls to $xADD$ and ℓ calls to $xDBL$
 - 5: $(x_0, x_1) \leftarrow ((X_P, Z_P), xDBL((X_P, Z_P)))$
 - 6: **for** $i = \ell - 2$ **down to** 0 **do**
 - if** $k_i = 0$ **then**
 - $(x_0, x_1) \leftarrow (xDBL(x_0), xADD(x_0, x_1, (X_P, Z_P)))$
 - else**
 - $(x_0, x_1) \leftarrow (xADD(x_0, x_1, (X_P, Z_P)), xDBL(x_1))$
 - 7: **return** x_0 (and optionally x_1)
-

In Algorithm 1, the difference between x_1 and x_0 is always $x(P)$. This allows us to perform the operations $xADD$ and $xDBL$. After iteration i in the loop, we always have $(x_0, x_1) = (x([\sum_{j=i}^{\ell-1} k_j 2^{j-i}]P), x([\sum_{j=i}^{\ell-1} k_j 2^{j-i} + 1]P))$. As such, depending on the value of k_{i+1} , at the next iteration, we will either multiply x_0 by 2 ($k_{i-1} = 0$) or add x_1 to x_0 ($k_{i-1} = 1$) which results into multiplying by 2 and adding one P but using only one $xADD$. We always update x_1 accordingly so that the relation $x_1 - x_0 = x(P)$ holds.

2.4 Isogenies

The presentation in this section largely follows the book published by Joseph H. Silverman on *the arithmetic of elliptic curves* [110]; definitions are adapted with minor changes to notation and ordering. Consider the set of elliptic curves defined over a finite field K . We investigate rational maps between those objects. Since an elliptic curve has a distinguished zero point, it is natural to single out maps that respect this property.

Definition 2.4.1 (Isogeny) *Let $\mathcal{E}_1$ and $\mathcal{E}_2$ be elliptic curves. An isogeny from $\mathcal{E}_1$ to $\mathcal{E}_2$ is a morphism*

$$\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2 \text{ satisfying } \phi(O) = O.$$

Two elliptic curves are isogenous if there is a nonzero isogeny between them.

Theorem 2.4.2 ([110, Chapter III.4]) *Let $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ be an isogeny between elliptic curves defined over K . Then*

$$\phi(P + Q) = \phi(P) + \phi(Q) \text{ for all } P, Q \in \mathcal{E}_1.$$

Elliptic curves are abelian groups, the maps between them i.e., isogenies also form a group. We denote the set of isogenies from $\mathcal{E}_1$ to $\mathcal{E}_2$ by $\text{Hom}_K(\mathcal{E}_1, \mathcal{E}_2)$.

2.4.1 The endomorphism ring

We consider the case where $\mathcal{E}_1 = \mathcal{E}_2 = \mathcal{E}$. Then, we can compose isogenies over K and we let

$$\text{End}_K(\mathcal{E}) = \text{Hom}_K(\mathcal{E}, \mathcal{E}).$$

$\text{End}(\mathcal{E})$ is a ring whose addition law is given by Theorem 2.4.2 and multiplication is the composition $(\phi\varphi)(P) = \phi(\varphi(P))$. We call it *the endomorphism ring of $\mathcal{E}$* . The invertible elements of $\mathcal{E}$ are automorphisms and form the group $\text{Aut}(\mathcal{E})$. We will give a general classification of the endomorphism ring.

Definition 2.4.3 (Order) *Let $\mathcal{K}$ be a (not necessarily commutative) $\mathbb{Q}$ -algebra that is finitely generated over $\mathbb{Q}$. An order $\mathcal{R}$ of $\mathcal{K}$ is a subring of $\mathcal{K}$ that is finitely generated as a $\mathbb{Z}$ -module and satisfies*

$$\mathcal{R} \otimes \mathbb{Q} = \mathcal{K}.$$

Definition 2.4.4 (Quaternion algebra) *A quaternion algebra is an algebra of the form*

$$\mathcal{K} = \mathbb{Q} + \mathbb{Q}\alpha + \mathbb{Q}\beta + \mathbb{Q}\alpha\beta$$

whose multiplication satisfies

$$\alpha^2, \beta^2 \in \mathbb{Q}, \quad \alpha^2 < 0, \quad \beta^2 < 0, \quad \beta\alpha = -\alpha\beta.$$

Theorem 2.4.5 ([110, Theorem III.9.3]) *Let $\mathcal{E}$ be an elliptic curve defined over K . Then $\text{End}(\mathcal{E})$ has one of the following forms:*

- $\text{End}(\mathcal{E}) \cong \mathbb{Z}$ (only if $\text{char}(K) = 0$).
- $\text{End}(\mathcal{E})$ is an order in an imaginary quadratic extension of $\mathbb{Q}$.
- $\text{End}(\mathcal{E})$ is an order in a quaternion algebra over $\mathbb{Q}$.

The main problem behind isogeny-based cryptography is the isogeny problem. This problem is considered to be both classically and quantumly hard.

Problem 2.4.6 (Isogeny Problem) *Let E_1 and E_2 be two isogenous elliptic curves defined over a finite field $\mathbb{F}_q$, with $\text{End}(E_1) \cong \text{End}(E_2)$. The Isogeny Problem asks: given the curves E_1 and E_2 , find an explicit isogeny*

$$\phi : E_1 \longrightarrow E_2$$

between them.

Remark 2.4.7 *The isogeny problem is fascinating. It has deep links to the endomorphism ring. The study of links and reductions between problems based on isogenies and their endomorphism rings is quite vast and out of scope of this thesis. We refer the reader to the work of Benjamin Wesolowski for more information [124, 125].*

2.4.2 Degree and factorisation

We define the notion of separability following [29].

Definition 2.4.8 (Separable polynomial) *A nonzero polynomial $f(X) \in K[X]$ is called separable when it has distinct roots in a splitting field over K . That is, each root of $f(X)$ has multiplicity 1. If $f(X)$ has a multiple root then $f(X)$ is called inseparable.*

Definition 2.4.9 If α is algebraic over K , it is called separable over K when its minimal polynomial in $K[X]$ is separable. If the minimal polynomial of α in $K[X]$ is inseparable then α is called inseparable over K .

Definition 2.4.10 (Separable extension) A finite extension L/K is called separable if every element of L is separable over K . When L/K is not separable, it is called inseparable.

Let $\mathcal{E}_1/K$ and $\mathcal{E}_2/K$ be elliptic curves. A non-constant isogeny $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ induces an injection of function fields fixing K :

$$\phi^* : \overline{K}(\mathcal{E}_2) \hookrightarrow \overline{K}(\mathcal{E}_1), \quad \phi^*(f) = f \circ \phi.$$

Definition 2.4.11 Let $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ be a non-constant isogeny (otherwise its degree is zero). The degree of ϕ is denoted by $\deg \phi$. We define it to be

$$[\overline{K}(\mathcal{E}_1) : \phi^* \overline{K}(\mathcal{E}_2)].$$

ϕ is said to be separable/inseparable/purely inseparable respectively if the field extension $\overline{K}(\mathcal{E}_1)/\phi^* \overline{K}(\mathcal{E}_2)$ is separable/inseparable/purely inseparable.

Example 2.4.12 For $m \neq 0$ non-divisible by p , the multiplication-by- m map on $\mathcal{E}/K$ is a finite separable endomorphism.

Definition 2.4.13 (Kernel) Let $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ be a nonzero isogeny. We define its kernel as

$$\ker \phi := \phi^{-1}(O).$$

It forms a finite group and its cardinality divides the degree of the isogeny.

Factorisation

In order to efficiently compute isogenies, the first step is to "factor" them. In the following, we state how an isogeny decomposes depending on its degree. Any isogeny of degree coprime to p is separable and all purely inseparable isogenies have degree equal to a power of p .

Theorem 2.4.14 ([110, Corollary III.4.11]) Let $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ and $\psi : \mathcal{E}_1 \rightarrow \mathcal{E}_3$ be nonconstant isogenies, and assume that ϕ is separable. If

$$\ker \phi \subseteq \ker \psi,$$

then there is a unique isogeny

$$\lambda : E_2 \rightarrow E_3,$$

satisfying

$$\psi = \lambda \circ \phi.$$

2.4.3 The dual isogeny

Definition 2.4.15 (Dual isogeny) Let $\phi : \mathcal{E}_1 \longrightarrow \mathcal{E}_2$ be a nonconstant isogeny of degree m . The dual isogeny of ϕ is the unique isogeny

$$\hat{\phi} : \mathcal{E}_2 \longrightarrow \mathcal{E}_1$$

satisfying

$$\hat{\phi} \circ \phi = [m],$$

where $[m]$ denotes the multiplication-by- m map on the elliptic curve $\mathcal{E}_1$.

Theorem 2.4.16 ([110, Theorem III.6.2]) Let

$$\phi : \mathcal{E}_1 \longrightarrow \mathcal{E}_2$$

be an isogeny.

(a) Let $m = \deg \phi$. Then

$$\hat{\phi} \circ \phi = [m] \text{ on } \mathcal{E}_1 \quad \text{and} \quad \phi \circ \hat{\phi} = [m] \text{ on } \mathcal{E}_2.$$

(b) Let $\lambda : \mathcal{E}_2 \rightarrow \mathcal{E}_3$ be another isogeny. Then

$$\widehat{\lambda \circ \phi} = \hat{\phi} \circ \hat{\lambda}.$$

(c) Let $\psi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ be another isogeny. Then

$$\widehat{\phi + \psi} = \hat{\phi} + \hat{\psi}.$$

(d) For all $m \in \mathbb{Z}$,

$$\widehat{[m]} = [m] \quad \text{and} \quad \deg[m] = m^2.$$

(e) $\deg \hat{\phi} = \deg \phi$.

(f) $\hat{\hat{\phi}} = \phi$.

2.4.4 Separable isogenies

In Chapters 4, 5 and 6, we will focus mostly on separable isogenies. They have some strong links with their kernels. Let's consider an isogeny between elliptic curves in Weierstrass form. Every finite subgroup $\mathcal{G}$ of an elliptic curve $\mathcal{E}_1$ determines a separable *quotient isogeny* $\mathcal{E}_1 \rightarrow \mathcal{E}_1/\mathcal{G}$ satisfying $\text{Ker } \phi = \mathcal{G}$. The *kernel polynomial* of ϕ is

$$D(X) := \prod_{P \in S} (X - x(P))$$

where $S \subset \text{Ker } \phi$ is any subset satisfying

$$S \cap -S = \emptyset \quad \text{and} \quad S \cup -S = \text{Ker } \phi \setminus \{O\}. \quad (2.2)$$

Every separable isogeny $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ defined over $\mathbb{F}_q$ can be represented by a rational map in the form

$$\phi : (x, y) \mapsto (\phi_x(x), \phi_y(x, y)) \quad (2.3)$$

with

$$\phi_x(x) = \frac{N(x)}{D(x)^2} \quad \text{and} \quad \phi_y(x, y) = c \cdot y \frac{d\phi_x}{dx}(x) \quad (2.4)$$

where D is the kernel polynomial of ϕ , N is a polynomial derived from D , and c is a normalizing constant in $\mathbb{F}_q$.

As such, we see a correspondence between separable isogenies and their kernels. Besides, we know that for all primes ℓ not dividing q , there are $\ell + 1$ cyclic subgroups of order ℓ in $\mathcal{E}[\ell]$. This is because $\mathcal{E}[\ell] \cong (\mathbb{Z}/\ell\mathbb{Z})^2$, which has exactly $\ell + 1$ subgroups of order ℓ . Thus, in this case, the number of ℓ -degree isogenies is exactly $\ell + 1$.

2.4.5 Vélú's formulæ

Theorem 2.4.17 *Let $\mathcal{E}_1$ be an elliptic curve and let $\mathcal{G}$ be a finite subgroup of $\mathcal{E}_1$. There are a unique (up to isomorphism) elliptic curve $\mathcal{E}_2$ and a separable isogeny*

$$\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$$

satisfying

$$\text{Ker } \phi = \mathcal{G}.$$

Given a curve $\mathcal{E}$ and a finite subgroup $\mathcal{G} \subset \mathcal{E}$, Vélú [119] gives explicit formulæ for the rational functions that define a separable isogeny $\phi : \mathcal{E} \rightarrow \mathcal{E}' := \mathcal{E}/\mathcal{G}$ with kernel $\mathcal{G}$, as well as the resulting codomain curve $\mathcal{E}'$. Although the quotient curve $\mathcal{E}'$ and the

isogeny ϕ are defined up to isomorphism, Vélu's formulæ construct the unique *normalized* isogeny (i.e., with $c = 1$ in (2.4)). We refer the reader to Chapter 4 for more details on Vélu's formulæ and associated algorithms as Vélu'sqrt.

Supersingular and ordinary curves

In isogeny-based cryptography, we work with curves defined over finite fields. As such, we land on two of the subcases in Theorem 2.4.5 ($\text{End}(\mathcal{E}) \neq \mathbb{Z}$). These two subcases define supersingular curves and ordinary curves. Even though the Isogeny Problem (see 2.4.6) is computationally hard, building a scheme using ordinary curves can be quite practically costly as we will investigate in Chapter 4. As so, most of the current research is aimed at supersingular curves.

Definition 2.4.18 *Let K be a field of characteristic $p > 0$ and $\mathcal{E}/K$ an elliptic curve. If $\mathcal{E}$ has the following equivalent properties, then we say $\mathcal{E}$ is supersingular, otherwise we say $\mathcal{E}$ is ordinary.*

- $\text{End}(\mathcal{E})$ is an order in a quaternion algebra over $\mathbb{Q}$.
- $\mathcal{E}[p^r] = 0$ for all $r \geq 1$.
- $\hat{\pi}_p^r$ is purely inseparable for all $r \geq 1$.
- The multiplication-by- $[p]$ map is purely inseparable and $j(\mathcal{E}) \in \mathbb{F}_{p^2}$.

Remark 2.4.19 *If $\mathcal{E}$ is an ordinary elliptic curve, then*

$$\mathcal{E}[p^r] = \mathbb{Z}/p^r\mathbb{Z} \quad \text{for all } r \geq 1.$$

The Frobenius endomorphism

The Frobenius map π_p as in Definition 2.3.4 defines an endomorphism on an elliptic curve $\mathcal{E}/\mathbb{F}_p$. We refer to it as $\pi_{\mathcal{E}}$.

Theorem 2.4.20 (Hasse) *Let $\mathcal{E}/\mathbb{F}_p$ be an elliptic curve defined over a finite field. We have*

$$\pi_{\mathcal{E}}^2 - t\pi_{\mathcal{E}} + p = 0.$$

where the integer t is called the trace of Frobenius endomorphism $\pi_{\mathcal{E}}$, and the Hasse bound holds:

$$|t| \leq 2\sqrt{p}.$$

The number of $\mathbb{F}_p$ -rational points on $\mathcal{E}$ satisfies the formula:

$$\#\mathcal{E}(\mathbb{F}_p) = p + 1 - t.$$

Remark 2.4.21 In the case of a supersingular curve defined over $\mathbb{F}_p$, we have $t = 0 \pmod{p}$ and so $\#\mathcal{E}(\mathbb{F}_p) = p + 1$.

Remark 2.4.22 Theorem 2.4.14 actually reduces to saying that every isogeny factors into a product of separable isogenies and powers of Frobenius.

Isogeny graphs

Given a (subset of) an isogeny class, one can represent the isogenies between curves in the class using a graph. Such isogeny graphs have rich and interesting combinatorial properties, and they have been studied extensively in the literature (see, for example, [22, 30, 34, 40, 66, 67]). However, a detailed exploration of this representation lies beyond the scope of this thesis and is not required for the results that we will present.

2.5 Class Group Action

We now enter the result that has motivated all of the previous mathematical background i.e., how to instantiate a group action using isogenies. References for the following theory can be found in Chapter 3 of the work by Waterhouse [123] and Chapter 42 of John Voight's book [120]. We see how to get an ideal from an isogeny.

Definition 2.5.1 (Kernel ideal) We consider an elliptic curve $\mathcal{E}/K$ defined over a field K . For any finite subgroup $G \subset \mathcal{E}$, we define its kernel ideal

$$I(G) = \{w \in \text{End}(\mathcal{E}) \mid w(G) = 0\}.$$

The kernel ideal is actually a left ideal in $\text{End}(\mathcal{E})$ with norm $n(I) = \#G$.

Definition 2.5.2 We consider an elliptic curve $\mathcal{E}/K$ defined over a field K . We take $I \subset \text{End}(\mathcal{E})$ to be an invertible ideal. The kernel of the ideal I is

$$\mathcal{E}[I] = \bigcap_{w \in I} \text{Ker } w.$$

Remark 2.5.3 In Theorem 2.4.17, we saw that any such finite subgroup gives birth to a unique separable isogeny $\phi : \mathcal{E} \mapsto \mathcal{E}/G$. In this case, $I_\phi := I(\text{ker } \phi)$. The kernel of an ideal I naturally gives rise to an isogeny $\phi_I : \mathcal{E} \mapsto \mathcal{E}/\mathcal{E}[I]$.

There is a 1-to-1 correspondence between those two notions. To extend this to inseparable isogenies we give a corresponding ideal to the Frobenius morphism of $\mathcal{E}$. It corresponds to the unique two-sided ideal of $\text{End}(\mathcal{E})$ with norm equal to p [73].

The class group

There is a correspondence between left-ideals of the endomorphism ring of a curve and isogenies. In the case where $\text{End}(\mathcal{E})$ is commutative (including supersingular curves over $\mathbb{F}_p$), this gives an action by the class group defined by the order equivalent to the endomorphism ring. For more details about the class group structure, we refer the reader to Chapter I. in *Algebraic Number Theory* by Jürgen Neukirch [86].

Definition 2.5.4 (Class group) *We consider an order $\mathcal{O} \subset \mathbb{Q}(\sqrt{-D})$. The class group is the quotient*

$$Cl(\mathcal{O}) = I(\mathcal{O})/P(\mathcal{O})$$

where $I(\mathcal{O})$ are the set of invertible ideals of $\mathcal{O}$ and $P(\mathcal{O})$ the set of principal ideals of $\mathcal{O}$.

Definition 2.5.5 ($\text{Ell}_q(\mathcal{O})$) *Let q be a prime power and let $\mathcal{O}$ be an order in a quadratic imaginary field. We define*

$$\text{Ell}_q(\mathcal{O}) := \{ \mathcal{E}/\overline{\mathbb{F}_q} \mid \text{End}(\mathcal{E}) \supseteq \mathcal{O} \} / \simeq,$$

where $\simeq$ denotes isomorphism of elliptic curves over $\overline{\mathbb{F}_q}$. In other words, $\text{Ell}_q(\mathcal{O})$ is the set of isomorphism classes of elliptic curves over $\overline{\mathbb{F}_q}$ whose endomorphism ring contains a subring isomorphic to $\mathcal{O}$.

There are two fundamental examples that will be of particular interest to us. First, consider *ordinary elliptic curves* $\mathcal{E}$ for which there exists an embedding

$$\mathcal{O} \hookrightarrow \text{End}(\mathcal{E}),$$

a situation known as complex multiplication. In this case, we fix an element of $\mathcal{O}$ of norm q whose image under the embedding corresponds to the Frobenius endomorphism of $\mathcal{E}$.

Second, consider *supersingular elliptic curves* defined over $\mathbb{F}_p$. Here the relevant order is $\mathcal{O} = \mathbb{Z}[\sqrt{-p}]$ where $\mathcal{O} \hookrightarrow \text{End}(\mathcal{E})$, and we take the element $\sqrt{-p} \in \mathcal{O}$ to map to the Frobenius endomorphism of $\mathcal{E}$ under the embedding.

Theorem 2.5.6 (Class group action, [40, Theorem III.1]) *Let $\mathbb{F}_q$ be a finite field, $\mathcal{O} \subset \mathbb{Q}(\sqrt{-D})$ an order in a quadratic imaginary field.*

Assume $\text{Ell}_q(\mathcal{O})$ is non-empty. Then it forms a principal homogeneous space under the action of the class group $\text{Cl}(\mathcal{O})$, given by

$$\begin{aligned} \text{Cl}(\mathcal{O}) \times \text{Ell}_q(\mathcal{O}) &\longrightarrow \text{Ell}_q(\mathcal{O}), \\ (a, \mathcal{E}) &\longmapsto a \star \mathcal{E} := \mathcal{E}/\mathcal{E}[a]. \end{aligned}$$

Remark 2.5.7 *The following equality holds,*

$$\#\text{Cl}(\mathcal{O}) = \#\text{Ell}_q(\mathcal{E}).$$

Thanks to the properties from the beginning of this section, we see that the set $\text{Cl}(\mathcal{O})$ for a maximal order in an imaginary quadratic extension of $\mathbb{Q}$ or a quaternion algebra over $\mathbb{Q}$, is in bijection with $\mathcal{S}(p)$ (set of supersingular j -invariants). This correspondence is called the Deuring correspondence. We won't go further into the mathematical details of it because we won't explore them in this work. However, what matters to us is that thanks to this correspondence, we unlock the possibility to practically implement the *class group action*. As such we give a table which summarizes the Deuring correspondence. This table was presented by Luca De Feo.

Elliptic curves	Number fields / Quaternion algebras
Endomorphisms	Algebraic integers
Endomorphism ring	(Maximal) order
Isogeny	Ideal
Isogeny degree	Ideal norm
Isogeny composition	Ideal multiplication
Isogenies	Ideal classes
Dual isogeny	Conjugate ideal

Figure 2.1: **The Deuring correspondence**

2.5.1 CRS

The Couveignes–Rostovtsev–Stolbunov (CRS) scheme was independently discovered by Couveignes [33] who built it using an isogeny-based EGA (requires the class group structure for ideal smoothing) and Rostostev and Stolbunov [105] who used an isogeny-based REGA (works without the class group structure). This scheme is an instantiation of the Diffie–Hellman protocol built from isogenies.

In its work, Couveignes suggests to use the action of the class group $\text{Cl}(\mathcal{O})$ on $\text{Ell}_q(\mathcal{O})$ as a Hard Homogeneous Space. Let's instantiate in the framework seen above. We take $G = \text{Cl}(\mathcal{O})$, $X = \text{Ell}_q(\mathcal{O})$ and P is some starting elliptic curve $\mathcal{E}_0/\mathbb{F}_q$. Remember that $\text{Ell}_q(\mathcal{O})$ can be represented by the j -invariants of the curves in that set. However, this fails to be a HHS because Problem 2.1.10 i.e., computing the group action for a generic element in $\text{Cl}(\mathcal{O})$ is actually quite difficult. Only algorithms with subexponential complexity (in the discriminant of $\mathcal{O}$, and thus generally in q) are known. Because of this, as suggested by Rostostev and Stolbunov [105], we use a REGA. We take a set S of primes ideals of $\mathcal{O}$. We consider the S -restricted action. These prime ideals can be represented by $(\pi - \lambda, \ell)$ ¹ for some eigenvalue λ modulo a prime ℓ where ℓ is the norm of the ideal.

As such, in the Diffie–Hellman protocol, a secret key will be a product of elements in S of the form $\prod_{\mathfrak{a}_i \in S} [\mathfrak{a}_i]^{e_i}$ for some bounded exponents. Computing the action is now equivalent to performing smaller degree isogenies part of the generating set.

The main bottleneck in CRS is that it is very slow due to slow isogeny computations. As we see in [43], to enable fast ℓ -isogeny computations based on Vélu's formulæ, we need points of order ℓ defined over small-degree extension fields. But it is hard to find ordinary curves over sufficiently large $\mathbb{F}_p$ meeting this criteria. Without these points, CRS takes minutes to run a key exchange. CSIDH provides a breakthrough in efficiency by switching to supersingular curves.

2.5.2 CSIDH

Building on CRS, Castryck, Lange, Martindale, Panny and Renes [20] introduced a fundamental change to make a more practical key exchange. CSIDH can be viewed as a somewhat practical version of CRS using supersingular curves instead of ordinary curves. This could be surprising at first because the supersingular curves set does not come naturally with complex multiplication. However, if we restrict to curves, endomorphisms and isogenies defined over $\mathbb{F}_p$ for some prime p , then we can consider that there is complex multiplication in the same way as for ordinary curves. Indeed, instead of considering the full ring of endomorphisms (which is an order in a quaternion algebra), we restrict to the subring of $\mathbb{F}_p$ -rational endomorphisms, which is an order in a quadratic imaginary field. As so, CSIDH use the HHS $S(p)$ (set of supersingular j -invariants) with action by $\text{Cl}(\mathbb{Z}[\sqrt{-p}])$.

¹This is the standard 2-element representation for ideals in Dedekind domains.

The idea for computing the group action faster for supersingular curves presented by De Feo, Kieffer and Smith [43] is what gave the idea for CSIDH. This impressive speed-up is enabled by the fact that for supersingular curves over $\mathbb{F}_p$, we have $\#\mathcal{E}(\mathbb{F}_p) = p + 1$, so $\#\mathcal{E}(\mathbb{F}_p)$ is divisible by all primes $\ell \mid p + 1$ which were used when building p . Finding an ordinary curve for which that idea is possible to be used is actually very hard and time consuming.

2.6 Signature protocol

Digital signatures answer a need for authenticity, integrity and non-repudiation. It can be used in practice for software updates, email signing, SSL/TLS certificates, to name but a few applications.

2.6.1 Signature scheme

A signature scheme is defined by three algorithms, with an implicit security parameter λ :

- **KeyGen**: returns a private key SK and a public key PK .
- **Sign**: given a private key SK and a message m , returns a signature σ .
- **Verify**: given a putative signature σ on m under a public key PK , returns **Accept** or **Reject**.

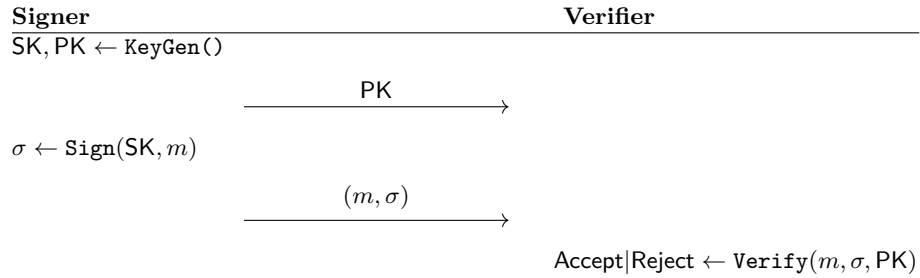


Figure 2.2: Signature and verification as a protocol.

A signature scheme must satisfy the following properties:

Correctness. For all key pairs (PK, SK) output by **KeyGen**, for all messages m , and for all σ produced by **Sign** (SK, m) , we have

$$\text{Verify}(PK, m, \sigma) = \text{Accept}.$$

Security. A signature scheme is *existentially unforgeable under chosen message attack* (EUF-CMA) if no efficient adversary, given access to a signing oracle for SK, can produce a signature such that $\text{Verify}(m, \sigma, \text{PK}) = \text{Accept}$ on a new message not queried to the oracle, except with negligible probability in the security parameter λ . By *n-bit hardness* (or *n-bit security*) we mean that the best known attack costs about 2^n basic operations.

2.6.2 GPV signatures

We introduce here a general GPV-style signature. Let $\mathcal{M}$ be a finitely generated module over an integral domain $\mathcal{R}$: in practice, $\mathcal{M}$ is either a lattice (with $\mathcal{R} = \mathbb{Z}$) or a code (with $\mathcal{R} = \mathbb{F}_q$). Fix a cryptographic hash function $\text{Hash} : \{0, 1\}^* \rightarrow \mathcal{M}$, i.e., a map for which it is hard to recover the preimage from the image or to find collisions. A public key is a random-looking $\mathbf{M} = (\mathbf{M}_0, \dots, \mathbf{M}_{n-1}) \in \mathcal{M}^n$ for some system parameter n . A signature on a message m under $\mathbf{M}$ is a tuple $\sigma = (\text{salt}, \mathbf{s})$ with $\text{salt} \in \{0, 1\}^\lambda$ (a random salt) and $\mathbf{s} \in \mathcal{R}^n$ such that

$$\text{CONSTRAINT}(\mathbf{s}) \quad \text{and} \quad \mathbf{sM} := \sum_{i=0}^{n-1} s_i \mathbf{M}_i = \text{Hash}(\text{salt} \parallel m) \quad (2.5)$$

where $\text{CONSTRAINT}(\mathbf{s})$ is a predicate on $\mathbf{s}$ such as having small norm or a fixed number of nonzero entries. An important variant has Hash mapping into (a subset of) $\mathcal{R}^n$ instead of $\mathcal{M}$, and (2.5) is replaced by

$$\text{CONSTRAINT}(\mathbf{s}) \quad \text{and} \quad (\text{Hash}(\text{salt} \parallel m) + \mathbf{s})\mathbf{M} = \mathbf{0}. \quad (2.6)$$

GPV signatures are mostly constructed in lattice-based cryptography. However, note that WAVE is a code-based scheme which idea was to adapt this framework to codes. Chapter 3 will be dealing with GPV signatures and so we introduce the necessary background on lattices and codes.

2.7 Lattices

The presentation in this section largely follows *A decade of lattice cryptography* by Chris Peikert [95]; definitions are adapted with minor changes to notation and ordering. In this section, we will very briefly introduce the concept of lattices. Lattice-based cryptography is one of the most promising areas for post-quantum security. In Chapter 3, we will focus on SQUIRRELS, a specific lattice-based scheme. We go through the basics of the mathematics and objects behind lattice-based cryptography as well as the computational problems it relies on.

Definition 2.7.1 (Lattice) An n -dimensional lattice $\mathcal{L}$ is an additive discrete subgroup of $\mathbb{R}^n$.

Example 2.7.2 $\mathbb{Z}^n$ is called the integer lattice.

Definition 2.7.3 (Lattice basis) A lattice $\mathcal{L}$ is the set of all integer linear combinations of some linearly independent basis vectors $\mathbf{B} = \{\mathbf{b}_1, \dots, \mathbf{b}_k\}$:

$$\mathcal{L} = \mathcal{L}(\mathbf{B}) := \mathbf{B} \cdot \mathbb{Z}^k = \left\{ \sum_{i=1}^k z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}.$$

The integer k is called the rank of the basis, and is an invariant of the lattice. In the sequel, we restrict our attention to full-rank lattices, where $k = n$. A lattice basis $\mathbf{B}$ is not unique: for any unimodular matrix $\mathbf{U} \in \mathbb{Z}^{n \times n}$, the product $\mathbf{B} \cdot \mathbf{U}$ is also a basis of $\mathcal{L}(\mathbf{B})$, because $\mathbf{U} \cdot \mathbb{Z}^n = \mathbb{Z}^n$.

Definition 2.7.4 (Coset) For a lattice $\mathcal{L}$ and an element $c \in \mathbb{R}^n$, we get a coset in the quotient group $\mathbb{R}^n / \mathcal{L}$,

$$c + \mathcal{L} = \{c + v \mid v \in \mathcal{L}\}.$$

Definition 2.7.5 (Minimum distance) Let $\mathcal{L}$ be an n -dimensional lattice. The minimum distance (or length of the shortest nonzero vector) of $\mathcal{L}$ is defined as

$$\lambda_1(\mathcal{L}) := \min \{\|\mathbf{v}\|, \mathbf{v} \in \mathcal{L} \setminus \{\mathbf{0}\}\},$$

where $\|\mathbf{v}\|$ denotes the Euclidean norm of the vector $\mathbf{v}$.

Hard worst-case problems

We introduce well-studied computational problems with hard worst-case complexity. The two following problems are core to lattice-based cryptography. The first one is on the search for the shortest vector in a lattice. The second is about the search of a "rather close" lattice vector to a target point in $\mathbb{R}^n$. We decide to introduce here the approximate decisional version of SVP as it is the one ultimately used in cryptography.

Problem 2.7.6 (Approximate Shortest Vector Problem ($\text{SVP}_{\gamma(n)}$)) Given a basis $\mathbf{B}$ of an n -dimensional lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$, and an approximation factor $\gamma(n) \geq 1$, find a nonzero vector $\mathbf{v} \in \mathcal{L}$ for which

$$\|\mathbf{v}\| \leq \gamma(n) \cdot \lambda_1(\mathcal{L}).$$

Problem 2.7.7 (Decisional Approximate SVP ($\text{GapSVP}_{\gamma(n)}$)) *Given a basis $\mathbf{B}$ of an n -dimensional lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$ where either $\lambda_1(\mathcal{L}) \leq 1$ or $\lambda_1(\mathcal{L}) > \gamma(n)$, determine which is the case.*

Problem 2.7.8 (Bounded Distance Decoding Problem ($\text{BDD}_{\gamma(n)}$)) *Given a basis $\mathbf{B}$ of an n -dimensional lattice $\mathcal{L} = \mathcal{L}(\mathbf{B})$ and a target point $\mathbf{t} \in \mathbb{R}^n$ with the guarantee that*

$$\text{dist}(\mathbf{t}, \mathcal{L}) < d = \frac{\lambda_1(\mathcal{L})}{2\gamma(n)},$$

find the unique lattice vector $\mathbf{v} \in \mathcal{L}$ such that

$$\|\mathbf{t} - \mathbf{v}\| < d.$$

These problems are polynomial-time in low dimensions, where we can apply classic algorithms like the algorithm by Lenstra, Lenstra and Lovász known as the LLL algorithm [71]. However, we only have exponential solutions for cryptographic-sized parameters, and no efficient quantum solutions [1, 77, 94, 95, 101]. This makes these two problems interesting to build quantum resistant cryptographic protocols.

Hard average-case problems

The first person to introduce a connection between worst-case and average-case lattice-based problems is Ajtai in 1996 [2]. It led to constructions of cryptographic primitives based on average-case problems. We first introduce the SIS problem.

Problem 2.7.9 (Short Integer Solution ($\text{SIS}_{n,q,\beta,m}$)) *Given m uniformly random vectors $\mathbf{a}_i \in \mathbb{Z}_q^n$, forming the columns of a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, find a nonzero integer vector $\mathbf{z} \in \mathbb{Z}^m$ of norm $\|\mathbf{z}\| \leq \beta$ such that*

$$f_{\mathbf{A}}(\mathbf{z}) := \mathbf{A}\mathbf{z} = \sum_i z_i \cdot \mathbf{a}_i = \mathbf{0} \in \mathbb{Z}_q^n.$$

In loose phrasing, the SIS problem is about finding a "short" preimage under the function $f_{\mathbf{A}}$. The hardness of this problem is tightly related to the choice of β as it would be easy to solve it via Gaussian elimination for a large enough β . For a solution to be guaranteed to exist we need $\beta \geq \sqrt{m}$. The SIS problem hardness leads to a family of collision resistant functions.

We now move on to the second famous average-case problem in lattice-based cryptography: learning with errors (LWE). This problem was introduced by Regev [101] in 2005. It is quite close to the SIS-problem and can be seen as its dual.

Problem 2.7.10 (Search Learning With Errors (Search $\text{LWE}_{n,q,\chi,m}$)) Let n, q positive integers, χ an error distribution over $\mathbb{Z}_q$ and $\mathbf{s}$ uniformly random in $\mathbb{Z}_q^n$. Given m independent samples of the form

$$(\mathbf{a}_i, b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \pmod{q}) \in \mathbb{Z}_q^n \times \mathbb{Z}_q,$$

where each $\mathbf{a}_i$ is uniformly at random in $\mathbb{Z}_q^n$, find the secret vector $\mathbf{s}$.

Problem 2.7.11 (Decisional Learning With Errors (Decisional $\text{LWE}_{n,m,q,\chi}$)) Under the same parameters (n, q, χ, m) , distinguish between two distributions over $\mathbb{Z}_q^n \times \mathbb{Z}_q$:

- The LWE distribution: $(\mathbf{a}, b = \langle \mathbf{a}, \mathbf{s} \rangle + e)$ for random $\mathbf{a} \in \mathbb{Z}_q^n$, fixed secret $\mathbf{s} \in \mathbb{Z}_q^n$, and $e \leftarrow \chi$,
- The uniform distribution: $(\mathbf{a}, b)$ chosen uniformly at random from $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

Note that χ is usually taken to be a discretisation of a Gaussian distribution centered at 0.

Reductions

The SIS problem is a case of the SVP problem for the following : Consider the family of q -ary lattices $\mathcal{L}^\perp(\mathbf{A}) := \{\mathbf{z} \in \mathbb{Z}^m : \mathbf{A}\mathbf{z} = \mathbf{0} \in \mathbb{Z}_q^n\} \supseteq q\mathbb{Z}^m$. The SIS problem is to find a short vector in $\mathcal{L}^\perp(\mathbf{A})$ with $\mathbf{A}$ being chosen at random. As such, we have the following result which gives us a hardness result about SIS. A worst-case average-case connection has also been proven for the LWE problem by Regev [101].

Theorem 2.7.12 ([95, Theorem 4.1.2]) For any $m = \text{poly}(n)$, any $\beta > 0$, and any sufficiently large $q \geq \beta \cdot \text{poly}(n)$, solving $\text{SIS}_{n,q,\beta,m}$ with non-negligible probability is at least as hard as solving the decisional approximate shortest vector problem GAPSVP_γ on arbitrary n -dimensional lattices (i.e., in the worst case) with overwhelming probability, for some $\gamma = \beta \cdot \text{poly}(n)$.

Theorem 2.7.13 ([95, Theorem 4.2.4]) For any $m = \text{poly}(n)$, any modulus $q \leq 2^{\text{poly}(n)}$, and any (discretized) Gaussian error distribution χ of parameter $\alpha q \geq 2\sqrt{n}$ where $0 < \alpha < 1$, solving the decisional- $\text{LWE}_{n,q,\chi,m}$ problem is at least as hard as quantumly solving GAPSVP_γ on arbitrary n -dimensional lattices, for some $\gamma = \tilde{O}(n/\alpha)$.

Example 2.7.14 (GPV signature(2.6.2) with lattices) In *SQUIRRELS* [51], $\mathcal{R} = \mathbb{Z}$ and $\mathcal{M} = \mathbb{Z}/\Delta$ for some large Δ (though later we lift to $\mathcal{M} = \mathbb{Z}$); verification uses (2.6)

where $\text{CONSTRAINT}(\mathbf{s})$ is $\|\mathbf{s}\|_2^2 \leq \lfloor \beta^2 \rfloor$ for a small system parameter β . For example, *SQUIRRELS-I* has $n = 1034$, Δ a 5048-bit modulus formed as the product of 165 31-bit primes, and $\lfloor \beta^2 \rfloor = 2026590$.

2.8 Codes

The presentation in this section largely follows the book *Introduction to coding theory* by J. H. van Lint [117] and [5]; definitions are adapted with minor changes to notation and ordering. One of the signature schemes we consider in Chapter 3 is WAVE [46], which is based on hard problems from linear codes. In this section, we recall the necessary background on coding theory needed to understand this scheme. More specifically, we focus on hard problems in a special class of linear codes, namely $(U \mid U + V)$ -codes, which plays a central role in the construction of WAVE. WAVE uses ternary codes, that is $q = 3$. Its hardness assumption relies on the Decoding Problem.

Definition 2.8.1 (Linear code) A q -ary linear code C is a linear subspace of $\mathbb{F}_q^n$. If C has dimension k , then C is called an $[n, k]_q$ code.

Definition 2.8.2 A generator matrix $\mathbf{G}$ for a linear code C is a $k \times n$ matrix for which the rows are a basis of C . We call an n -dimensional matrix H the parity check matrix of C if $\mathbf{x} \in C \leftrightarrow \mathbf{x}H^\perp = 0$.

Definition 2.8.3 (Decoding Problem (DP)) Let C be an $[n, k]_q$ linear code over $\mathbb{F}_q$. Given a target vector $\mathbf{y} \in \mathbb{F}_q^n$ and an integer w , find $\mathbf{x} \in C$ such that the Hamming distance between $\mathbf{x}$ and $\mathbf{y}$ is exactly w ;

$$d(\mathbf{x}, \mathbf{y}) = w,$$

meaning that $\mathbf{x}$ and $\mathbf{y}$ differ in exactly w coordinates.

Depending on the choice of parameters, the Decoding Problem can be either solved in polynomial time or hard.

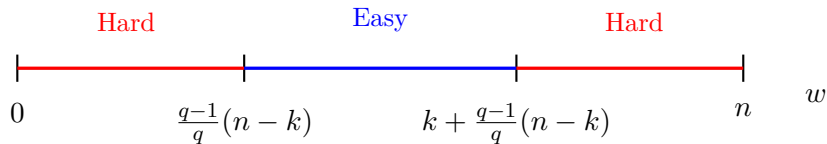


Figure 2.3: Hardness (with respect to w), given a random $[n, k]_q$ -code $C \subseteq \mathbb{F}_q^n$ and $\mathbf{y} \in \mathbb{F}_q^n$, of finding $\mathbf{x} \in C$ at Hamming distance w from $\mathbf{y}$ [5].

Definition 2.8.4 ((U|U+V)-code) Let n , k_U , and k_V be integers with n even and $k_U, k_V \leq n/2$. Let U be an $[n/2, k_U]_q$ -code with generator (resp. parity-check) matrix $\mathbf{G}_U$ (resp. $\mathbf{H}_U$), and let V be an $[n/2, k_V]_q$ -code with generator (resp. parity-check) matrix $\mathbf{G}_V$ (resp. $\mathbf{H}_V$). Let π be a permutation in $\mathfrak{S}_n$, and let $\mathbf{b}$ and $\mathbf{c}$ be vectors in $\mathbb{F}_q^{n/2}$ with $c(i) \neq 0$ for all $i \in [0, n/2)$.

The permuted generalized $(U|U+V)$ -code associated to $(U, V, \pi, \mathbf{b}, \mathbf{c})$ is the $[n, k_U + k_V]_q$ -code admitting the generator matrix $\mathbf{G}$ and parity-check matrix $\mathbf{H}$ defined by

$$\mathbf{G} = \left(\begin{array}{c|c} \mathbf{G}_U & \mathbf{c} \star \mathbf{G}_U \\ \mathbf{b} \star \mathbf{G}_V & \mathbf{d} \star \mathbf{G}_V \end{array} \right)^\pi \quad \text{and} \quad \mathbf{H} = \left(\begin{array}{c|c} \mathbf{d} \star \mathbf{H}_U & -\mathbf{b} \star \mathbf{H}_U \\ -\mathbf{c} \star \mathbf{H}_V & \mathbf{H}_V \end{array} \right)^\pi$$

where $\mathbf{d} := \mathbf{1} + \mathbf{b} \star \mathbf{c}$.

Example 2.8.5 (GPV signature(2.6.2) with codes) In WAVE [46], $\mathcal{R} = \mathbb{F}_3$ and $\mathcal{M} = \mathbb{F}_3^{n-k}$ for system parameters n and k ; verification uses (2.5) where $\text{CONSTRAINT}(\mathbf{s})$ is $\#\{i \mid s_i \neq 0\} = w$ for some (large) $w < n$. For example: WAVE822 has $n = 8576$, $k = 4288$, and $w = 7668 \approx 0.9n$.

Chapter 3

Eval-itation: Compressed verification for post-quantum signatures with long-term public keys

Publication note

This chapter is based on: *Compressed verification for post-quantum signatures with long-term public keys*, Gustavo Banegas and Anaëlle Le Dévéhat and Benjamin Smith, Proceedings of CANS 2025. Minor edits have been made for the thesis. URL : [IACR ePrint 2025/1576](#)

3.1 Introduction

Post-quantum lattice- and code-based signatures can be roughly classified as *conservative* or *structured*, according to whether their underlying hard problems involve general codes and lattices, or involve special algebraic structure. These structures facilitate important practical improvements (often, much smaller public keys); but they also allow

the possibility of specialized attacks, so structured schemes generally have weaker security arguments (i.e., their assumptions are stronger).

For example: compare the structured lattice scheme FALCON [54] with the conservative lattice scheme SQUIRRELS [51]. Both are based on the same GPV design [61] (see Section 2.6.2). At NIST post-quantum security level 1, FALCON and SQUIRRELS have comparable signature sizes: 666 and 1019 bytes, respectively. But while the structured lattices of FALCON give 897-byte public keys, the unstructured lattices of SQUIRRELS push public-key sizes up to 665 *kilobytes*.

Signature schemes with large public keys are unsuitable for applications where public keys are regularly transmitted, such as TLS certificates. They are better-suited to applications where

- the public key is pre-installed on the verifier’s device (for verifying signed software updates, for example, or root certificates), or
- the public key is transmitted, but the cost of transmission is amortised over many subsequent verifications (in `ssh` authentication, for example).

These applications often involve public keys with *very* long lifetimes: 20-30 years for root certificates like ISRG Root X1 and GlobalSign Root R1, for example, and a decade or more for IoT code-signing certificates and government-issued digital IDs.

The long-term nature of keys in these applications makes conservative security assumptions reassuring, but working with very large public keys remains expensive and inconvenient. This makes hash-based signatures like SLH-DSA (SPHINCS+) [10, 85] an interesting choice: they offer conservative security assumptions *and* very small public keys—but at the cost of large signatures and computationally intensive verification. SQUIRRELS offers shorter signatures and faster verification, but its 665 kB public keys make long-term storage impractical.

3.1.1 Compressed verification

We want to reduce public key storage for conservative GPV-style signatures (see Section 2.6.2). The core idea is that the verifier can (pre-)process the public key PK once to derive a much smaller verification key VK (private to the verifier), which they can then use in place of PK for confident—and often much faster—signature verification.

More formally: suppose we are given a signature scheme defined by three algorithms, with an implicit security parameter λ :

- **KeyGen**: returns a private key SK and a public key PK.

- **Sign**: given a private key SK and a message m , returns a signature σ .
- **Verify**: given a putative signature σ on m under a public key PK , returns **Accept** or **Reject**.

We will define three additional algorithms to be used by the verifier:

- **CKeyGen**: returns a (private) compression key CK .
- **VKeyGen**: given a public key PK and a (private) compression key CK , return a private verification key VK .
- **CVerify**: given a putative signature σ on m and a verification key VK , returns **Accept** or **Reject**.

The goal is to define these functions such that if $VK = VKeyGen(CK, PK)$ for some public key PK and some CK output by **CKeyGen**, then

1. if $Verify(\sigma, m, PK) = \text{Accept}$ then $CVerify(\sigma, m, VK) = \text{Accept}$;
2. if $Verify(\sigma, m, PK) = \text{Reject}$ then $CVerify(\sigma, m, VK) = \text{Reject}$ with probability $\geq 1 - 1/2^\mu$ for a second security parameter μ ; and
3. the size of VK is much smaller than the size of PK .

In terms of storage, CK is generated randomly and—once it has been used in **VKeyGen**—need not be stored. Therefore, the verifier only needs to retain VK .

The verifier is free to choose μ ; our goal is that the probability that **CVerify** accepts one forgery after Q attempts is on the order of $Q/\#\mathcal{S}$, where $\mathcal{S}$ is the verifier’s compression-keyspace. We will generally take $\#\mathcal{S} \approx 2^\mu$ with $\mu \approx \lambda$, assuming Q is relatively small (in any case, $Q \leq 2^{64}$). The verifier may force a limit on Q by refreshing VK after a given number of rejections.

Figure 3.1 illustrates the signature protocol with compressed verification. CK and VK are private *to the verifier* and are derived only from the signer’s public key PK , and not the signer’s private key SK . Additionally, the verification algorithm **CVerify** does not require PK or CK . From the signer’s point of view, the original signature scheme is unchanged.

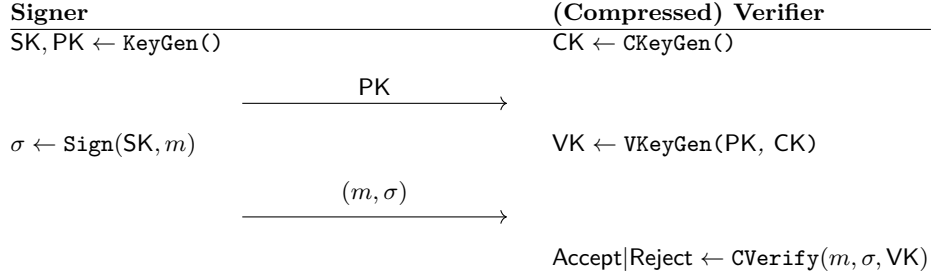


Figure 3.1: Compressed verification as a protocol.

3.1.2 Results

We introduce a framework for *compressed verification* in GPV-style signatures. The verifier chooses a private homomorphism ϕ and uses it to compress incoming public keys to compact *private verification keys*. We give security arguments for the general construction, and consider two conservative instantiations: SQUIRRELS [51], a lattice-based signature, and WAVE [5, 46], a code-based signature.

SQUIRRELS and WAVE both have little special algebraic structure. This builds confidence in their security, but it comes at an important practical cost: as we see in Table 3.1, their public keys are *very* big. Indeed, among the submissions to the NIST Postquantum Signatures on-ramp with no vulnerabilities found during Round 1, SQUIRRELS and WAVE had the largest public keys at most security levels.¹ These oversized public keys would be a major factor in the non-selection of these schemes for Round 2 of the standardization process. They also make SQUIRRELS and WAVE prime candidates to demonstrate the effectiveness of our technique.

We developed two implementations of both SQUIRRELS and WAVE:

1. A comprehensive implementation in Python, and
2. A C implementation, based on the reference implementations of SQUIRRELS [113] and WAVE, used to measure performance improvements.

Table 3.1 shows the significant reduction in verifier storage achieved for both SQUIRRELS and WAVE while maintaining security levels (and backward compatibility with the original schemes). Specifically, we achieve a **compression ratio of up to 34x** for WAVE at higher security levels, and **over 26x** for SQUIRRELS at comparable levels. Compressed

¹The *classic* parameters of UOV, a conservative multivariate scheme, had larger public keys than SQUIRRELS at NIST PQ Security Level 5, but other UOV variants had smaller keys. In any case, WAVE is the undisputed super-heavyweight champion: its Level I public keys were larger than even the Level V keys of any other scheme.

Table 3.1: Signature scheme parameters and verification times. All sizes are in bytes. Compressed verification parameters are chosen such that $\mu \approx \lambda$ (see Section 3.4 and Section 3.5 for details). Verification times are cycle counts on an Intel Core i7-1365U processor running Arch Linux Kernel 6.11.5-arch1. “Reference” refers to C reference implementations, and “Compressed” to our own C code. WAVE signatures are variable-length; sizes here are upper bounds, and WAVE signature length roughly doubles for compressed verification.

	Reference		Compressed verification			Verification time (kCycles)		
	$ \sigma $	$ \text{PK} $	$ \text{VK} $	(CK)	$ \text{PK} / \text{VK} $	Reference	Compressed	Speedup
<i>NIST PQ Security Level 1, classical $\lambda = 128$</i>								
SQUIRRELS-I	1 019	681 780	20 700	(3 360)	<i>32.9×</i>	280	254	<i>9.3%</i>
WAVE822	822	3 677 390	171 594	(83 822)	<i>21.4×</i>	1 101	762	<i>30.8%</i>
SPHINCS+-SHAKE-128s	7 856	32	—	—	—	3 285	—	—
XMSS ^{MT} -20-2	4 963	64	—	—	—	2 868	—	—
<i>NIST PQ Security Level 3, classical $\lambda = 192$</i>								
SQUIRRELS-III	1 554	1 629 640	49 824	(8 480)	<i>32.7×</i>	551	520	<i>5.69%</i>
WAVE1249	1 249	7 867 598	266 188	(183 134)	<i>29.6×</i>	2 330	1 865	<i>19.9%</i>
SPHINCS+-SHAKE-192s	16 224	48	—	—	—	5 374	—	—
XMSS ^{MT} -40-4	9 893	64	—	—	—	5 729	—	—
<i>NIST PQ Security Level 5, classical $\lambda = 256$</i>								
SQUIRRELS-V	2 025	2 786 580	90 598	(15 048)	<i>30.8×</i>	916	898	<i>1.9%</i>
WAVE1644	1 644	13 632 308	370 436	(321 507)	<i>36.8×</i>	3 911	3 198	<i>18.2%</i>
SPHINCS+-SHAKE-256s	29 792	64	—	—	—	7 567	—	—
XMSS ^{MT} -60-6	14 824	64	—	—	—	8 743	—	—

verification can also reduce verification time: verification is **up to 9.26% faster** than ordinary verification for SQUIRRELS and **up to 30% faster** for WAVE.

As a baseline for conservative signatures, Table 3.1 includes the hash-based schemes SPHINCS+ and XMSS^{MT} [64]. For SPHINCS+, the authors provide a benchmark script, which we ran in our environment; in the table we report the fastest result produced by that script [112]. For XMSS^{MT}, we used the reference code [65] with the smallest parameters for each security level.

Remark 3.1.1 *Compressed verification may also benefit schemes like Ajtai-based hash-and-sign [24, 77], which offer strong SIS-based security. The benefit is limited for structured GPV-style schemes such as FALCON [54]: public keys are already compact, and compressed verification requires the full (s_1, s_2) signature rather than just s_2 , thus doubling signature size.*

Remark 3.1.2 *Our method is compatible with the PS-3 [99] and BUFF [36] transforms, which strengthen security in certain attack models. However, both require hashing PK with the message—a costly step when PK is large. We suggest storing the much smaller Hash(PK) and hashing that with the message instead.*

3.1.3 Related work: flexible signatures and progressive verification

Fischlin’s *progressive verification* for MACs [53] can probabilistically Reject early or Accept with reduced confidence; [53, §4] suggests an extension to signatures. Le, Kelkar, and Kate’s *flexible signatures* [70], verified up to a real “confidence level” $0 \leq \alpha \leq 1$, quantify “partial” verification when expensive verification operations are interrupted voluntarily by the user or forcibly by the OS. This can improve fault-tolerance and reduce the cost of verification in embedded applications, but it does not reduce public-key or signature sizes. Indeed, the main targets in [70] are hash-based signatures, where public keys are already extremely compact; but extensions to GPV signatures are projected in [70, §5.3].

Taleb and Vergnaud revisit progressive verification in [116], analyzing Bernstein’s RSA trick (see Section 3.2) and GPV signatures (including WAVE). They propose verifying GPV signatures using a small set of linear combinations of columns based on a random linear code, achieving exponential confidence growth with runtime, unlike the linear growth in [53] and [70]. However, their approach significantly increases public key size, which is the opposite of our goal.

Boschini, Fiore, Pagnin, Torresetti, and Visconti [17] propose an *efficient verification* for signatures which verify using a matrix-vector product $\mathbf{M}\mathbf{v}^\top$. In an “offline” phase they compute a matrix $\mathbf{M}'$ formed by k random linear combinations of the n rows of $\mathbf{M}$; then, in an “online” phase, they verify using $\mathbf{M}'\mathbf{v}^\top$, with reduced confidence but with a speed-up of n/k . This is very similar to what we do with WAVE in Section 3.5, but they repeat the offline phase for every verification rather than maintaining the same $\mathbf{M}'$; indeed, their focus is minimising online verification latency, rather than reducing overall verification time or key sizes.

3.2 Warm-up: Bernstein’s trick for Rabin–Williams

As a warm-up, we recall Bernstein’s fast Rabin–Williams signature verification [8]. We write $\text{PRIMES}(\mu)$ for the set of (exactly) μ -bit primes: that is,

$$\text{PRIMES}(\mu) := \{2^{\mu-1} < p < 2^\mu \mid p \text{ is prime}\}.$$

3.2.1 Verifying Rabin–Williams signatures

A Rabin–Williams signature [100, 126] on a message m under a public key $N = pq$ is a tuple $\sigma = (e, f, \text{salt}, s)$ such that

$$efs^2 \equiv \text{Hash}(\text{salt} \parallel m) \pmod{N} \quad (3.1)$$

where $1 < s < N$, salt is a salt value, and $e \in \{-1, 1\}$ and $f \in \{1, 2\}$ are chosen such that s exists for the given salt , m , and N . That is: given $\sigma = (e, f, \text{salt}, s)$, m , and $\text{PK} = N$, $\text{Verify}(\sigma, m, \text{PK})$ returns **Accept** if and only if (3.1) holds.

If (3.1) is satisfied, then there is a unique integer $-2N < t < 2N$ such that

$$efs^2 - tN = \text{Hash}(\text{salt} \parallel m). \quad (3.2)$$

(The sign of t is equal to e .) Note that (3.2) holds over $\mathbb{Z}$, not just over $\mathbb{Z}/N\mathbb{Z}$; and any solution to (3.2) yields a solution to (3.1) and vice versa, so verifying (3.1) or (3.2) is mathematically (though not algorithmically) equivalent.

Bernstein suggested speeding up verification by including t in σ and verifying (3.2) modulo a random λ -bit prime ℓ (with λ the security parameter). The verifier picks $\ell \in \text{PRIMES}(\lambda)$, computes $N_\ell := N \bmod \ell$, and upon receiving $\sigma = (e, f, \text{salt}, s, t)$, checks

$$efs_\ell^2 - t_\ell N_\ell \equiv h_\ell \pmod{\ell}, \quad (3.3)$$

where $s_\ell := s \bmod \ell$, $t_\ell := t \bmod \ell$, and $h_\ell := \text{Hash}(\text{salt} \parallel m) \bmod \ell$. Since $\ell \ll N$, this is faster than computing $s^2 \bmod N$; the speed-up increases with λ . The trade-off: including t doubles the size of σ , and generating ℓ is relatively costly. However, as Bernstein notes, ℓ can be reused across multiple verifications—even for different public keys—if kept secret, amortizing the cost.

This trick was proposed to speed up verification. We observe that it also saves space if many signatures are verified under the same N , since N_ℓ can be stored instead of N (and this saves even more time, since N_ℓ need not be recomputed).

In terms of our framework above,

- **CKeyGen** samples a random λ -bit prime ℓ ;
- **VKeyGen**($\text{CK} = \ell$, $\text{PK} = N$) returns $\text{VK} = (\ell, N_\ell := N \bmod \ell)$;
- **CVerify**(m , $\sigma = (e, f, \text{salt}, s, t)$, $\text{VK} = (\ell, N_\ell)$) returns **Accept** if and only if $efs^2 - tN_\ell \equiv \text{Hash}(\text{salt} \parallel m) \pmod{\ell}$.

As Bernstein observes, the same technique applies to RSA signatures. However, if e is the public exponent, the resulting integer t is roughly N^{e-1} , making the signature e times longer than a standard RSA signature.

3.2.2 Security argument for Bernstein’s trick

Let’s say that an ℓ -forgery on a message m for a public key $\text{PK} = N$ is a vector $(e, f, \text{salt}, s, t)$ such that (3.2) (and hence (3.1)) fails, but (3.3) holds. That is: an ℓ -forgery is a putative expanded Rabin–Williams signature that `Verify` with $\text{PK} = N$ would safely reject, but `CVerify` with $\text{VK} = (\ell, N \bmod \ell)$ would accept. (We assume that forging a signature for (3.1) or (3.2) is infeasible.)

An adversary that knows ℓ can easily construct ℓ -forgeries for any m and N . Take a random `salt`, and find a t such that $x := \text{Hash}(\text{salt} \parallel m) + tN$ is a square modulo ℓ ; then, compute $s := x^{1/2} \pmod{\ell}$ (which is easy because ℓ is prime); finally, set $e := 1$ and $f := 1$.

Conversely, if we can find an ℓ -forgery $\sigma = (e, f, \text{salt}, s, t)$ for (m, N) then

$$\ell \mid \Xi(\sigma, m, N) \quad \text{where} \quad \Xi(\sigma, m, N) := efs^2 - tN - \text{Hash}(\text{salt} \parallel m).$$

At just λ bits, the prime ℓ is sufficiently small (for cryptographic values of λ) to be recovered from $\Xi(\sigma, m, N)$ with ECM [72, 80, 127].²

An adversary $\mathcal{A}$ who can compute an ℓ -forgery can therefore find ℓ , and vice versa. But $\mathcal{A}$ ’s interaction with the verifier is limited to submitting tuples (σ, m, N) , and observing whether they are accepted or rejected: that is, whether the unknown ℓ divides $\Xi(\sigma, m, N)$ or not. Let

$$\mu := \log_2 \ell \quad \text{and} \quad \kappa := \lfloor (\log_2 N) / \mu \rfloor.$$

Observe that $\mathcal{A}$ learns nothing from an ℓ -forgery attempt (σ, m, N) such that $\Xi(\sigma, m, N)$ is not divisible by at least one μ -bit prime, and that if $\Xi(\sigma, m, N) \neq 0$, then it is divisible by at most 2κ μ -bit primes (because $|\Xi(\sigma, m, N)| < 2N^2$). Therefore, if an adversary makes at most Q forgery attempts against a verifier using a μ -bit prime ℓ for VK , then their success probability is at most

$$P(N, \mu, Q) := \frac{2\kappa Q}{\#\text{PRIMES}(\mu)}. \quad (3.4)$$

It follows from [49, Corollary 5.3] that

$$0.975 \frac{2^{\mu-1}}{(\mu-1) \log 2} < \#\text{PRIMES}(\mu) < \frac{2^{\mu-1}}{(\mu-1) \log 2}. \quad (3.5)$$

Thus, $P(N, \mu, Q) \approx Q \cdot (\log N) / 2^{\mu-2}$. For $\mu \approx \lambda$, this bound remains negligible even across more verifications than could feasibly be generated, without needing to refresh ℓ or track rejections.

²Further: if we can find two ℓ -forgeries σ_1 and σ_2 —not necessarily for the same m and N —then $\ell \mid g := \gcd(\Xi(\sigma_1, m_1, N_1), \Xi(\sigma_2, m_2, N_2))$, and in fact probably $\ell = g$.

3.3 The general approach

3.3.1 Compressed verification

Let Σ be a general GPV-style signature on an $\mathcal{R}$ -module $\mathcal{M}$ as described in Chapter 2, and fix a set $\mathcal{S}$ of $\mathcal{R}$ -submodules of $\mathcal{M}$. We define a compressed-verification signature scheme Σ_{comp} with the same **KeyGen** and **Sign** as in Σ , but with **Verify** replaced by the following **CKeyGen**, **VKeyGen**, and **CVerify**:

- **CKeyGen**: samples a random $\mathcal{K}$ from $\mathcal{S}$, and returns the quotient homomorphism $\text{CK} := \phi : \mathcal{M} \rightarrow \overline{\mathcal{M}} := \mathcal{M}/\mathcal{K}$ (which is unique up to isomorphism, and has kernel $\ker \phi = \mathcal{K}$).
- **VKeyGen**: takes $\text{PK} = \mathbf{M}$ and returns $\text{VK} := (\phi, \overline{\mathbf{M}} := (\phi(\mathbf{M}_1), \dots, \phi(\mathbf{M}_n)))$.
- **CVerify**: given m , $\sigma = (\text{salt}, \mathbf{s})$, and VK , let $\mathbf{h} = \text{Hash}(\text{salt} \parallel m)$. If σ would normally be verified with $(??)$, then **CVerify** returns **Accept** if and only if $\text{CONSTRAINT}(\mathbf{s})$ and $\phi(\mathbf{sM}) = \phi(\mathbf{h})$; and since ϕ is a homomorphism of $\mathcal{R}$ -modules, this means

$$\text{CONSTRAINT}(\mathbf{s}) \quad \text{and} \quad \sum_{i=0}^{n-1} s_i \overline{\mathbf{M}}_i = \phi(\mathbf{h}) \quad \text{in } \overline{\mathcal{M}}. \quad (3.6)$$

If verification normally uses $(??)$, then **CVerify** returns **Accept** if and only if

$$\text{CONSTRAINT}(\mathbf{s}) \quad \text{and} \quad \sum_{i=0}^{n-1} (s_i + h_i) \overline{\mathbf{M}}_i = \mathbf{0} \quad \text{in } \overline{\mathcal{M}}. \quad (3.7)$$

If (3.7) is used for verification, then $\text{CK} = \phi$ need not be included in VK .

If the verifier verifies many signatures from the same signer, then the cost of **CKeyGen** and **VKeyGen** is amortised over the many subsequent **CVerify** calls. A good choice of ϕ can also reduce the time required for each verification.

3.3.2 Correctness and security

The correctness of the signature scheme Σ_{comp} described above follows from ϕ being a homomorphism: $(??)$ implies (3.6) and $(??)$ implies (3.7). Our security goal is EUF-CMA (Existential unforgeability under adaptive chosen-message attacks). Recall that a signature scheme $\Sigma = (\text{Gen}, \text{Sign}, \text{Verify})$ is said to be EUF-CMA if for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ wins the following game is negligible in the security parameter λ :

1. The challenger gets $(\text{pk}, \text{sk}) \leftarrow \text{Gen}(1^\lambda)$ and gives pk to the adversary $\mathcal{A}$.

2. The adversary $\mathcal{A}$ has access to a signing oracle $\mathcal{O}_{\text{Sign}}$ and a verification oracle $\mathcal{O}_{\text{Verify}}$. It may adaptively query $\mathcal{O}_{\text{Sign}}$ on messages $m_1, \dots, m_q$ and receive valid signatures σ_i for each. It also may query $\mathcal{O}_{\text{Verify}}$ on signatures σ_i and know if it is valid signature or not.
3. $\mathcal{A}$ outputs a pair (m^*, σ^*) .
4. $\mathcal{A}$ wins the game if:
 - a) $\text{Verify}_{\text{pk}}(m^*, \sigma^*) = 1$ (i.e., the signature is valid), and
 - b) $m^* \notin \{m_1, \dots, m_q\}$ (i.e., m^* was not queried to the signing oracle).

We want to relate the EUF-CMA security of a GPV signature with compressed verification to the assumed EUF-CMA security of the original scheme. We model forgery attempts as attempts at solving a hidden-structure problem:

Definition 3.3.1 (Submodule Element Guessing Problem $\text{SEGP}(\mathcal{S}, \mathcal{T})$) *Let $\mathcal{M}$ be an $\mathcal{R}$ -module, $\mathcal{T}$ a finite subset of $\mathcal{M}$, and $\mathcal{S}$ a set of $\mathcal{R}$ -submodules of $\mathcal{M}$. The **Submodule Element Guessing Problem** $\text{SEGP}(\mathcal{S}, \mathcal{T})$ is: given a membership oracle $\mathcal{O}_{\mathcal{K}}$ for an unknown submodule $\mathcal{K} \in \mathcal{S}$ taking input in $\mathcal{T}$ (i.e.: $\mathcal{O}_{\mathcal{K}}$ takes $\mathbf{t} \in \mathcal{T}$ and returns True if $\mathbf{t} \in \mathcal{K}$ and False otherwise), find an element $\mathbf{t}^* \neq \mathbf{0} \in \mathcal{T}$ such that $\mathcal{O}_{\mathcal{K}}(\mathbf{t}^*) = \text{True}$, i.e., $\mathbf{t}^* \in \mathcal{K}$.*

The set $\mathcal{S}$ in Definition 3.3.1 represents the set of possible kernels of the secret homomorphism ϕ , and $\mathcal{T}$ represents the vectors constructed by forgery attempts before input to ϕ .

Definition 3.3.2 *Let Σ be a general GPV signature scheme as above. We define*

$$\mathcal{T}(\Sigma) := \{\mathbf{sM} - \mathbf{h} : \mathbf{h} \in \text{Im}(\text{Hash}), \mathbf{s} \in \mathcal{R}^n \mid \text{CONSTRAINT}(\mathbf{s})\}$$

if Σ uses (??) for verification, or

$$\mathcal{T}(\Sigma) := \{(\mathbf{h} + \mathbf{s})\mathbf{M} : \mathbf{h} \in \text{Im}(\text{Hash}), \mathbf{s} \in \mathcal{R}^n \mid \text{CONSTRAINT}(\mathbf{s})\}$$

if Σ uses (??) for verification.

Theorem 3.3.3 *Let Σ be a general GPV-style signature scheme on an $\mathcal{R}$ -module $\mathcal{M}$, and fix a set $\mathcal{S}$ of $\mathcal{R}$ -submodules of $\mathcal{M}$. For each $\mathcal{K}$ in $\mathcal{S}$, let $\Sigma_{\text{comp}}^{\mathcal{K}}$ be the instance of Σ_{comp} where CKeyGen samples $\mathcal{K}$ from $\mathcal{S}$. If $\mathcal{A}$ is an algorithm running in time T that*

wins the EUF-CMA game for $\Sigma_{\text{comp}}^{\mathcal{K}}$ with probability P , then there exists an algorithm $\mathcal{B}$ running in time $T + O(1)$ that succeeds with probability P in winning the EUF-CMA game for Σ , or solving the $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ instance corresponding to $\mathcal{K}$.

Proof. The challenger for EUF-CMA game of Σ gives $\text{PK} = \mathbf{M}$ to the algorithm $\mathcal{B}$. $\mathcal{B}$ has access to a signing oracle $\mathcal{O}_{\text{sign}}$, a verification oracle $\mathcal{O}_{\text{verify}}$ both associated to Σ and a submodule membership oracle $\mathcal{O}_K$ associated to $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$. $\mathcal{B}$ calls $\mathcal{A}$ on $\text{PK} = \mathbf{M}$. $\mathcal{B}$ answers a signing oracle query m from $\mathcal{A}$ by $\mathcal{O}_{\text{sign}}(m)$ as the signing process is the same for Σ and $\Sigma_{\text{comp}}^{\mathcal{K}}$. If $\mathcal{A}$ makes a query to the verification oracle for $\Sigma_{\text{comp}}^{\mathcal{K}}$ on a signature $\sigma = (\text{salt}, \mathbf{s})$, $\mathcal{B}$ queries $\mathcal{O}_{\text{verify}}(\sigma)$. If $\mathcal{O}_{\text{verify}}(\sigma) = 1$ then $\mathcal{B}$ answers 1 to $\mathcal{A}$. Otherwise, If Σ verifies with $(??)$ then $\mathcal{B}$ sets $\mathbf{t} := \mathbf{sM} - \text{Hash}(\text{salt} \parallel m)$; if Σ verifies with $(??)$ instead, then $\mathcal{B}$ sets $\mathbf{t} := (\text{Hash}(\text{salt} \parallel m) + \mathbf{s})\mathbf{M}$. $\mathcal{B}$ answers the verification oracle query by $\mathcal{O}_K(\mathbf{t})$.

If $\mathcal{A}$ fails, then $\mathcal{B}$ fails. Otherwise, it receives a $(m^*, \sigma^* = (\text{salt}^*, \mathbf{s}^*))$ such that $\text{CONSTRAINT}(\mathbf{s}^*)$ holds and $\text{CVerify}(m^*, \sigma^*, \text{VK}) = \text{Accept}$. $\mathcal{B}$ computes $\mathbf{t}^*$, if $\mathbf{t}^* = \mathbf{0}$, then $\text{Verify}(m^*, \sigma^*, \text{PK})$ would return Accept , so $\mathcal{B}$ returns (m^*, σ^*) and wins the EUF-CMA game associated to Σ . Otherwise, $\mathbf{t}^*$ is a nonzero element of $\mathcal{K}$, so $\mathcal{B}$ returns $\mathbf{t}^*$ and finds a solution to $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$. $\square$

Theorem 3.3.3 tells us that if Σ is EUF-CMA secure, then forging a signature for $\Sigma_{\text{comp}}^{\mathcal{K}}$ is *at least as hard* as solving the $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ instance corresponding to $\mathcal{K}$. We can therefore choose secure parameters for Σ_{comp} by choosing the set $\mathcal{S}$ of compression keys such that random instances of $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ are hard.

3.3.3 The hardness of SEGP

The hardness of $\text{SEGP}(\mathcal{S}, \mathcal{T})$ depends on the choice of $\mathcal{S}$ and $\mathcal{T}$, and also on the properties of $\mathcal{R}$ and $\mathcal{M}$ (and $\mathcal{M}/\mathcal{K}$ for $\mathcal{K}$ in $\mathcal{S}$). There are a few general things that we can say before returning to the problem in the concrete cases of SQUIRRELS and WAVE later. Let $\mathcal{O}$ be a membership oracle for a secret $\mathcal{K}$ sampled uniformly random from $\mathcal{S}$, and accepting only queries from $\mathcal{T}$; and let $\phi : \mathcal{M} \rightarrow \mathcal{M}/\mathcal{K}$ be the quotient homomorphism. We can assume $\phi(\mathcal{T}) = \mathcal{M}/\mathcal{K}$. The adversary's goal is to find some $\mathbf{t} \neq \mathbf{0}$ in $\mathcal{T}$ such that $\mathcal{O}(\mathbf{t}) = \text{True}$: implicitly, $\mathbf{t} \in (\mathcal{K} \setminus \{\mathbf{0}\}) \cap \mathcal{T}$.

The adversary makes a series of adaptive queries $\mathbf{t}^{(1)}, \mathbf{t}^{(2)}, \dots$ to $\mathcal{O}$. Suppose $\mathcal{O}(\mathbf{t}^{(i)}) = \text{False}$ for $1 \leq i \leq Q$. The adversary wants $\phi(\mathbf{t}^{(Q+1)}) \neq \phi(\mathbf{t}^{(i)})$ for $1 \leq i \leq Q$, because none of the $\phi(\mathbf{t}^{(i)})$ were $\mathbf{0}$. In the best case for the adversary all these values are distinct,³

³Note that collisions $\phi(\mathbf{t}^{(i)}) = \phi(\mathbf{t}^{(j)})$ are not useful to the adversary, because they cannot detect them without querying $\mathcal{O}$ on all the $\mathbf{t}^{(i)} - \mathbf{t}^{(j)}$.

so if the adversary chooses $\mathbf{t}^{(Q+1)}$ arbitrarily then

$$P[\mathcal{O}(\mathbf{t}^{(Q+1)})] \leq \frac{1}{\#(\mathcal{M}/\mathcal{K}) - Q}.$$

(If the adversary can choose $\mathbf{t}^{(Q+1)}$ such that it maps into a proper submodule $\mathcal{N} \subset \mathcal{M}/\mathcal{K}$ then we can replace $\#(\mathcal{M}/\mathcal{K})$ with $\#\mathcal{N}$ and Q with the number of prior queries landing in $\mathcal{N}$, but this is not significant in our applications.)

In the meantime, the adversary has also learned that

$$\mathcal{K} \notin \bigcup_{i=1}^Q \mathcal{S}_{\mathbf{t}^{(i)}} \quad \text{where} \quad \mathcal{S}_{\mathbf{t}} := \{\mathcal{K} \in \mathcal{S} \mid \mathbf{t} \in \mathcal{K}\} \subset \mathcal{S}.$$

In our applications $\mathcal{S}$ is finite, so there exists an integer

$$\kappa_{\mathcal{T}} := \max\{\#\mathcal{S}_{\mathbf{t}} : \mathbf{t} \in \mathcal{T}\};$$

each unsuccessful query eliminates up to $\kappa_{\mathcal{T}}$ candidate kernels from consideration. The adversary must choose $\mathbf{t}^{(Q+1)}$ such that $\mathcal{S}_{\mathbf{t}^{(Q+1)}} \not\subset \bigcup_{i=1}^Q \mathcal{S}_{\mathbf{t}^{(i)}}$ to have any chance of success. If $\mathbf{t}^{(Q+1)}$ is chosen arbitrarily among the elements of $\mathcal{T}$ such that $\mathcal{S}_{\mathbf{t}^{(Q+1)}} \setminus \bigcup_{i=1}^Q \mathcal{S}_{\mathbf{t}^{(i)}}$ is maximal, then the probability of success is

$$P[\mathcal{O}(\mathbf{t}^{(Q+1)})] \leq \frac{\#(\mathcal{S}_{\mathbf{t}^{(Q+1)}} \setminus \bigcup_{i=1}^Q \mathcal{S}_{\mathbf{t}^{(i)}})}{\#\mathcal{S} - \#\bigcup_{i=1}^Q \mathcal{S}_{\mathbf{t}^{(i)}}} \leq \frac{\kappa_{\mathcal{T}}}{\#\mathcal{S} - \kappa_{\mathcal{T}}Q}. \quad (3.8)$$

For EUF-CMA security with parameter μ against this adversary, we need to ensure the success probability after Q queries is $\leq 2^{-\mu}$, so

$$\min(\#\mathcal{S}/\kappa_{\mathcal{T}}, \#(\mathcal{M}/\mathcal{K})) \geq 2^{\mu} + Q. \quad (3.9)$$

We should therefore sample $\mathcal{K}$ from an $\mathcal{S}$ chosen such that (at least)

$$\#\mathcal{S}/\kappa_{\mathcal{T}} \geq 2^{\mu} \quad \text{and} \quad \#(\mathcal{M}/\mathcal{K}) \geq 2^{\mu} \text{ for each } \mathcal{K} \in \mathcal{S}, \quad (3.10)$$

and we should replace the verification key when the number of (failed) compressed verifications approaches 2^{μ} .

Heuristically, we will suppose that the adversary cannot improve on any strategy that simply enumerates queries $\mathbf{t}^{(i+1)}$ while maximising $\#\mathcal{S}_{\mathbf{t}^{(i+1)}} \setminus \bigcup_{j=1}^i \mathcal{S}_{\mathbf{t}^{(j)}}$. Indeed, to do so they would need more information on $\phi(\mathbf{x})$ for unqueried $\mathbf{x}$. Our heuristic is that this information has to come from exploiting the $\mathcal{R}$ -module structures of $\mathcal{M}$ and $\mathcal{M}/\mathcal{K}$, but these structures generally do not help.

First, $(\phi(\mathbf{x}) \neq \mathbf{0}) \wedge (\phi(\mathbf{y}) \neq \mathbf{0}) \not\Rightarrow \phi(\mathbf{x} + \mathbf{y}) \neq \mathbf{0}$ unless $\mathbf{y} \in \mathcal{R}\mathbf{x}$ or $\mathbf{x} \in \mathcal{R}\mathbf{y}$. This tells us that given the results of $\mathcal{O}(\mathbf{t}^{(i)})$ for $1 \leq i \leq Q$, we cannot predict the result of $\mathcal{O}(\mathbf{t}^{(Q+1)})$ for general linear combinations $\mathbf{t}^{(Q+1)} = \sum_{i=1}^Q \alpha_i \mathbf{t}^{(i)}$.

Looking at scalar multiplication, there are two cases.

1. If $\alpha \neq 0 \in \mathcal{R}$ is invertible on $\mathcal{M}/\mathcal{K}$, then $\phi(\mathbf{x}) = \mathbf{0} \iff \phi(\alpha\mathbf{x}) = \mathbf{0}$ for all $\mathbf{x}$. In this case, if we know $\mathcal{O}(\mathbf{x}) = \text{False}$, then we can predict that $\mathcal{O}(\alpha\mathbf{x}) = \text{False}$ *without* querying $\mathcal{O}$ on $\alpha\mathbf{x}$ (and vice versa). But these “virtual” queries can not help the adversary, because $\mathcal{S}_{\alpha\mathbf{x}} = \mathcal{S}_{\mathbf{x}}$ for all such α .
2. If $\alpha \neq 0 \in \mathcal{R}$ is *not* invertible on $\mathcal{M}/\mathcal{K}$, then $\phi(\mathbf{x}) = \mathbf{0} \implies \phi(\alpha\mathbf{x}) = \mathbf{0}$ for all $\mathbf{x}$, but the converse does not hold; likewise, $\mathcal{S}_{\mathbf{x}} \subset \mathcal{S}_{\alpha\mathbf{x}}$ but the inclusion may be strict. If such elements α are known, then the adversary should query on $\alpha\mathbf{x}$ instead of $\mathbf{x}$ to maximise $\#\mathcal{S}_{\alpha\mathbf{x}}$ (and hence $\#\mathcal{S}_{\mathbf{t}^{(i+1)}} \setminus \bigcup_{j=1}^i \mathcal{S}_{\mathbf{t}^{(j)}}$). If these α exist but are *not* known to the adversary, then they should try to guess them in order to approach the ideal bound of (3.8).

In our application to WAVE, $\mathcal{R} = \mathbb{F}_3$ is a field, so we are always in the first situation. For Rabin–Williams and SQUIRRELS, $\mathcal{R} = \mathbb{Z}$ and $\mathcal{M}/\mathcal{K} = \mathbb{Z}/d\mathbb{Z}$ for some d unknown to the adversary. Every query is $\alpha \cdot 1$ for some α , and the adversary’s goal is precisely to find $\alpha \neq 0$ divisible by the unknown d : that is, they are (or are trying to be) in the second situation.

Remark 3.3.4 *The bounds in (3.10) may be overly pessimistic: even if $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ is hard, forging signatures in $\Sigma_{\text{comp}}^{\mathcal{K}}$ for random $\mathcal{K}$ in $\mathcal{S}$ may be significantly harder. With GPV-style signatures, a solution $\mathbf{t}$ to the $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ instance for $\mathcal{K}$ gives a forgery against $\Sigma_{\text{comp}}^{\mathcal{K}}$ only if we can construct $(m, \sigma = (\text{salt}, \mathbf{s}))$ mapping to $\mathbf{t}$; and this is made difficult by the need to satisfy $\text{CONSTRAINT}(\mathbf{s})$. After all, if we could find (m, σ) for arbitrary $\mathbf{t}$ then we could find them for $\mathbf{0}$, and thus construct forgeries in the original scheme Σ . For a computationally bounded adversary, it may be infeasible to construct (m, σ) yielding implicit queries $\mathbf{t}$ with large $\#\mathcal{S}_{\mathbf{t}}$, which means the success probability is actually much lower—and then we can make $\mathcal{S}$ (and thus, potentially, $|\text{VK}|$) much smaller. Hence, while setting parameters to make SEGP hard will guarantee unforgeability, these parameters may also be much larger than what is required for EUF-CMA in practice.*

3.4 Compressed verification for Squirrels

Now we turn our attention to SQUIRRELS. The challenge here is to define compressed verification algorithms that, like SQUIRRELS, avoid multiprecision arithmetic. To simplify presentation, we use the following notation:

Definition 3.4.1 *Given a list of primes $\mathbf{m} = (m_1, \dots, m_s)$, we write*

$$[[x]]_{\mathbf{m}} := (x \bmod m_1, \dots, x \bmod m_s) \quad \text{for all } x \in \mathbb{Z}.$$

3.4.1 The Squirrels signature scheme

SQUIRRELS is a GPV signature on unstructured lattices. More precisely, it uses co-cyclic lattices: n -dimensional lattices $\mathcal{L}$ such that $\mathbb{Z}^n/\mathcal{L} = \mathbb{Z}/\Delta\mathbb{Z}$ for some Δ . Co-cyclic lattices are dominant among full-rank integer lattices [87]: their natural density is $\approx 85\%$. SQUIRRELS works with co-cyclic lattices $\mathcal{L}$ of determinant

$$\Delta := p_1 \cdots p_s$$

where $\mathbf{p} = (p_1, \dots, p_s)$ is a fixed tuple of 31-bit primes (the length s depends on the security parameter). Table 3.2 gives the SQUIRRELS parameter sets.

Table 3.2: Parameters for SQUIRRELS instances.

NIST Security Level	1	2	3	4	5
Lattice dimension n	1034	1164	1556	1718	2056
Hash space size q	4096	4096	4096	4096	4096
Max. signature square norm $\lfloor \beta^2 \rfloor$	2 026 590	2 442 439	4 512 242	3 659 372	5 370 115
Number s of small primes	165	188	262	275	339
Bitlength of Δ	5048	5738	8017	8402	10347
Signature Size (B)	1019	1147	1554	1676	2025
Public Key Size (B)	681 780	874 576	1 629 640	1 888 700	2 786 580

SQUIRRELS is built on the one-way function

$$\begin{aligned} f : D_n &\longrightarrow \mathbb{Z}/\Delta\mathbb{Z} \\ \mathbf{x} &\longmapsto \mathbf{x}\mathbf{A}^T \pmod{\Delta}, \end{aligned} \tag{3.11}$$

where $\mathbf{A}$ is the matrix defining $\mathcal{L}$ and $D_n = \{\mathbf{e} \in \mathbb{Z}^n \mid \|\mathbf{e}\| \leq \beta\}$. One-wayness depends on the hardness of $GSIS_{n,\Delta,\beta}$, that is, finding a vector $\mathbf{x}$ such that $\mathbf{x}\mathbf{A}^T \equiv 0 \pmod{\Delta}$ and $\|\mathbf{x}\| \leq \beta$ for some small β .

Suppose $\mathcal{L}$ is co-cyclic of dimension n and determinant Δ . We can specify $\mathcal{L}$ with the row-HNF of its generating matrix, which is determined by a vector

$$\mathbf{v}_{\text{check}} = (\mathbf{v}_{\text{check},1}, \dots, \mathbf{v}_{\text{check},n}) \in (\mathbb{Z}/\Delta\mathbb{Z})^n \quad \text{with} \quad \mathbf{v}_{\text{check},n} = -1.$$

The public key encodes $\mathbf{v}_{\text{check}}$ as a list of lists of residues mod the small primes:

$$\text{PK} = ((v_{i,j} := \mathbf{v}_{\text{check},i} \bmod p_j)_{i=1}^{n-1})_{j=1}^s$$

(since $\mathbf{v}_{\text{check},n} = -1$ by convention, there is no need to store it or any of the $v_{n,j}$). The $v_{i,j}$ are encoded as *signed* twos-complement 32-bit integers, but they are all non-negative

(except the $v_{n,j}$, which are all -1 and not stored anyway); in particular, $0 \leq v_{i,j} < 2^{31}$ for all $1 \leq i < n$ and $1 \leq j \leq s$.

The private key encodes a “good” basis for $\mathcal{L}$, which allows sampling short vectors in $\mathcal{L}$ following a Gaussian distribution using Klein’s trapdoor sampler. **KeyGen** ensures that each $\mathbf{v}_{\text{check},i}$ looks like a uniform random integer modulo Δ .

We now focus on SQUIRRELS verification (**KeyGen** and **Sign** are detailed in [51]). The verifier accepts $\sigma = (\mathbf{s}, \text{salt})$ if two conditions are met:

1. The vector $\mathbf{s}$ is short: $\|\mathbf{s}\| \leq \beta$, which is more easily checked as

$$\text{CONSTRAINT}(\mathbf{s}) : \|\mathbf{s}\|^2 \leq \lfloor \beta^2 \rfloor.$$

2. The vector $\mathbf{c} := \mathbf{s} + \mathbf{h}$ (where $\mathbf{h} := \text{Hash}(\text{salt} \parallel \mathbf{m})$) is in $\mathcal{L}$. That is,

$$\sum_{i=1}^n c_i \mathbf{v}_{\text{check},i} \equiv 0 \pmod{\Delta}, \quad (3.12)$$

or equivalently (by the CRT)

$$\sum_{i=1}^n c_i v_{i,j} \equiv 0 \pmod{p_j} \quad \text{for all } 1 \leq j \leq s. \quad (3.13)$$

We can thus verify by checking (3.13) for each of the p_j in turn, as in Algorithm 2.

Algorithm 2 Verification algorithm for SQUIRRELS.

Parameters : $q, n, \lfloor \beta^2 \rfloor$, and $P_\Delta = (p_1, \dots, p_m)$

Input: Signature $\sigma = (\text{salt}, \mathbf{s})$, message m , public key $\text{PK} = ((v_{i,j})_{i=1}^{n-1})_{j=1}^m$ with $v_{i,j} = \mathbf{v}_{\text{check},i} \bmod p_j$

Output: Accept if σ is a valid signature on m under PK, otherwise Reject.

```

1  $\mathbf{s} \leftarrow \text{Decompress}(\mathbf{s})$   if  $\mathbf{s} = \perp$  or  $\|\mathbf{s}\|_2^2 > \lfloor \beta^2 \rfloor$  then
2   return Reject
3  $\mathbf{c} \leftarrow \mathbf{s} + \text{HashToPoint}(m \parallel \text{salt}, q, n)$   foreach  $1 \leq j \leq m$  do           // Trivially parallelizable
4    $S \leftarrow \sum_{i=0}^{n-1} c_i v_{i,j} \bmod p_j$   if  $S - c_n \neq 0 \bmod p_j$  then           // Uses  $\mathbf{v}_{\text{check},n} = -1$ 
5   return Reject
6 return Accept
```

3.4.2 Homomorphisms for Squirrels verification

SQUIRRELS is an instance of our general framework with $\mathcal{R} = \mathbb{Z}$ and $\mathcal{M} = \mathbb{Z}/\Delta\mathbb{Z}$. But the only homomorphisms from $\mathbb{Z}/\Delta\mathbb{Z}$ map through $\mathbb{Z}/\Delta'\mathbb{Z}$ for $\Delta'|\Delta$, and while a verifier could choose a secret ϕ by choosing a secret subset of the p_j , an adversary who could forge a signature for a large subset of the p_j would fool many verifiers. There are many more homomorphisms from $\mathbb{Z}$, and we can lift SQUIRRELS trivially to $\mathcal{M} = \mathbb{Z}$ if we replace (3.12) with the equivalent condition

$$\sum_{i=1}^n c_i \cdot \mathbf{v}_{\text{check},i} = k\Delta \quad \text{for some integer } k. \quad (3.14)$$

Let the verifier choose a secret list of secret 31-bit primes $\mathbf{r} = (r_1, \dots, r_t)$, each prime to Δ . The parameter t is a function of the desired verification security level, to be determined later in Section 3.4.5. Now the verifier could check

$$\sum_{i=1}^n c_i \cdot (\mathbf{v}_{\text{check},i} \bmod r_j) \equiv k(\Delta \bmod r_j) \pmod{r_j} \quad \text{for } 1 \leq j \leq t \quad (3.15)$$

—but k is not included in the signature, and re-computing it is the same computation as a full SQUIRRELS verification. Instead, we will verify by implicitly recovering k modulo $\prod_j r_j$, and checking that it is in the appropriate range.

First, we need to compute each of the $[[\mathbf{v}_{\text{check},i}]]_{\mathbf{r}}$ from the $[[\mathbf{v}_{\text{check},i}]]_{\mathbf{p}} = (v_{i,j} = \mathbf{v}_{\text{check},i} \bmod p_j)_{j=1}^s$. In the spirit of SQUIRRELS, we want to avoid multiprecision integer arithmetic, so we need to compute the $[[\mathbf{v}_{\text{check},i}]]_{\mathbf{r}}$ *without reconstructing* any of the integers $\mathbf{v}_{\text{check},i}$. Our main tool is the *explicit CRT*.⁴

Definition 3.4.2 *With the notation above: for each $1 \leq i \leq s$, we write Δ_i for Δ/p_i , and let q_i be the the unique integer in $(1, p_i)$ such that $q_i \Delta_i \equiv 1 \pmod{p_i}$.*

We will never explicitly compute with the Δ_i (they are a notational convenience). We *will* need the q_i , and these can be precomputed in advance (using e.g. Algorithm 3). Each is a positive 31-bit integer.

We maintain the notation of Section 3.4: $\mathbf{p} = (p_1, \dots, p_s)$ is a list of distinct primes, $\Delta := \prod_{i=1}^s p_i$ is their product, $\Delta_i := \Delta/p_i$ and $q_i := \Delta_i^{-1} \pmod{p_i}$ for $1 \leq i \leq s$. Algorithm 3 computes $(q_1, \dots, q_s)$. Algorithm 4 computes $([[\Delta_i]]_{\mathbf{r}})_{i=1}^s$ and $[[\Delta]]_{\mathbf{r}}$ given another list of primes $\mathbf{r} = (r_1, \dots, r_t)$ (all prime to Δ). These algorithms are not optimal, but they avoid multiprecision arithmetic.

Algorithm 3 Explicit Modular CRT setup: q -coefficients.

Input: $\mathbf{p} = (p_1, \dots, p_s)$

Output: $(q_1, \dots, q_s)$ s.t. $0 < q_i < p_i$ and $q_i(\Delta/p_i) \equiv 1 \pmod{p_i}$ for $1 \leq i \leq s$

```

7 function qCoefficients( $(p_1, \dots, p_s)$ )
8    $(q_1, \dots, q_s) \leftarrow (1, \dots, 1)$  foreach  $1 \leq i \leq s$  do
9     foreach  $1 \leq j \leq s, j \neq i$  do
10       $q_i \leftarrow q_i \cdot p_j \bmod p_i$ 
11       $q_i \leftarrow q_i^{-1} \bmod p_i$ 
12 return  $(q_1, \dots, q_s)$ 
```

⁴This is similar to the modular reduction in RNS arithmetic in [12], but there, $\mathbf{p}$ can be freely chosen to optimise computations on the operands; here, $\mathbf{p}$ is fixed.

Algorithm 4 Explicit Modular CRT setup.

Input: $\mathbf{p} = (p_i)_{i=1}^s$ and $\mathbf{r} = (r_j)_{j=1}^t$.

Output: $([\Delta_i]_{\mathbf{r}})_{i=1}^s$ and $[\Delta]_{\mathbf{r}}$.

```

13 function ModECRTSetup( $\mathbf{p}, \mathbf{r}$ )
14    $\mathbf{m} \leftarrow [[1]]_{\mathbf{r}}$   ( $\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(s)} \leftarrow ([1]]_{\mathbf{r}}, \dots, [[1]]_{\mathbf{r}}$ )  foreach  $1 \leq i \leq s$  do
15      $\mathbf{u} \leftarrow [[p_i]]_{\mathbf{r}}$   //  $u_j = p_j$  or  $p_j - r_j$ 
16      $\mathbf{m} \leftarrow (m_1 u_1 \bmod r_1, \dots, m_t u_t \bmod r_t)$   foreach  $1 \leq j < i$  and  $i < j \leq s$  do
17        $\mathbf{c}^{(j)} \leftarrow (c_1^{(j)} u_1 \bmod r_1, \dots, c_t^{(j)} u_t \bmod r_t)$ 
18   return  $(\mathbf{c}^{(1)}, \dots, \mathbf{c}^{(s)}), \mathbf{m}$ 

```

Lemma 3.4.3 (Explicit CRT) *If $0 \leq x < \Delta$ and $(x_1, \dots, x_s) = [[x]]_{\mathbf{p}}$, then*

$$x = \alpha\Delta - \lfloor \alpha \rfloor \Delta \quad \text{where} \quad \alpha = \sum_{i=1}^s x_i q_i / p_i. \quad (3.16)$$

Proof. Observe that α is a rational number, but $\alpha\Delta$ is an integer. The CRT says $x \equiv \alpha\Delta \pmod{\Delta}$, so obviously $\alpha\Delta - \lfloor \alpha \rfloor \Delta \equiv x \pmod{\Delta}$. But $0 \leq \alpha - \lfloor \alpha \rfloor < 1$ by construction, so $0 \leq \alpha\Delta - \lfloor \alpha \rfloor \Delta < \Delta$, so $\alpha\Delta - \lfloor \alpha \rfloor \Delta = x$. $\square$

Lemma 3.4.3 gives an exact expression for $0 \leq x < \Delta$ in terms of $[[x]]_{\mathbf{p}}$ that we can use to compute $[[x]]_{\mathbf{r}}$ by computing the integers $\alpha\Delta$ and $\lfloor \alpha \rfloor \Delta$ modulo each r_j . We precompute the q_i , $[\Delta_i]_{\mathbf{r}}$, and $[\Delta]_{\mathbf{r}}$. Computing $\alpha\Delta = \sum_i x_i q_i \Delta_i$ modulo r_j is straightforward. The interesting part is determining $\lfloor \alpha \rfloor$, and thus computing $\lfloor \alpha \rfloor \bmod r_j$, without computing α . We will do this using fixed-point approximations, as in [12]. Lemma 3.4.4, an adaptation of [12, Lemma 3.1], shows that with a relatively low precision we can determine $\lfloor \alpha \rfloor$ up to a possible error of 1.

Lemma 3.4.4 *Let $\alpha_1, \dots, \alpha_s$ be non-negative real numbers, and set $\alpha := \sum_{j=1}^s \alpha_j$. Fix some integer $a \geq \log_2 s + 1$. Then*

$$f := \left\lfloor \frac{s}{2^a} + \frac{1}{2^a} \sum_{j=1}^s \lfloor 2^a \alpha_j \rfloor \right\rfloor \quad \text{is either } \lfloor \alpha \rfloor \text{ or } \lfloor \alpha \rfloor + 1.$$

Further, if $\alpha - \lfloor \alpha \rfloor < 1 - s/2^a$ then f is exactly $\lfloor \alpha \rfloor$.

Proof. Note that $2^a \geq 2s$, so $s/2^a \leq 1/2$. Let $q := (1/2^a) \sum_j \lfloor 2^a \alpha_j \rfloor$. By construction, $0 \leq 2^a \alpha_j - \lfloor 2^a \alpha_j \rfloor < 1$ for each $1 \leq j \leq s$. Summing over j gives $0 \leq 2^a \alpha - 2^a q < s$, so $0 \leq \alpha - q < s/2^a$; so $\lfloor \alpha \rfloor < s/2^a + q \leq s/2^a + \alpha$. Taking floors gives $\lfloor \alpha \rfloor \leq f \leq \lfloor s/2^a + \alpha \rfloor$. But $\lfloor s/2^a + \alpha \rfloor$ is either $\lfloor \alpha \rfloor$ or $\lfloor \alpha \rfloor + 1$, because $0 < s/2^a \leq 1/2$, proving the first statement. For the second, if $\alpha - \lfloor \alpha \rfloor < 1 - s/2^a$ then $\alpha + s/2^a < \lfloor \alpha \rfloor + 1$, so $\lfloor \alpha + s/2^a \rfloor = \lfloor \alpha \rfloor$, and thus $f = \lfloor \alpha \rfloor$. $\square$

Theorem 3.4.5 Fix an integer $a \geq \log_2(s) + 1$. Given $\mathbf{r}$, $[[\Delta]]_{\mathbf{r}}$, $([[\Delta_i]]_{\mathbf{r}})_{i=1}^s$, and $[[x]]_{\mathbf{p}}$ for some $0 \leq x < \Delta$, Algorithm 5 returns $[[z]]_{\mathbf{r}}$ where z is either x or $x - \Delta$. Further: if $x < (1 - s/2^a)\Delta$, then $z = x$.

Proof. Algorithm 5 evaluates (3.16) modulo r_j for $1 \leq j \leq t$, computing the floor using Lemma 3.4.4 with $\alpha_i = x_i q_i / p_i$ for $1 \leq i \leq s$. $\square$

Algorithm 5 Explicit CRT: computing $[[x]]_{\mathbf{r}}$ or $[[x - \Delta]]_{\mathbf{r}}$ from $[[x]]_{\mathbf{p}}$

Parameters : SQUIRRELS prime vector $\mathbf{p} = (p_1, \dots, p_s)$ and CRT coefficients $(q_1, \dots, q_s)$; an integer $a \geq \log_2 s + 1$ (fixed-point precision)

Input: Secret prime list $\mathbf{r}$; $[[\Delta]]_{\mathbf{r}}$, $([[\Delta_i]]_{\mathbf{r}})_{i=1}^s$, and $[[x]]_{\mathbf{p}}$ for $0 \leq x < \Delta$

Output: $[[z]]_{\mathbf{r}}$, where $z = x$ or $x - \Delta$ (if $x < (1 - s/2^a)\Delta$, then $z = x$)

```

19 function ModECRT( $\mathbf{r}$ ,  $[[\Delta]]_{\mathbf{r}}$ ,  $([[\Delta_i]]_{\mathbf{r}})_{i=1}^s$ ,  $[[x]]_{\mathbf{p}}$ )
20    $(z_1, \dots, z_t) \leftarrow (0, \dots, 0)$     $f \leftarrow s$    foreach  $1 \leq j \leq s$  do
21      $y_j \leftarrow x_j \cdot q_j$     $f \leftarrow f + \lfloor 2^a y_j / p_j \rfloor$            // See Remark 3.4.7
22     foreach  $1 \leq k \leq t$  do
23        $y_{j,k} \leftarrow y_j \bmod r_k$     $z_k \leftarrow (z_k + y_{j,k} \cdot \Delta_{j,k}) \bmod r_k$ 
24    $f \leftarrow \lfloor f / 2^a \rfloor$            // Right-shift  $f$  by  $a$ 
25   foreach  $1 \leq k \leq t$  do
26      $z_k \leftarrow (z_k - f \cdot \Delta_k) \bmod r_k$ 
27   return  $(z_1, \dots, z_t)$ 

```

Remark 3.4.6 Theorem 3.4.5 recovers $[[x]]_{\mathbf{r}}$ or $[[x - \Delta]]_{\mathbf{r}}$ from $[[x]]_{\mathbf{p}}$. To guarantee a result of $[[x]]_{\mathbf{r}}$ requires increasing the precision to $a \sim \log_2 \Delta$, which means working with integers the size of Δ ; but then we may as well reconstruct x .

Remark 3.4.7 As noted in [12], the floors in Line 21 of Algorithm 5 can be computed by repeatedly doubling y_j modulo p_j and counting overflows.

3.4.3 Compressed verification for Squirrels

Recall that our goal is to verify (3.14) by evaluating (3.15), namely

$$\sum_{i=1}^n c_i \cdot \mathbf{v}_{\text{check},i} \equiv k\Delta \pmod{r_j} \quad \text{for each } 1 \leq j \leq t,$$

but *without* knowing k . Given a public key PK and a compression key $\text{CK} := (\mathbf{r}, ([[\Delta_i]]_{\mathbf{r}})_{i=1}^s, [[\Delta]]_{\mathbf{r}}, (I_1, \dots, I_t))$, for each $1 \leq i \leq n$ we use ModECRT to compute

$$\begin{aligned} [[\bar{v}_i]]_{\mathbf{r}} &:= \text{ModECRT}(\mathbf{r}, [[\Delta]]_{\mathbf{r}}, ([[\Delta_j]]_{\mathbf{r}})_{j=1}^s, [[\mathbf{v}_{\text{check},i}]]_{\mathbf{p}} = (v_{i,j})_{j=1}^s) \\ &= [[\mathbf{v}_{\text{check},i} - \epsilon_i \Delta]]_{\mathbf{r}} \quad \text{where } \epsilon_i \in \{0, 1\} \text{ is unknown.} \end{aligned}$$

The r_j are chosen such that $r_j \nmid \Delta$, so if we let

$$k'_j := \left(\sum_{i=1}^n c_i \bar{v}_{i,j} \right) I_j \bmod r_j \quad \text{where } I_j := \Delta^{-1} \bmod r_j \quad \text{for } 1 \leq j \leq t,$$

then the system of verification equations (3.15) becomes

$$k'_j \equiv k' \pmod{r_j} \quad \text{where} \quad k' := k + \sum_{i=1}^{n-1} \epsilon_i c_i. \quad (3.17)$$

Lemma 3.4.8 below shows that

- If (3.14) holds, then k' is a small integer (Table 3.3 gives bounds on k' for each SQUIRRELS instance), small enough that $k'_j = k'$ as an integer for each j .
- If (3.14) does not hold, then k' does not exist, and the k'_j look like random (and generally large) values modulo each of the r_j .

This distinction is the basis of our **CVerify** for SQUIRRELS: we compute $(k'_1, \dots, k'_t)$, and **Accept** if $k'_1 = \dots = k'_t$ and k'_1 is within the bounds on k' ; otherwise, we **Reject**. Note that even if the adversary has full control over the c_i , they cannot control the k'_j , because $\mathbf{r}$ and the I_j are unknown.

Lemma 3.4.8 *With SQUIRRELS parameters $q \ll \Delta$ and $\lfloor \beta^2 \rfloor \ll \Delta$: if $\|\mathbf{s}\|_2^2 \leq \lfloor \beta^2 \rfloor$, then the integer k' of (3.17) satisfies*

$$k'_{\min} \leq k' \leq k'_{\max} \quad \text{where} \quad \begin{cases} k'_{\min} := -\lfloor 2\sqrt{n\lfloor \beta^2 \rfloor} \rfloor - 1, \\ k'_{\max} := 2(n-1)(q-1) + \lfloor 2\sqrt{n\lfloor \beta^2 \rfloor} \rfloor + 1. \end{cases}$$

Proof. By definition,

$$k' = \left(\sum_{i=1}^{n-1} c_i \beta_i \right) - \frac{c_n}{\Delta} \quad \text{with} \quad \beta_i = \frac{\mathbf{v}_{\text{check},i}}{\Delta} + \epsilon_i$$

for $1 \leq i < n$, so

$$k' + E = H + S$$

where

$$H = \sum_{i=1}^{n-1} h_i \beta_i, \quad S = \sum_{i=1}^{n-1} s_i \beta_i, \quad E = \frac{c_n}{\Delta}.$$

For $1 \leq i < n$, we have $0 \leq \mathbf{v}_{\text{check},i} < \Delta$, so $0 \leq \beta_i < 2$.

Thus,

$$-S' - E < k' < H' + S' + E$$

where

$$H' = 2 \sum_{i=1}^{n-1} h_i \quad \text{and} \quad S' = 2 \sum_{i=1}^n |s_i|$$

Table 3.3: Values of $k'_{\min}$ and $k'_{\max}$ from Lemma 3.4.8 for the SQUIRRELS instances in [51]. Note that $k'_{\max} - k'_{\min}$ is a 24-bit integer except for SQUIRRELS-V, where it is 25 bits.

Instance	SQUIRRELS-I	SQUIRRELS-II	SQUIRRELS-III	SQUIRRELS-IV	SQUIRRELS-V
$k'_{\min}$	-91554	-106640	-144446	-15879	-210152
$k'_{\max}$	8551824	9631610	9603896	14220809	17040602

(note: we include $|s_n|$ in S').

But $0 \leq |E| \leq \frac{h_n + |s_n|}{\Delta} < 1$ because

$$h_n < q \ll \Delta \quad \text{and} \quad |s_n| \leq \lfloor \beta^2 \rfloor \ll \Delta,$$

so

$$-S' - 1 < k' < H' + S' + 1.$$

Now,

$$0 \leq S' \leq 2\sqrt{n}\|\mathbf{s}\|_2 \leq 2\sqrt{n\lfloor \beta^2 \rfloor}, \quad 0 \leq H' \leq 2(n-1)(q-1)$$

and k' is an integer, so

$$\left\lceil -2\sqrt{n\lfloor \beta^2 \rfloor} \right\rceil - 1 \leq k' \leq 2(n-1)(q-1) + \left\lfloor 2\sqrt{n\lfloor \beta^2 \rfloor} \right\rfloor + 1,$$

and the result follows. $\square$

The verification key VK is $(\mathbf{r}, (I_1, \dots, I_t), ([\bar{v}_i]_{\mathbf{r}})_{i=1}^{n-1})$, so

$$|\text{VK}| = 4(n+1)t \text{ bytes} \quad \text{and} \quad |\text{PK}| = 4(n-1)s \text{ bytes}.$$

The key-compression ratio is $|\text{PK}| : |\text{VK}| \approx s : t$. See Table 3.4 for sample values.

The compression key CK requires $4(s+3)t$ bytes. The values of $([\Delta_i]_{\mathbf{r}})_{i=1}^s$, $[\Delta]_{\mathbf{r}}$, and $(I_1, \dots, I_t)$ may be left out of CK, thus reducing $|\text{CK}|$ to $4s$ bytes, but this implies recomputing them from $\mathbf{r}$ and $\mathbf{p}$ for every verification key generation.

3.4.4 The algorithms

Algorithms 6, 7, and 8 formalise CKeyGen, VKeyGen, and CVerify for SQUIRRELS.

Algorithm 6 CKeyGen (compression key generation) for SQUIRRELS.

Parameters : $n, s, \Delta, \mathbf{p}, (\Delta_1, \dots, \Delta_s), [[\Delta_i]]_{\mathbf{p}}, t$

Output: CK

28 Sample a list $\mathbf{r} = (r_1, \dots, r_t)$ of random 31-bit primes

// Use Lemma 3.4.9

29 Compute $([[\Delta_i]_{\mathbf{r}}]_{i=1}^s)$ and $[\Delta]_{\mathbf{r}}$ from $\mathbf{p}$

// Using e.g. Algorithm 4.

30 for $1 \leq j \leq t$ do

31 $I_j \leftarrow \Delta^{-1} \bmod r_j$

// Use $I_j = (\Delta \bmod r_j)^{-1} \bmod r_j$

32 return CK := $(\mathbf{r}, ([[\Delta_i]_{\mathbf{r}}]_{i=1}^s, [\Delta]_{\mathbf{r}}, (I_1, \dots, I_t))$

Algorithm 6 requires randomly sampling 31-bit primes, which is easy using the criteria of [98]. Recall that an odd integer $r = d2^u + 1$ is a *strong pseudoprime to the base a* if $a^d \equiv 1 \pmod{r}$ or $a^{d2^v} \equiv -1 \pmod{r}$ for some $0 \leq v < u$.

Lemma 3.4.9 *Let $2^{30} < r < 2^{31}$ be odd. If r is a strong pseudoprime to the bases 2, 3, and 5, then either r is prime or $r = 1157839381 = 24061 \cdot 48121$.*

Proof. See [98, Page 1022]. □

Algorithm 7 VKeyGen (verification key generation) for SQUIRRELS.

Parameters : $n, s, \Delta, \mathbf{p}, (\Delta_1, \dots, \Delta_s), [[\Delta_i]]_{\mathbf{p}}, t$
Input: CK = $(\mathbf{r}, ([[\Delta_i]]_{\mathbf{r}})_{i=1}^s, [[\Delta]]_{\mathbf{r}}, (I_1, \dots, I_t))$, PK = $(\mathbf{v}_{\text{check}, i})_{i=1}^{n-1}$
Output: VK
33 **foreach** $1 \leq i < n$ **do** // May be parallelized
34 $[[\bar{v}_i]]_{\mathbf{r}} \leftarrow \text{ModECRT}(\mathbf{r}, [[\Delta]]_{\mathbf{r}}, ([[\Delta_i]]_{\mathbf{r}})_{i=1}^t, \mathbf{v}_{\text{check}, i})$ // $=[[\mathbf{v}_{\text{check}, i} + \epsilon_i \Delta]]_{\mathbf{r}}$
35 **return** $(\mathbf{r}, (I_1, \dots, I_t), (\bar{v}_{i,1}, \dots, \bar{v}_{i,t})_{i=1}^{n-1})$

Algorithm 8 defines CVerify. Lines 38-45 must be implemented with constant-time techniques to ensure no information on $\mathbf{r}$ is leaked in the event of rejection⁵.

Algorithm 8 CVerify (compressed verification) for SQUIRRELS.

Parameters : $n, s, \Delta, \mathbf{p}, (\Delta_1, \dots, \Delta_s), [[\Delta_i]]_{\mathbf{p}}, t$
Input: $\sigma = (\text{salt}, \underline{s})$, m , VK = $(\mathbf{r}, (I_1, \dots, I_t), (\bar{v}_{i,1}, \dots, \bar{v}_{i,t})_{i=1}^n)$
Output: Accept or Reject
36 $\mathbf{s} \leftarrow \text{Decompress}(\underline{s})$ **if** $\mathbf{s} = \perp$ **or** $\|\mathbf{s}\|_2^2 > \lfloor \beta^2 \rfloor$ **then**
37 **return** Reject
38 $\mathbf{c} \leftarrow \mathbf{s} + \text{HashToPoint}(m \parallel \text{salt}, q, n)$ $(a_1, \dots, a_t) \leftarrow (\text{True}, \dots, \text{True})$ **foreach** $1 \leq j \leq t$ **do** // May be parallelized
39 $k'_j \leftarrow ((\sum_{i=1}^n c_i \bar{v}_{i,j}) I_j - k'_{\min}) \bmod r_j$ // $k'_{\min}$: see Lemma 3.4.8
40 **if** $k'_j > k'_{\max} - k'_{\min}$ **then** // $k'_{\max} - k'_{\min}$: see Lemma 3.4.8
41 $a_j \leftarrow \text{False}$
42 **if** $(a_1 \wedge \dots \wedge a_t) \wedge (k'_1 = \dots = k'_t)$ **then**
43 **return** Accept
44 **else**
45 **return** Reject

3.4.5 Security argument

If Σ is SQUIRRELS, then $\mathcal{R} = \mathcal{M} = \mathbb{Z}$. Compressed SQUIRRELS samples $\mathcal{K}$ from

$$\mathcal{S} = \{\Delta' \mathbb{Z} : \Delta' \text{ is a product of } t \text{ 31-bit primes}\}.$$

⁵The implementation can be made constant-time by using constant-time arithmetic routines (e.g. multiplication/modular reduction [16]) and replacing any conditional branches with bit-masking operations.

Theorem 3.3.3 ensures EUF-CMA security for compressed SQUIRRELS provided $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ is hard. For SQUIRRELS parameters, $\mathcal{T}(\Sigma) \subset [0, 2q\sqrt{n}\lfloor\beta^2\Delta\rfloor] \subset [0, 2^{30}\Delta]$: at most s 31-bit primes can divide any $\mathbf{t} \in \mathcal{T}(\Sigma)$, so $\kappa_{\mathcal{T}} \approx s!/(t!(s-t)!)$. Now $\#\mathcal{S} = P_{31}!/(t!(P_{31}-t)!)$ where $P_{31} := \#\text{PRIMES}(31)$, and $\#(\mathcal{M}/\mathcal{K}) \approx 2^{31t}$. When $t \ll s$, we have $s!/(s-t)! \sim s^t$ and $P_{31}!/(P_{31}-t)! \sim P_{31}^t$. Looking at [121, Table 3], we find that

$$P_{31} := \#\text{PRIMES}(31) = 105097565 - 54400028 \approx 2^{25.6}.$$

The heuristic of Section 3.3.3 therefore suggests taking $t \approx \mu/(25.6 - \log_2(s))$, where μ is the targeted security level.

In reality, it is computationally infeasible to construct forgery attempts (m, σ) that yield $\mathbf{t}$ with $\#\mathcal{S}_{\mathbf{t}} \approx \kappa_{\mathcal{T}}$: this would mean constructing (m, σ) such that $\mathbf{t}$ is divisible by another product Δ' of s 31-bit primes, and this is essentially as hard as forging a full SQUIRRELS signature: that is, solving $\text{GSIS}_{n, \Delta, \beta}$. If we want an (m, σ) mapping to a $\mathbf{t}$ with k 31-bit prime factors, and thus $\#\mathcal{S}_{\mathbf{t}} = \binom{k}{t}/\#S$, we must solve a single GSIS instance with modulus $\Delta^* \ll \Delta$. Indeed, it requires a nontrivial computational effort to even construct a forgery attempt (m, σ) yielding $\mathbf{t}$ with $\#\mathcal{S}_{\mathbf{t}} > 0$. In our security estimates, we therefore model the expected value of $\#\mathcal{S}_{\mathbf{t}}$ as a small constant, and hence we choose t such that $P_{31}!/(t!(P_{31}-t)!)$ is on the order of 2^λ , which suggests taking t as in Table 3.4. While the resulting security level μ is slightly smaller than λ in some cases, it should be remembered that each forgery attempt requires an interaction with the verifier, and is therefore substantially more expensive than (e.g.) the AES circuit evaluations used to model post-quantum cryptographic attack costs.

Table 3.4: Suggested values for the number t of secret primes in $\mathbf{r}$.

Instance	Original scheme				Compressed verification				
	λ	s	$ \text{PK} $	$ \sigma $	t	μ	$ \text{CK} $	$ \text{VK} $	$ \text{PK} : \text{VK} $
SQUIRRELS-I	128	165	681780	1019	5	121.1	3360	20700	32.94
SQUIRRELS-II	128	188	874576	1147	5	121.1	3820	23300	37.54
SQUIRRELS-III	192	262	1629640	1554	8	189.5	8480	49824	32.71
SQUIRRELS-IV	192	275	1888700	1676	8	189.5	8896	55008	34.34
SQUIRRELS-V	256	339	2786580	2025	11	256.3	15048	90508	30.79

3.5 Compressed verification for Wave

WAVE is a GPV-style signature based on hard problems in ternary linear codes. Very briefly: a WAVE public key is a matrix $\text{PK} = \mathbf{R} \in \mathbb{F}_3^{k \times (n-k)}$ such that $\mathbf{M} = (\mathbf{I}_{n-k} | \mathbf{R})^\top$ in $\mathbb{F}_3^{n \times (n-k)}$ is a parity-check matrix for a permuted generalized $(U|U+V)$ -code C ;

knowledge of the relation between C and the component codes U and V is a trapdoor allowing the signer to generate vectors $\mathbf{s}$ in $\mathbb{F}_3^n$ such that

$$\text{CONSTRAINT}(\mathbf{s}) \quad \text{and} \quad \mathbf{s}\mathbf{M} = \text{Hash}(\text{salt}, m), \quad (3.18)$$

where $\text{CONSTRAINT}(\mathbf{s})$ is that $\mathbf{s}$ have a fixed, high weight w . An “original” WAVE signature (as in [46]) is $\sigma = (\text{salt}, \mathbf{s})$. In the WAVE NIST submission [5], $\mathbf{s}$ is truncated to its last k entries (the other $n - k$ entries are implicitly recovered in verification). The special form of $\mathbf{M} = (\mathbf{I}_{n-k} | \mathbf{R})^\top$ allows us to rewrite (3.18) as

$$\text{CONSTRAINT}(\mathbf{s}) \quad \text{and} \quad \mathbf{t}\mathbf{M} = \mathbf{0} \quad \text{where} \quad \mathbf{t} := \mathbf{s} - (\text{Hash}(\text{salt}, \mathbf{s}) | \mathbf{0}_k). \quad (3.19)$$

WAVE has exceptionally large public keys: for example, WAVE822, WAVE1249, and WAVE1644 require 3.5 MB, 7.5 MB, and 13 MB, respectively. This motivates the use of *compressed verification*, where the verifier privately replaces the large matrix $\mathbf{M}$ with a much smaller secret key $\mathbf{VK}$ derived via a compression matrix.

VKeyGen computes $\mathbf{VK} := (\mathbf{I}_{n-k} | \mathbf{R})^\top \mathbf{C}$, where $\mathbf{C}$ is the verifier’s compression matrix. Only the bottom $n - k - c$ rows of $\mathbf{VK}$ need to be stored, yielding a total size of $c(n - c)/4$ bytes. **CVerify** then replaces the full-rank check $\mathbf{t}\mathbf{M} = \mathbf{0}$ with a lower-dimensional test $\mathbf{t}\mathbf{VK} = \mathbf{0}$ over $\mathbb{F}_3^c$, as detailed in Algorithm 9. Compressed verification requires “original” WAVE signatures (as in [46]) and is incompatible with the truncated versions from [5]. Although full signatures can be recovered from truncated ones using the public key, doing so during verification defeats compression by reintroducing computational and storage overhead.

Algorithm 9 CVerify (compressed verification) for WAVE

Input: message m , signature $\sigma = (\text{salt}, \mathbf{s})$, and verification key $\mathbf{VK}$

Output: Accept or Reject

```

46 if Weight( $\mathbf{s}$ )  $\neq W$  then
47   return Reject
48  $\mathbf{t} \leftarrow \mathbf{s} - (\text{Hash}(\text{salt} \parallel m) \parallel \mathbf{0}_k)$     $\mathbf{r} \leftarrow \mathbf{t}\mathbf{VK}$    if  $\mathbf{r} \neq \mathbf{0}$  then
49   return Reject
50 return Accept

```

The security argument for compressed WAVE is particularly simple. Theorem 3.3.3 ensures that $\Sigma_{\text{comp}}^{\mathcal{K}}$ is EUF-CMA secure if $\text{SEGP}(\mathcal{S}, \mathcal{T}(\Sigma))$ is hard, where $\mathcal{S}$ is the set of codimension- c subspaces of $\mathcal{M} = \mathbb{F}_3^{n-k}$ and $\mathcal{T}(\Sigma) = \mathcal{M}$. We have $\#\mathcal{S} = \begin{bmatrix} n-k \\ n-k-c \end{bmatrix}_3$, where $[\cdot]_3$ is the 3-binomial coefficient, $\#(\mathcal{M}/\mathcal{K}) = 3^{n-k-c}$, and $\#\mathcal{S}_{\mathbf{t}} = \kappa_{\mathcal{T}(\Sigma)} = \begin{bmatrix} n-k-1 \\ n-k-c-1 \end{bmatrix}_3$ for every $\mathbf{t} \neq \mathbf{0} \in \mathcal{M}$. This tells us that a naive adversary that simply tries random forgery

Table 3.5: Parameters for (compressed) WAVE. Sizes are in bytes. WAVE signatures are variable-length: the values of $|\sigma|$ here are upper bounds, and $|\sigma|$ roughly doubles for the “original” (non-truncated) signatures required for compressed verification.

Instance	Security level		Original scheme		Compressed verification					
	NIST PQ	λ	$ \sigma $	$ \mathbf{PK} $	c	μ	$ \mathbf{CK} $	$ \mathbf{VK} $	$ \mathbf{PK} : \mathbf{VK} $	
WAVE822	1	128	822	3 677 390	80	126.8	83 822	171 594		21.4
WAVE1249	3	192	1249	7 867 598	120	190.2	183 134	266 188		29.6
WAVE1644	5	256	1644	13 632 308	160	253.6	321 507	370 436		36.8

attempts (m, σ) is already close to optimal. Intuitively: since $\mathbf{C}$ is a random projection, if $\mathbf{t} \neq \mathbf{0}$ then $\mathbf{tC}$ is a random element of $\mathbb{F}_3^c$, which is $\mathbf{0}$ (leading to an **Accept**) with probability $1/3^c$.

Indeed, if we admit the heuristic of Section 3.3.3 then we can take $2^\mu \approx 3^c$: that is, $c \approx \log_3(2)\mu$. Taking c to be a multiple of 8 simplifies implementation. Table 3.5 lists suggested values of c and corresponding sizes of $\mathbf{CK}$ and $\mathbf{VK}$ for WAVE822, WAVE1249, and WAVE1644.

Chapter 4

Fast and Frobenius: Rational Isogeny Evaluation over Finite Fields

Publication note

This chapter is based on: *Fast and Frobenius: Rational Isogeny Evaluation over Finite Fields*, Gustavo Banegas and Valérie Gilchrist and Anaëlle Le Dévéhat and Benjamin Smith, Proceedings of LATINCRYPT 2023, pp 129–148. Minor edits have been made for the thesis. DOI : https://doi.org/10.1007/978-3-031-44469-2_7

4.1 Introduction

A major drawback of isogeny-based cryptosystems is their relatively slow performance compared with many other post-quantum systems. In this paper, we improve evaluation times for isogenies of many prime degrees $\ell > 3$ given a generator of the kernel; these computations are the fundamental building blocks of most isogeny-based cryptosystems. Specifically, we propose simple alternative differential addition chains to enumerate points of (subsets of) the kernel more efficiently. This speeds up many ℓ -isogeny computations

over the base field by a factor depending on ℓ , and also permits a full additional factor-of- k speed-up for ℓ -isogenies over $\mathbb{F}_q$ whose kernel generators are defined over an extension $\mathbb{F}_{q^k}$. Similar ideas have been explored in [50].

Our techniques have constructive and destructive applications. First, accelerating basic isogeny computations can speed up isogeny-based cryptosystems. The methods in Section 4.4 apply for many $\ell > 3$, so they would naturally improve the performance of commutative isogeny-based schemes such as CSIDH [20], and CSI-FiSh [13] and its derivatives (such as [35] and [41]), which require computing many ℓ -isogenies for various primes ℓ . They may also improve the performance of other schemes like SQISign [44], which computes many ℓ -isogenies in its signing process. (We discuss applications further in Section 4.6.)

In Section 4.5 we focus on rational isogenies with irrational kernels; our methods there could further accelerate the improvements of [43] for Couveignes–Rostovtsev–Stolbunov key exchange (CRS) and related protocols of Stolbunov [33, 105, 114, 115]. This is a small step forward on the road to making CRS a practical “ordinary” fallback for CSIDH in the event of new attacks making specific use of the full supersingular isogeny graph (continuing the approach of [21], for example).

Our results also have applications in cryptanalysis: the best classical and quantum attacks on commutative isogeny-based schemes involve computing massive numbers of group actions, each comprised of a large number of ℓ -isogenies (see e.g. [11] and [27]). Any algorithm that reduces the number of basic operations per ℓ -isogeny will improve the effectiveness of these attacks.

Proof-of-concept implementations of our algorithms in SageMath are available at <https://github.com/vgilchri/k-velu>. The scripts include operation-counting code to verify the counts claimed in this article.

Disclaimer. In this paper, we quantify potential speedups by counting finite field operations. Real-world speed increases depend on many additional variables including parameter sizes; the application context; implementation choices; specificities of the runtime platform (including architecture, vectorization, and hardware acceleration); and the availability of optimized low-level arithmetic.

4.2 Background

We work over (extensions of) the base field $\mathbb{F}_q$, where q is a power of a prime $p > 3$. The symbol ℓ always denotes a prime $\neq p$. In our applications, $3 < \ell \ll p$.

Elliptic curves. For simplicity, elliptic curves are supposed to be in a general Weierstrass form $\mathcal{E} : y^2 = f(x)$. Our algorithms focus on *Montgomery models*

$$\mathcal{E} : By^2 = x(x^2 + Ax + 1) \quad \text{where} \quad B(A^2 - 4) \neq 0.$$

but our results extend easily to other models such as twisted Edwards and short Weierstrass models. The multiplication-by- m map is denoted by $[m]$. The q -power Frobenius endomorphism is $\pi : (x, y) \mapsto (x^q, y^q)$.

Field operations. While the curve $\mathcal{E}$ will always be defined over $\mathbb{F}_q$, we will often work with points defined over $\mathbb{F}_{q^k}$ for $k \geq 1$. We write $\mathbf{M}$, $\mathbf{S}$, and $\mathbf{a}$ for the cost of multiplication, squaring, and adding (respectively) in $\mathbb{F}_{q^k}$. We write $\mathbf{C}$ for the cost of multiplying an element of $\mathbb{F}_{q^k}$ by an element of $\mathbb{F}_q$ (typically a curve constant, or an evaluation-point coordinate). Note that $\mathbf{C} \approx (1/k)\mathbf{M}$ (when k is not too large). Later, we will write $\mathbf{F}$ for the cost of evaluating the Frobenius map on $\mathbb{F}_{q^k}$; see Section 4.5.1 for discussion on this.

x -only arithmetic. Recall the following from Section 2.3.3. Montgomery models are designed to optimize x -only arithmetic (see [80] and [32]). The **xADD** operation is

$$\mathbf{xADD} : (x(P), x(Q), x(P - Q)) \mapsto x(P + Q);$$

it can be computed at a cost of $4\mathbf{M} + 2\mathbf{S} + 6\mathbf{a}$ using the formulæ

$$\begin{cases} X_+ = Z_- [(X_P - Z_P)(X_Q + Z_Q) + (X_P + Z_P)(X_Q - Z_Q)]^2, \\ Z_+ = X_- [(X_P - Z_P)(X_Q + Z_Q) - (X_P + Z_P)(X_Q - Z_Q)]^2 \end{cases} \quad (4.1)$$

where $(X_P : Z_P)$, $(X_Q : Z_Q)$, $(X_+ : Z_+)$, and $(X_- : Z_-)$ are the x -coordinates $x(P)$, $x(Q)$, $x(P + Q)$, and $x(P - Q)$, respectively (so $x(P) = \frac{X_P}{Z_P}$, and so on).

The **xDBL** operation is

$$\mathbf{xDBL} : x(P) \mapsto x([2]P);$$

it can be computed at a cost of $2\mathbf{M} + 2\mathbf{S} + \mathbf{C} + 4\mathbf{a}$ using the formulæ

$$\begin{cases} X_{[2]P} = (X_P + Z_P)^2(X_P - Z_P)^2, \\ Z_{[2]P} = (4X_P Z_P)((X_P - Z_P)^2 + ((A + 2)/4)(4X_P Z_P)). \end{cases} \quad (4.2)$$

Isogenies. Let $\mathcal{E}_1, \mathcal{E}_2$ be elliptic curves over a finite field $\mathbb{F}_q$. An isogeny $\phi : \mathcal{E}_1 \rightarrow \mathcal{E}_2$ is a non-constant morphism mapping the identity point of $\mathcal{E}_1$ to the identity point of $\mathcal{E}_2$. Such a morphism is automatically a homomorphism. For more details see [110, Chapter 3, §4]. The kernel of ϕ is a finite subgroup of $\mathcal{E}_1$, and vice versa.

4.3 Evaluating isogenies

Let $\mathcal{E}$ be an elliptic curve over $\mathbb{F}_q$, and let $\langle G \rangle$ be a subgroup of prime order ℓ (where ℓ is not equal to the field characteristic p) and $G \in \mathcal{E}(\mathbb{F}_{q^k})$. We suppose $\langle G \rangle$ is defined over $\mathbb{F}_q$; then, the quotient isogeny $\phi : \mathcal{E} \rightarrow \mathcal{E}/\langle G \rangle$ is also defined over $\mathbb{F}_q$.

When we say $\langle G \rangle$ is defined over $\mathbb{F}_q$, this means $\langle G \rangle$ is *Galois stable*: that is, $\pi(\langle G \rangle) = \langle G \rangle$ (where π is the q -power Frobenius endomorphism). We will mostly be concerned with algorithms taking $x(G)$ as an input, so it is worth noting that

$$x(G) \in \mathbb{F}_{q^{k'}} \quad \text{where} \quad k' := \begin{cases} k & \text{if } k \text{ is odd,} \\ k/2 & \text{if } k \text{ is even.} \end{cases}$$

The set of projective x -coordinates of the nonzero kernel points is

$$\mathcal{X}_G := \{(X_P : Z_P) = x(P) : P \in \langle G \rangle \setminus \{0\}\} \subset \mathbb{P}^1(\mathbb{F}_{q^{k'}});$$

each X_P/Z_P corresponds to a root of the kernel polynomial $D(X)$, and vice versa. If $\#\langle G \rangle$ is an odd prime ℓ , then $\#\mathcal{X}_G = (\ell - 1)/2$.

4.3.1 The isogeny evaluation problem

We want to evaluate the isogeny $\phi : \mathcal{E} \rightarrow \mathcal{E}/\langle G \rangle$. More precisely, we want efficient solutions to the problem of Definition 4.3.1:

Definition 4.3.1 (Isogeny Evaluation) *Given an elliptic curve $\mathcal{E}$ over $\mathbb{F}_q$, a list of points $(P_1, \dots, P_n)$ in $\mathcal{E}(\mathbb{F}_q)$, and a finite subgroup $\mathcal{G}$ of $\mathcal{E}$ corresponding to the separable isogeny $\phi : \mathcal{E} \rightarrow \mathcal{E}/\mathcal{G}$, compute $(\phi(P_1), \dots, \phi(P_n))$.*

In most cryptographic applications n is relatively small, especially compared to ℓ . We do *not* assume the codomain curve $\mathcal{E}/\mathcal{G}$ is known; if required, an equation for $\mathcal{E}/\mathcal{G}$ can be interpolated through the image of well-chosen evaluation points.

For each separable isogeny ϕ of degree d defined over $\mathbb{F}_q$, there exists a sequence of primes $(\ell_1, \dots, \ell_n)$ and a sequence of isogenies $(\phi_1, \dots, \phi_n)$, all defined over $\mathbb{F}_q$, such that $\phi_n \circ \dots \circ \phi_1$ and

- $\phi_i = [\ell_i]$ (the non-cyclic case) or
- ϕ_i has cyclic kernel of order ℓ_i .

The kernel of ϕ_1 is $\ker \phi \cap \mathcal{E}[\ell_1]$, and so on. The maps $[\ell_i]$ can be computed in $O(\log \ell_i)$ $\mathbb{F}_q$ -operations, so we reduce quickly to the case where ϕ has prime degree ℓ , assuming the factorization of d is known (as it is in our applications).

In general, the isogeny evaluation problem can be reduced to evaluating the map $\alpha \mapsto D(\alpha)$, where D is the kernel polynomial and α is in $\mathbb{F}_q$ or some $\mathbb{F}_q$ -algebra (see e.g. [9, §4]). The polynomial D need not be explicitly computed.

4.3.2 The Costello–Hisil algorithm

The Costello–Hisil algorithm [31], generalized in [102], is the state-of-the-art for evaluating isogenies of Montgomery models. This algorithm is a variation of Vélu’s formulæ working entirely on the level of x -coordinates, using the fact that for an ℓ -isogeny ϕ with kernel $\langle G \rangle$, the rational map on x -coordinates is

$$\phi_x(x) = x \cdot \left(\prod_{i=1}^{(\ell-1)/2} \left(\frac{x \cdot x([i]G) - 1}{x - x([i]G)} \right) \right)^2. \quad (4.3)$$

Moving to projective coordinates $(U : V)$ such that $x = U/V$ and using the fact that $\mathcal{X}_G = \{(x([i]G) : 1) : 1 \leq i \leq (\ell-1)/2\}$, Eq. (4.3) becomes

$$\phi_x((U : V)) = (U' : V'), \quad \begin{cases} U' = U [\prod_{(X_Q : Z_Q) \in \mathcal{X}_G} (UX_Q - VZ_Q)]^2, \\ V' = V [\prod_{(X_Q : Z_Q) \in \mathcal{X}_G} (UZ_Q - VX_Q)]^2. \end{cases} \quad (4.4)$$

Algorithm 10 (from [31]) and Algorithm 11 (our space-efficient variant) compute ϕ_x at a series of input points using an efficient evaluation of the expressions in (4.4). For the moment, we assume that we have subroutines

- **KernelPoints** (see Section 4.4): given $(X_G : Z_G)$, returns $\mathcal{X}_G$ as a list.
- **KernelRange** (see Section 4.4): a *generator* coroutine which, given $(X_G : Z_G)$, constructs and yields the elements of $\mathcal{X}_G$ to the caller one by one.
- **CrissCross** [31, Algorithm 1]: takes $(\alpha, \beta, \gamma, \delta)$ in $\mathbb{F}_{q^k}^4$ and returns $(\alpha\delta + \beta\gamma, \alpha\delta - \beta\gamma)$ in $\mathbb{F}_{q^k}^2$ at a cost of $2\mathbf{M} + 2\mathbf{a}$.

4.4 Accelerating Vélu: faster iteration over the kernel

Let $\mathcal{E}/\mathbb{F}_q$ be an elliptic curve, and let G be a point of prime order ℓ in $\mathcal{E}$. For simplicity, in this section we will assume that G is defined over $\mathbb{F}_q$, but all of the results here apply

Algorithm 10 Combines Algorithms 3 and 4 from [31] to evaluate an ℓ -isogeny of Montgomery models at a list of input points. The total cost is $2n\ell\mathbf{M} + 2n\mathbf{S} + ((n+1)(\ell+1) - 2)\mathbf{a}$, *plus* the cost of `KernelPoints`.

Input: The x -coordinate $(X_G : Z_G)$ of a generator G of the kernel of an ℓ -isogeny ϕ , and a list of evaluation points $((U_i : V_i) : 1 \leq i \leq n)$

Output: The list of images $((U'_i : V'_i) = \phi_x((U_i : V_i)) : 1 \leq i \leq n)$

```

51  $((X_1, Z_1), \dots, (X_{(\ell-1)/2}, Z_{(\ell-1)/2})) \leftarrow \text{KernelPoints}((X_G : Z_G))$  // See Section 4.4
52 for  $1 \leq i \leq (\ell-1)/2$  do
53    $(\hat{X}_i, \hat{Z}_i) \leftarrow (X_i + Z_i, X_i - Z_i)$  // 2a
54 for  $i = 1$  to  $n$  do
55    $(\hat{U}_i, \hat{V}_i) \leftarrow (U_i + V_i, U_i - V_i)$  // 2a
56    $(U'_i, V'_i) \leftarrow (1, 1)$  for  $j = 1$  to  $(\ell-1)/2$  do
57      $(t_0, t_1) \leftarrow \text{CrissCross}(\hat{X}_j, \hat{Z}_j, \hat{U}_i, \hat{V}_i)$  // 2M + 2a
58      $(U'_i, V'_i) \leftarrow (t_0 \cdot U'_i, t_1 \cdot V'_i)$  // 2M
59    $(U'_i, V'_i) \leftarrow (U_i \cdot (U'_i)^2, V_i \cdot (V'_i)^2)$  // 2M + 2S
60 return  $((U'_1, V'_1), \dots, (U'_n, V'_n))$ 

```

Algorithm 11 A generator-based version of Algorithm 10, with much lower space requirements when $\ell \gg n$. The total cost is $2n\ell\mathbf{M} + 2n\mathbf{S} + (2n + (\ell-1)(n+1))\mathbf{a}$, *plus* the cost of a full run of `KernelRange`.

Input: The x -coordinate $(X_G : Z_G)$ of a generator G of the kernel of an ℓ -isogeny ϕ , and a list of evaluation points $((U_i : V_i) : 1 \leq i \leq n)$

Output: The list of images $((U'_i : V'_i) = \phi_x((U_i : V_i)) : 1 \leq i \leq n)$

```

61 for  $1 \leq i \leq n$  do
62    $(\hat{U}_i, \hat{V}_i) \leftarrow (U_i + V_i, U_i - V_i)$  // 2a
63    $(U'_i, V'_i) \leftarrow (1, 1)$ 
64 for  $(X : Z)$  in KernelRange $((X_G : Z_G))$  do // See Section 4.4
65    $(\hat{X}, \hat{Z}) \leftarrow (X + Z, X - Z)$  // 2a
66   for  $1 \leq i \leq n$  do
67      $(t_0, t_1) \leftarrow \text{CrissCross}(\hat{X}, \hat{Z}, \hat{U}_i, \hat{V}_i)$  // 2M + 2a
68      $(U'_i, V'_i) \leftarrow (t_0 \cdot U'_i, t_1 \cdot V'_i)$  // 2M
69 for  $1 \leq i \leq n$  do
70    $(U'_i, V'_i) \leftarrow (U_i \cdot (U'_i)^2, V_i \cdot (V'_i)^2)$  // 2M + 2S
71 return  $((U'_1, V'_1), \dots, (U'_n, V'_n))$ 

```

when G is defined over an extension $\mathbb{F}_{q^k}$: in that case $\mathbf{M}$, $\mathbf{S}$, and $\mathbf{a}$ represent operations in the extension field $\mathbb{F}_{q^k}$, while $\mathbf{C}$ represents multiplication of an element of $\mathbb{F}_{q^k}$ by a curve constant of the subfield $\mathbb{F}_q$ (which is roughly k times cheaper than $\mathbf{M}$). We return to the case where $k > 1$ in Section 4.5.

4.4.1 Kernel point enumeration and differential addition chains

We now turn to the problem of enumerating the set $\mathcal{X}_G$. This process, which we call *kernel point enumeration*, could involve constructing the entire set (as in `KernelPoints`) or constructing its elements one by one (for `KernelRange`).

For $\ell = 2$ and 3 , there is nothing to be done because $\mathcal{X}_G = \{(X_G : Z_G)\}$; so from now on we consider the case $\ell > 3$.

We allow ourselves two curve operations for kernel point enumeration: `xADD` and `xDBL`. In Section 4.5, where G is assumed to be defined over a nontrivial extension of the base field, we will also allow the Frobenius endomorphism.

Every algorithm constructing a sequence of elements of $\mathcal{X}_G$ using a series of `xADD` and `xDBL` instructions corresponds to a *modular differential addition chain*.

Definition 4.4.1 *A Modular Differential Addition Chain (MDAC) for a set $S \subset \mathbb{Z}/\ell\mathbb{Z}$ is a sequence of integers $(c_0, c_1, c_2, \dots, c_n)$ such that*

1. *every element of S is represented by some $c_i \pmod{\ell}$,*
2. *$c_0 = 0$ and $c_1 = 1$, and*
3. *for each $1 < i \leq n$ there exist $0 \leq j(i), k(i), d(i) < i$ such that $c_i \equiv c_{j(i)} + c_{k(i)} \pmod{\ell}$ and $c_{j(i)} - c_{k(i)} \equiv c_{d(i)} \pmod{\ell}$.*

Algorithms to enumerate $\mathcal{X}_G$ using `xADD` and `xDBL` correspond to MDACs $(c_0, \dots, c_n)$ for $\{1, \dots, (\ell-1)/2\}$: the algorithm starts with $x([c_0]G) = x(0) = (1 : 0)$ and $x([c_1]G) = x(G) = (X_G : Z_G)$, then computes each $x([c_i]G)$ using

$$x([c_i]G) = \begin{cases} \text{xADD}(x([c_{j(i)}]G), x([c_{k(i)}]G), x([c_{d(i)}]G)) & \text{if } d(i) \neq 0, \\ \text{xDBL}([c_{j(i)}]G) & \text{if } d(i) = 0. \end{cases}$$

4.4.2 Additive kernel point enumeration

The classic approach is to compute $\mathcal{X}_G$ using repeated `xADDs`. Algorithm 12 is Costello and Hisil's `KernelPoints` [31, Algorithm 2], corresponding to the MDAC $(0, 1, 2, 3, \dots, (\ell -$

1)/2) computed by repeatedly adding 1 (except for 2 which is computed by doubling 1). The simplicity of this MDAC means that Algorithm 12 converts to a `KernelRange` with a small internal state: to generate the next $(X_{i+1} : Z_{i+1})$, we only need the values of $(X_i : Z_i)$, $(X_{i-1} : Z_{i-1})$, and $(X_1 : Z_1)$.

Algorithm 12 Basic kernel point enumeration by repeated addition. Uses exactly 1 `xDBL` and $(\ell - 5)/2$ `xADD` operations (for prime $\ell > 3$).

Input: The x -coordinate $(X_G : Z_G) = x(G)$ of a point G of order ℓ in $\mathcal{E}(\mathbb{F}_q)$

Output: $\mathcal{X}_G$ as a list

```

72  $(X_1 : Z_1) \leftarrow (X_G : Z_G)$    $(X_2 : Z_2) \leftarrow \text{xDBL}((X_G : Z_G))$   for  $i = 3$  to  $(\ell - 1)/2$  do
    // Invariant:  $(X_i : Z_i) = x([i]G)$ 
73    $(X_i : Z_i) \leftarrow \text{xADD}((X_{i-1} : Z_{i-1}), (X_G : Z_G), (X_{i-2}, Z_{i-2}))$ 
74 return  $((X_1 : Z_1), \dots, (X_{(\ell-1)/2} : Z_{(\ell-1)/2}))$ 

```

4.4.3 Replacing `xADDs` with `xDBLs`

Comparing x -only operations on Montgomery curves, replacing an `xADD` with an `xDBL` trades $2\mathbf{M}$ and $2\mathbf{a}$ for $1\mathbf{C}$. We would therefore like to replace as many `xADDs` as possible in our kernel enumeration with `xDBLs`.

As a first attempt, we can replace Line 73 of Algorithm 12 with

$$(X_i : Z_i) \leftarrow \begin{cases} \text{xDBL}((X_{i/2} : Z_{i/2})) & \text{if } i \text{ is even,} \\ \text{xADD}((X_{i-1} : Z_{i-1}), (X_G : Z_G), (X_{i-2}, Z_{i-2})) & \text{if } i \text{ is odd.} \end{cases}$$

But applying this trick systematically requires storing many more intermediate values, reducing the efficiency of `KernelRange`. It also only replaces half of the `xADDs` with `xDBLs`, and it turns out that we can generally do much better.

4.4.4 Multiplicative kernel point enumeration

We can do better for a large class of ℓ by considering the quotient

$$M_\ell := (\mathbb{Z}/\ell\mathbb{Z})^\times / \langle \pm 1 \rangle.$$

(note: M_ℓ is a quotient of the *multiplicative* group.) For convenience, we write

$$m_\ell := \#M_\ell = (\ell - 1)/2.$$

We can now reframe the problem of enumerating $\mathcal{X}_G$ as the problem of enumerating a complete set of representatives for M_ℓ . The MDAC of Algorithm 12 computes the

set of representatives $\{1, 2, \dots, m_\ell\}$, but for the purposes of enumerating $\mathcal{X}_G$, any set of representatives will do. Example 4.4.2 is particularly useful.

Example 4.4.2 Suppose 2 generates M_ℓ . This is the case if 2 is a primitive element modulo ℓ —that is, if 2 has order $(\ell - 1)$ modulo ℓ —but also if 2 has order $(\ell - 1)/2$ modulo ℓ and $\ell \equiv 3 \pmod{4}$. In this case

$$M_\ell = \{2^i \bmod \ell : 0 \leq i < m_\ell\},$$

so $(0, 1, 2, 4, 8, \dots, 2^{m_\ell})$ is an MDAC for M_ℓ using only doubling, and no differential additions. The corresponding `KernelPoints` replaces all of the `xADDs` in Algorithm 12 with cheaper `xDBLs`, trading $(\ell - 5)\mathbf{M} + (\ell - 5)\mathbf{a}$ for $(\ell - 5)/2 \mathbf{C}$. The corresponding `KernelRange` is particularly simple: each element depends only on its predecessor, so the state consists of a single $(X_i : Z_i)$.

How often does this trick apply? The quantitative form of Artin’s primitive root conjecture (see [122]) says that $M_\ell = \langle 2 \rangle$ for a little over half of all ℓ . Experimentally, 5609420 of the first 10^7 odd primes ℓ satisfy $M_\ell = \langle 2 \rangle$.

One might generalize Example 4.4.2 to other generators of M_ℓ : for example, if $M_\ell = \langle 3 \rangle$, then we could find an MDAC for $\{3^i \bmod \ell : 0 \leq i < (\ell - 1)/2\}$. But this is counterproductive: x -only tripling is *slower* than differential addition.

4.4.5 Stepping through cosets

What can we do when $M_\ell \neq \langle 2 \rangle$? A productive generalization is to let

$$A_\ell := \langle 2 \rangle \subseteq M_\ell \quad \text{and} \quad a_\ell := \#A_\ell,$$

and to try to compute a convenient decomposition of M_ℓ into cosets of A_ℓ . Within each coset, we can compute elements using repeated `xDBLs` as in Example 4.4.2; then, it remains to step from one coset into another using differential additions.

This can be done in a particularly simple way for the primes ℓ such that

$$M_\ell = \langle 2, 3 \rangle, \quad \text{so} \quad M_\ell = \bigsqcup_{i=0}^{m_\ell/a_\ell-1} 3^i A_\ell. \quad (*)$$

We can move from the i -th to the $(i + 1)$ -th coset using the elementary relations

$$\begin{cases} c \cdot 2^{j+1} + c \cdot 2^j = 3c \cdot 2^j \\ c \cdot 2^{j+1} - c \cdot 2^j = c \cdot 2^j \end{cases} \quad \text{for all integers } c \text{ and } j \geq 0. \quad (4.5)$$

In particular, having enumerated $3^i A_\ell$ by repeated doubling, we can compute an element of $3^{i+1} A_\ell$ by applying a differential addition to any two consecutive elements of $3^i A_\ell$ (and the difference is the first of them). Algorithm 13 minimizes storage overhead by using the last two elements of the previous coset to generate the first element of the next one. The **KernelRange** of Algorithm 13 therefore has an internal state of only two x -coordinates—so not only is it faster than the **KernelRange** of Algorithm 12, but it also has a smaller memory footprint.

Algorithm 13 Kernel enumeration for $\ell > 3$ satisfying (*). Cost: $(1 - 1/a_\ell) \cdot m_\ell$ **xDBLs** and $m_\ell/a_\ell - 1$ **xADDs**.

Input: Projective x -coordinate $(X_G : Z_G)$ of the generator G of a cyclic subgroup of order ℓ in $\mathcal{E}(\mathbb{F}_q)$, where ℓ satisfies (*).

Output: $\mathcal{X}_G$ as a list

```

75  $(a, b) \leftarrow (a_\ell, m_\ell/a_\ell)$   for  $i = 0$  to  $b - 1$  do                                // Invariant:
     $(X_{ai+j} : Z_{ai+j}) = x([3^i 2^{i(a-2)+(j-1)}]G)$ 
76   if  $i = 0$  then
77      $(X_1 : Z_1) \leftarrow (X_G : Z_G)$ 
78   else                                     // Compute new coset representative
79      $(X_{ai+1} : Z_{ai+1}) \leftarrow \mathbf{xADD}((X_{ai} : Z_{ai}), (X_{ai-1} : Z_{ai-1}), (X_{ai-1} : Z_{ai-1}))$ 
80   for  $j = 2$  to  $a$  do                       // Exhaust coset by doubling
81      $(X_{ai+j} : Z_{ai+j}) \leftarrow \mathbf{xDBL}((X_{ai+j-1} : Z_{ai+j-1}))$ 
82 return  $((X_1 : Z_1), \dots, (X_{(\ell-1)/2} : Z_{(\ell-1)/2}))$ 

```

Algorithm 13 performs better the closer a_ℓ is to m_ℓ . In the best case, when $A_\ell = M_\ell$, it uses $m_\ell - 1$ **xDBLs** and no **xADDs** at all. The worst case is when the order of 2 in M_ℓ is as small as possible: that is, $\ell = 2^k - 1$. In this case $a_\ell = k$, and compared with Algorithm 12 we still reduce the **xADDs** by a factor of k .

4.4.6 The remaining primes

While 1878 of the 2261 odd primes $\ell \leq 20000$ satisfy (*), there are still 383 primes that do not. We can, to some extent, adapt Algorithm 13 to handle these primes, but on a case-by-case basis and with somewhat less satisfactory results.

For example, the CSIDH-512 parameter set specifies 74 isogeny-degree primes

$$\ell = 3, 5, 7, 11, 13, \dots, 367, 373, \text{ and } 587.$$

All but seven of these ℓ satisfy (*): the exceptions are $\ell = 73, 97, 193, 241, 313$, and 337 . Table 4.1 lists a candidate decomposition of M_ℓ for each of these ℓ . In each case, we need

to produce an element of either $5A_\ell$ or $7A_\ell$. This can certainly be done using previously-computed elements, but this requires a larger internal state and a more complicated execution pattern, depending on ℓ .

Table 4.1: Primes ℓ in the CSIDH-512 parameter set that do not satisfy (*).

Prime ℓ	a_ℓ	$[M_\ell : \langle 2, 3 \rangle]$	Coset decomposition of M_ℓ	Notes
73	9	2	$M_{73} = A_{73} \sqcup 3A_{73} \sqcup 5A_{73} \sqcup 5 \cdot 3A_{73}$	
97	24	2	$M_{97} = A_{97} \sqcup 5A_{97}$	3 is in A_{97}
193	48	2	$M_{193} = A_{193} \sqcup 5A_{193}$	3 is in A_{193}
241	12	2	$M_{241} = (\bigsqcup_{i=0}^4 3^i A_{241}) \sqcup (\bigsqcup_{i=0}^4 7 \cdot 3^i A_{241})$	
307	51	3	$M_{307} = A_{307} \sqcup 5A_{307} \sqcup 7A_{307}$	3 is in A_{313}
313	78	2	$M_{313} = A_{313} \sqcup 5A_{313}$	3 is in A_{193}
337	21	2	$M_{337} = (\bigsqcup_{i=0}^3 3^i A_{337}) \sqcup (\bigsqcup_{i=0}^3 5 \cdot 3^i A_{337})$	

Example 4.4.3 Consider $\ell = 97$. Now $3 \equiv 2^{19} \pmod{97}$, so 3 is in A_{97} , and in fact $M_{97} = A_{97} \sqcup 5A_{97}$. To adapt Algorithm 13 to this case, we can still enumerate A_{97} using repeated doubling. Then, we must construct an element of $5A_{97}$ from elements of A_{97} , using a differential addition like $5 \cdot 2^i = 2^{i+2} + 2^i$ (difference $3 \cdot 2^i$) or $5 \cdot 2^i = 2^{i+1} + 3 \cdot 2^i$ (difference 2^i). Each involves near powers of 2 (modulo 97), but also $3 \cdot 2^i \equiv 2^{i+19} \pmod{97}$, which must be stored while enumerating A_{97} . This gives an algorithm using one `xADD` and 48 `xDBLs`, just like Algorithm 13, but with a slightly larger state and a more complicated execution pattern specific to $\ell = 97$. Alternatively, after enumerating A_{97} , we could redundantly recompute $3 = 1 + 2$ (difference 1) to get 5 as $1 + 4$ (difference 3) or $2 + 3$ (difference 1).

Ultimately, there does not seem to be a “one size fits all” generalization of Algorithm 13 for enumerating $\mathcal{X}_G$ without a more complicated state or redundant recomputations. We can get reasonable results for many ℓ not satisfying (*) by finding a good MDAC for $M_\ell/\langle 2, 3 \rangle$ and then using Algorithm 13 to exhaust the coset containing each representative but the savings are generally not optimal.

4.4.7 (In)Compatibility with Vélsqrt

It is natural to ask whether these techniques can be used to further accelerate the Vélsqrt algorithm [9], which evaluates isogenies of large prime degree ℓ in $\tilde{O}(\sqrt{\ell})$ time (with $O(\sqrt{\ell})$ space). Vélsqrt never explicitly computes all of $\mathcal{X}_G$. Instead, it relies on the

existence of a decomposition

$$S := \{1, 3, 5, \dots, \ell - 2\} = (I + J) \sqcup (I - J) \sqcup K \quad (4.6)$$

where I , J , and K are sets of integers of size $O(\sqrt{\ell})$ such that the maps $(i, j) \rightarrow i + j$ and $(i, j) \rightarrow i - j$ are injective with disjoint images. In [9], these sets are

$$\begin{aligned} I &:= \{2b(2i + 1) : 0 \leq i < b'\} && \text{("giant steps")}, \\ J &:= \{2j + 1 : 0 \leq j < b\} && \text{("baby steps")}, \\ K &:= \{4bb' + 1, \dots, \ell - 4, \ell - 2\} && \text{("the rest")}, \end{aligned}$$

where $b := \lfloor \sqrt{\ell - 1}/2 \rfloor$ and $b' := \lfloor (\ell - 1)/4b \rfloor$.

The key thing to note here is that this decomposition is essentially additive, and the elements of I , J , and K are in arithmetic progression. Algorithm 13, however, is essentially multiplicative: it works with subsets in geometric progression. We cannot exclude the existence of subsets I , J , and K of size $O(\sqrt{\ell})$ satisfying (4.6) and which are amenable to enumeration by a variant of Algorithm 13 for some ℓ , but it seems difficult to construct nontrivial and useful examples.

4.5 Irrational kernel points: exploiting Frobenius

Now suppose G is defined over a nontrivial extension $\mathbb{F}_{q^k}$ of $\mathbb{F}_q$, but $\langle G \rangle$ is defined over the subfield $\mathbb{F}_q$: that is, it is Galois-stable. In particular, the q -power Frobenius endomorphism π of $\mathcal{E}$, which maps points in $\mathcal{E}(\mathbb{F}_{q^k})$ to their conjugates under $\text{Gal}(\mathbb{F}_{q^k}/\mathbb{F}_q)$, maps $\langle G \rangle$ into $\langle G \rangle$, and hence restricts to an endomorphism of $\langle G \rangle$. But since the endomorphisms of $\langle G \rangle$ are $\mathbb{Z}/\ell\mathbb{Z}$, and Frobenius has no kernel (so π is not 0 on $\langle G \rangle$), it must act as multiplication by an eigenvalue $\lambda \neq 0$ on $\langle G \rangle$. The precise value of λ is not important here, but we will use the fact that λ has order k in $(\mathbb{Z}/\ell\mathbb{Z})^\times$ and order k' in $(\mathbb{Z}/\ell\mathbb{Z})^\times / \langle \pm 1 \rangle$.

Now let

$$F_\ell := \langle \lambda \rangle \subseteq M_\ell \quad \text{and} \quad c_F := [M_\ell : F_\ell] = m_\ell/k'.$$

Let R_0 be a set of representatives for M_ℓ/F_ℓ ; set $S_0 := \{[r]G : r \in R_0\}$, and note

$$\#S_0 = (\ell - 1)/k'.$$

4.5.1 The cost of Frobenius

We want to use the Galois action to replace many **M** and **S** with a few **F**. For this to be worthwhile, **F** must be cheap: and it is, even if this is not obvious given the definition of

the Frobenius map on $\mathbb{F}_{q^k}$ as q -th powering. Indeed, we do not compute Frobenius by powering. Instead, we use the fact that Frobenius is $\mathbb{F}_q$ -linear, acting as a $k \times k$ matrix (with entries in $\mathbb{F}_q$) on the coefficient vectors of elements in $\mathbb{F}_{q^k}$. The form of this matrix, and the cost of applying it, depends on the basis of $\mathbb{F}_{q^k}/\mathbb{F}_q$. For example:

1. If $k = 2$ and $\mathbb{F}_{q^2} = \mathbb{F}_q(\sqrt{\Delta})$, then Frobenius simply negates $\sqrt{\Delta}$ and the matrix is $\text{diag}(1, -1)$, so $\mathbf{F} \approx 0$.
2. If $\mathbb{F}_{q^k}/\mathbb{F}_q$ is represented with a normal basis, then the matrix represents a cyclic permutation, and again $\mathbf{F} \approx 0$.

Even in the worst case where the basis of $\mathbb{F}_{q^k}/\mathbb{F}_q$ has no special Galois structure, $\mathbf{F}$ is just the cost of multiplying a k -vector by a $k \times k$ matrix over $\mathbb{F}_q$: that is, k^2 multiplications and $k^2 - k$ additions. This is close to the cost of one “schoolbook” $\mathbb{F}_{q^k}$ -multiplication; so when $k \leq 12$, we have $\mathbf{F} \approx \mathbf{M}$.

4.5.2 Galois orbits

Each point $P \in \mathcal{E}(\mathbb{F}_{q^k})$ is contained in a *Galois orbit* containing all the conjugates of P . The kernel subgroup $\langle G \rangle$ breaks up (as a set) into *Galois orbits*: if we write

$$\mathcal{O}_P := \{P, \pi(P), \dots, \pi^{k-1}(P)\} \quad \text{for } P \in \mathcal{E}(\mathbb{F}_{q^k}),$$

then

$$\langle G \rangle = \{0\} \sqcup \begin{cases} \bigsqcup_{P \in S_0} \mathcal{O}_P & \text{if } k \text{ is even,} \\ (\bigsqcup_{P \in S_0} \mathcal{O}_P) \sqcup (\bigsqcup_{P \in S_0} \mathcal{O}_{-P}) & \text{if } k \text{ is odd.} \end{cases} \quad (4.7)$$

To get a picture of where we are going, recall from Section 4.3 that in general, isogeny evaluation can be reduced to evaluations of the kernel polynomial

$$D(X) := \prod_{P \in S} (X - x(P)),$$

where $S \subset \langle G \rangle$ is any subset such that $S \cap -S = \emptyset$ and $S \cup -S = \langle G \rangle \setminus \{0\}$. The decomposition of (4.7) can be seen in the factorization of $D(X)$ over $\mathbb{F}_{q^k}$:

$$D(X) = \prod_{P \in S} (X - x(P)) = \prod_{P \in S_0} \prod_{i=0}^{k'-1} (X - x(\pi^i(P))) = \prod_{P \in S_0} \prod_{i=0}^{k'-1} (X - x(P)^{q^i}),$$

and the factors corresponding to each P in S_0 are the irreducible factors of D over $\mathbb{F}_q$. Transposing the order of the products, if we let

$$D_0(X) := \prod_{P \in S_0} (X - x(P))$$

then for α in the base field $\mathbb{F}_q$, we can compute $D(\alpha)$ using

$$D(\alpha) = \text{Norm}(D_0(\alpha)) \quad \text{for all } \alpha \in \mathbb{F}_q$$

where

$$\text{Norm}(x) := \prod_{i=0}^{k'-1} x^{q^i} = x(x(\cdots (x(x)^q)^q \cdots)^q)^q,$$

which can be computed for the cost of $(k-1)\mathbf{F} + (k-1)\mathbf{M}$ (some multiplications can be saved with more storage, but for small k this may not be worthwhile).

Similarly, we can rewrite the rational map ϕ_x from (4.3) as

$$\phi_x(x) = x \cdot \left[\prod_{P \in S} \left(\frac{x \cdot x(P) - 1}{x - x(P)} \right) \right]^2 = x \cdot \left[\prod_{P \in S_0} \prod_{i=0}^{k'-1} \left(\frac{x \cdot x(P)^{q^i} - 1}{x - x(P)^{q^i}} \right) \right]^2.$$

Evaluating ϕ_x at α in $\mathbb{F}_q$, rearranging the products gives

$$\phi_x(\alpha) = \alpha \cdot \left[\prod_{P \in S_0} \prod_{i=0}^{k'-1} \left(\frac{\alpha \cdot x(P)^{q^i} - 1}{\alpha - x(P)^{q^i}} \right) \right]^2 = \alpha \cdot \text{Norm}(\bar{\phi}_x(\alpha))^2,$$

where

$$\bar{\phi}_x(X) := \prod_{P \in S_0} \frac{X \cdot x(P) - 1}{X - x(P)}.$$

Projectively, from (4.4) we get $\phi_x : (U : V) \mapsto (U' : V')$ where

$$\begin{cases} U' = U \cdot \left[\prod_{i=0}^{k'-1} \prod_{P \in S_0} (U X_P^{q^i} - Z_P^{q^i} V) \right]^2, \\ V' = V \cdot \left[\prod_{i=0}^{k'-1} \prod_{P \in S_0} (U Z_P^{q^i} - X_P^{q^i} V) \right]^2, \end{cases}$$

so if we set

$$F(U, V) := \prod_{P \in S_0} (U \cdot X_P - Z_P \cdot V) \quad \text{and} \quad G(U, V) := \prod_{P \in S_0} (U \cdot Z_P - X_P \cdot V),$$

then for α and β in $\mathbb{F}_q$ we get

$$\phi_x((\alpha : \beta)) = (\alpha' : \beta') := (\alpha \cdot \text{Norm}(F(\alpha, \beta))^2 : \beta \cdot \text{Norm}(G(\alpha, \beta))^2).$$

4.5.3 Enumerating representatives for the Galois orbits.

We now need to enumerate a set S_0 of representatives for the Galois orbits modulo ± 1 or, equivalently, a set of representatives R_0 for the cosets of F_ℓ in M_ℓ . Given an MDAC driving enumeration of the coset representatives, there are obvious adaptations of Algorithms 10 and 11 to this extension field case. Rather than iterating over all of the

kernel x -coordinates, we just iterate over a subset representing the cosets of $\mathbb{F}_\ell$, and then compose with the norm.

Concretely, in Algorithm 11, we should

1. Replace `KernelRange` in Line 4 with a generator driven by an efficient MDAC for M_ℓ/F_ℓ ;
2. Replace Line 10 with $(U'_i, V'_i) \leftarrow (U_i \cdot \text{Norm}(U'_i)^2, V_i \cdot \text{Norm}(V'_i)^2)$.

First, we can consider Algorithm 12: that is, enumerating M_ℓ/F_ℓ by repeated addition. Unfortunately, we do not have a nice bound on the length of this MDAC: the coset representatives may not be conveniently distributed over M_ℓ , so we could end up computing a lot of redundant points.

Example 4.5.1 Consider the “naive” S_0 comprised of the minimal elements (up to negation) in each Galois orbit. We computed the percentage of primes $3 \leq \ell < 10^4$ where an optimal MDAC (without redundant values) exists for this S_0 :

k	1	2	3	4	5	6	7	8	9	10	11	12
%	100	100	100	100	84	86	76	67	60	56	45	42

For an example of what can go wrong, take $(\ell, k) = (89, 11)$. In this case, we get $R_0 = \{1, 3, 5, 13\}$; the shortest MDAC is $(0, 1, 2, 3, 5, 8, 13)$, which enumerates the kernel using one `xDBL` operation and six `xADD` operations, but requires the computation of two intermediate points not used in the final result.

But when we say that the coset representatives are not conveniently distributed over M_ℓ , we mean convenient with respect to addition. If we look at M_ℓ multiplicatively, then the path to efficient MDACs is clearer.

If $M_\ell = \langle 2, \lambda \rangle$ then we can take $R_0 = \{2^i : 0 \leq i < c_F\}$, which brings us to the 2-powering MDAC of Example 4.4.2—except that we stop after $c_F - 1$ `xDBL`s. We thus reduce the number of `xDBL`s by a factor of $\approx k'$, at the expense of two norm computations. This MDAC actually applies to more primes ℓ here than in Section 4.4, because we no longer need 2 to generate all of M_ℓ ; we have λ to help. (In fact, the suitability of this MDAC depends not only on ℓ , but also on k .)

We can go further if we assume

$$M_\ell = \langle 2, 3, \lambda \rangle. \tag{**}$$

To simplify notation, we define

$$a_{\ell,k} := [\langle 2, \lambda \rangle : F_\ell], \quad b_{\ell,k} := [\langle 2, 3, \lambda \rangle : \langle 2, \lambda \rangle] = c_F / a_{\ell,k}.$$

Algorithm 14 is a truncated version of Algorithm 13 for computing S_0 instead of $\mathcal{X}_G$ when $(**)$ holds. Algorithm 15 uses Algorithm 14 to evaluate an ℓ -isogeny over $\mathbb{F}_q$ with kernel $\langle G \rangle$ at n points of $\mathcal{E}(\mathbb{F}_q)$, where $x(G)$ is in $\mathbb{F}_{q^{k'}}$ with $k' > 1$.

Table 4.2 compares the total costs of Algorithms 15 and 14 with Algorithms 10 and 12. In both algorithms, we can take advantage of the fact that many of the multiplications have one operand in the smaller field $\mathbb{F}_q$: notably, the multiplications involving coordinates of the evaluation points. In the context of isogeny-based cryptography (where curve constants look like random elements of $\mathbb{F}_q$), this means that in Algorithm 10, we can replace the $2\mathbf{M} + 2\mathbf{a}$ in Line 57 and the $2\mathbf{M} + 2\mathbf{S}$ in Line 59 with $2\mathbf{C} + 2\mathbf{a}$ and $2\mathbf{C} + 2\mathbf{S}$, respectively. Table 4.3 gives examples of the resulting costs for various (ℓ, k) with a single evaluation point.

Table 4.2: ℓ -isogeny evaluation comparison for kernels $\langle G \rangle$ defined over $\mathbb{F}_q$ but with $x(G) \in \mathbb{F}_{q^{k'}}$. Here, $\mathbf{C}$ denotes multiplications of elements of $\mathbb{F}_{q^{k'}}$ by elements of $\mathbb{F}_q$.

	Costello–Hisil (Algorithms 10 and 12)	This work (Algorithm 15)
M	$(\ell - 1)n + 2\ell - 8$	$2(c_F + k' - 1)n + 2c_F + 2b_{\ell,k} + 4$
S	$2n + \ell - 3$	$2n + 2c_F - 2$
C	$(\ell + 1)n + 1$	$2(c_F + 1)n + c_F - b_{\ell,k}$
a	$(n + 1)(\ell + 1) + 3\ell + 17$	$2c_F(n + 1) + 4c_F + 4b_{\ell,k} - 6$
F	0	$2(k' - 1)n$

Table 4.3: Examples of costs for evaluating an ℓ -isogeny at a single point over $\mathbb{F}_q$, with $x(G) \in \mathbb{F}_{q^{k'}}$, using Costello–Hisil (Algorithm 10 with 12, in white) and Algorithm 15 (in gray). For these k , it is reasonable to use the approximation $\mathbf{F} \approx \mathbf{M}$ (see Section 4.5.1).

$\ell = 13$						$\ell = 19$						$\ell = 23$					
k'	M	S	C	a	F	k'	M	S	C	a	F	k'	M	S	C	a	F
any	30	12	15	54	0	any	48	18	21	84	0	any	60	22	25	104	0
1	22	12	19	46	0	1	34	18	28	70	0	1	42	22	34	86	0
3	10	4	7	14	4	3	14	6	10	22	4	11	22	2	4	6	20
						9	18	2	4	6	16						

Algorithm 14 Compute S_0 when $(**)$ holds. Cost: $b_{\ell,k} - 1$ xADDs and $(c_F - b_{\ell,k})$ xDBLs, or $(2c_F + 2b_{\ell,k} + 4)\mathbf{M} + (2c_F - 2)\mathbf{S} + (c_F - b_{\ell,k})\mathbf{C} + (4c_F + 4b_{\ell,k} - 6)\mathbf{a}$

Input: Projective x -coordinate $(X_G : Z_G)$ of the generator G of a cyclic subgroup of order ℓ in $\mathcal{E}(\mathbb{F}_{q^k})$, where ℓ satisfies $M_\ell = \langle 2, 3, \lambda \rangle$.

Output: S_0 as a list

```

83 Function SZeroPoints( $(X_G : Z_G)$ )
84    $(a, b) \leftarrow (a_{\ell,k}, b_{\ell,k})$  for  $i = 0$  to  $b - 1$  do                                // Invariant:
       $x_{ai+j} = x([3^i 2^{i(a-2)+(j-1)}]G)$ 
85   | if  $i = 0$  then
86   | |  $x_1 \leftarrow (X_G : Z_G)$ 
87   | else                                     // Compute new coset representative
88   | |  $x_{ai+1} \leftarrow \text{xADD}(x_{ai}, x_{ai-1}, x_{ai-1})$ 
89   | for  $j = 2$  to  $a$  do                       // Exhaust coset by doubling
90   | |  $x_{ai+j} \leftarrow \text{xDBL}(x_{ai+j-1})$ 
91 | return  $(x_1, \dots, x_{c_F})$ 

```

Algorithm 15 Isogeny evaluation using **SZeroPoints** and Frobenius. Cost: $2(c_F + k - 1)n\mathbf{M} + 2n\mathbf{S} + 2(c_F + 1)n\mathbf{C} + 2c_F(n + 1)\mathbf{a} + 2(k - 1)n\mathbf{F}$ plus the cost of **SZeroPoints**.

Input: The x -coordinate $(X_G : Z_G)$ of a generator G of the kernel of an ℓ -isogeny ϕ , and a list of evaluation points $((U_i : V_i) : 1 \leq i \leq n)$

Output: The list of images $((U'_i : V'_i) = \phi_x((U_i : V_i)) : 1 \leq i \leq n)$

```

92  $((X_1, Z_1), \dots, (X_{c_F}, Z_{c_F})) \leftarrow \text{SZeroPoints}((X_G : Z_G))$            // Algorithm 14
93 for  $1 \leq i \leq c_F$  do
94 |  $(\hat{X}_i, \hat{Z}_i) \leftarrow (X_i + Z_i, X_i - Z_i)$                                 // 2a
95 for  $i = 1$  to  $n$  do
96 |  $(\hat{U}_i, \hat{V}_i) \leftarrow (U_i + V_i, U_i - V_i)$                                 // 2a
97 |  $(U'_i, V'_i) \leftarrow (1, 1)$  for  $j = 1$  to  $c_F$  do
98 | |  $(t_0, t_1) \leftarrow \text{CrissCross}(\hat{X}_j, \hat{Z}_j, \hat{U}_i, \hat{V}_i)$                        // 2C + 2a
99 | |  $(U'_i, V'_i) \leftarrow (t_0 \cdot U'_i, t_1 \cdot V'_i)$                            // 2M
100 |  $(U'_i, V'_i) \leftarrow (\text{Norm}(U'_i), \text{Norm}(V'_i))$                              //  $2(k' - 1)\mathbf{M} + 2(k' - 1)\mathbf{F}$ 
101 |  $(U'_i, V'_i) \leftarrow (U_i \cdot (U'_i)^2, V_i \cdot (V'_i)^2)$                      // 2C + 2S
102 return  $((U'_1, V'_1), \dots, (U'_n, V'_n))$ 

```

4.6 Applications to key exchange

Our algorithms could be applied in any cryptosystem involving isogenies of prime degree $\ell > 3$. We focus on key exchanges like CSIDH [20] here, but similar remarks apply for other schemes such as SQISign [44, 45], SeaSign [42], and CSI-FiSh [13].

4.6.1 CSIDH and constant-time considerations

CSIDH is a post-quantum non-interactive key exchange based on the action of the class group of the imaginary quadratic order $\mathbb{Z}[\sqrt{-p}]$ on the set of supersingular elliptic curves $\mathcal{E}/\mathbb{F}_p$ with $\text{End}_{\mathbb{F}_p}(\mathcal{E}) \cong \mathbb{Z}[\sqrt{-p}]$. The action is computed via compositions of ℓ_i -isogenies for a range of small primes $(\ell_1, \dots, \ell_m)$.

CSIDH works over prime fields $\mathbb{F}_p$, so the methods of Section 4.5 do not apply; but Algorithm 13 may speed up implementations at least for the ℓ_i satisfying (*). (We saw in Section 4.4.6 that 67 of the 74 primes ℓ_i in the CSIDH-512 parameter set met (*)).

The true speedup depends on two factors. The first is the number of evaluation points. Costello and Hisil evaluate at a 2-torsion point other than $(0, 0)$ in order to interpolate the image curve. The constant-time CSIDH of [76] evaluates at one more point (from which subsequent kernels are derived)—that is, $n = 2$; [91] uses $n = 3$; [28] discusses $n > 3$. For large n , the cost of Algorithm 10 overwhelms kernel enumeration, but our results may still make a simple and interesting improvement when n is relatively small.

The second factor is the organisation of primes into batches for constant-time CSIDH implementations. CTIDH [4] hides the degree ℓ using the so-called *matryoshka* property: ℓ_i -isogeny evaluation is a sub-computation of ℓ_j -isogeny computation whenever $\ell_i < \ell_j$ using Algorithms 10 and 12. Organising primes into similar-sized batches, we can add dummy operations to disguise smaller-degree isogenies as isogenies of the largest degree in their batch.

Our Algorithm 13 has a limited matryoshka property: ℓ_i -isogenies are sub-computations of ℓ_j -isogenies if $a_{\ell_i} \leq a_{\ell_k}$ and $m_{\ell_i}/a_{\ell_i} \leq m_{\ell_j}/a_{\ell_j}$. For constant-time implementations, it would make more sense to make all primes in a batch satisfying (*) a sub-computation of an algorithm using the maximum a_ℓ and maximum m_ℓ/a_ℓ over ℓ in the batch. Redistributing batches is a delicate matter with an important impact on efficiency; therefore, while our work improves the running time for a fixed ℓ , its impact on batched computations remains uncertain, and ultimately depends on specific parameter choices.

4.6.2 CRS key exchange

The historical predecessors of CSIDH, due to Couveignes [33] and Rostovtsev and Stolbunov [105, 114, 115], are collectively known as CRS. Here the class group of an quadratic imaginary order $\mathcal{O}$ acts on an isogeny (sub)class of elliptic curves $\mathcal{E}$ with $\text{End}(\mathcal{E}) \cong \mathcal{O}$. CRS performance was greatly improved in [43] using Vélú-style isogeny evaluation, but this requires finding ordinary isogeny classes over $\mathbb{F}_p$ with rational ℓ_i -torsion points over $\mathbb{F}_{q^{k_i}}$ with k_i as small as possible for as many ℓ_i as possible.

One such isogeny class over a 512-bit prime field is proposed in [43, §4]. The curves ℓ -isogenies with kernel generators over $\mathbb{F}_p$ for $\ell = 3, 5, 7, 11, 13, 17, 103, 523$, and 821 , and over $\mathbb{F}_{p^k}$ for $\ell = 19, 29, 31, 37, 61, 71, 547, 661, 881, 1013, 1181, 1321$, and 1693 . These “irrational” ℓ are an interesting basis of comparison for our algorithms: Table 4.4 shows that there are substantial savings to be had.

Table 4.4: Costello–Hisil (Algorithms 10 and 12, in white) vs. Algorithm 15 (in gray) for the CRS parameters with $k > 1$ proposed in [43]. We omit $(\ell, k) = (1321, 5)$, since in this case $M_\ell \neq \langle 2, 3, \lambda \rangle$. Here **M**, **S**, **a**, and **F** refer to operations on elements of $\mathbb{F}_{q^{k'}}$, while **C** denotes multiplications of elements of $\mathbb{F}_{q^{k'}}$ by elements of $\mathbb{F}_q$.

k	ℓ	$a_{\ell,k}$	$b_{\ell,k}$	M	S	C	a	F
3	19	3	1	18n + 30	2n + 16	20n + 1	20n + 64	0
				10n + 4	2n + 4	8n + 2	8n + 14	4n
	661	110	1	660n + 1314	2n + 658	662n + 1	662n + 2632	0
				224n + 218	2n + 218	222n + 109	222n + 656	4n
4	1013	23	11	1012n + 2018	2n + 1010	1014n + 1	1014n + 4040	0
				48n + 524	2n + 504	48n + 242	48n + 1074	2n
	1181	59	5	1180n + 2354	2n + 1178	1182n + 1	1182n + 4712	0
				120n + 596	2n + 588	120n + 290	120n + 1302	2n
5	31	1	3	30n + 54	2n + 28	32n + 1	32n + 112	0
				10n + 8	2n + 4	4n	4n + 14	8n
	61	6	1	60n + 114	2n + 58	62n + 1	62n + 232	0
				20n + 10	2n + 10	14n + 5	14n + 32	8n
7	29	2	1	28n + 50	2n + 26	30n + 1	30n + 104	0
				16n + 2	2n + 2	6n + 1	6n + 8	12n
	71	5	1	70n + 134	2n + 68	72n + 1	72n + 272	0
				22n + 8	2n + 8	12n + 4	12n + 26	12n
	547	39	1	546n + 1086	2n + 544	548n + 1	548n + 2176	0
				90n + 76	2n + 76	80n + 38	80n + 230	12n
8	881	55	2	880n + 1754	2n + 878	882n + 1	882n + 3512	0
				116n + 220	2n + 218	112n + 108	112n + 548	6n
9	37	2	1	36n + 66	2n + 34	38n + 1	38n + 136	0
				20n + 2	2n + 2	6n + 1	6n + 8	16n
	1693	94	1	1692n + 3378	2n + 1690	1694n + 1	1694n + 6760	0
				204n + 186	2n + 186	190n + 93	190n + 560	16n

Chapter 5

Kangaroos: Classical algorithms for restricted hidden shifts and high-dimensional discrete logarithms

5.1 Introduction

In this chapter, we consider classical (i.e., non-quantum) algorithms to solve two closely-related problems: the Multidimensional Discrete Logarithm Problem (MDLP) and the Bounded Hidden Shift Problem (BHSP).

Let $\mathcal{G}$ be a finite abelian group (written additively). Recall the *Discrete Logarithm Problem* (DLP): given P and Q , compute m such that $Q = [m]P$. We can always solve DLP instances in $O(\sqrt{\#\mathcal{G}})$ group operations (and low memory) using random-walk algorithms such as Pollard ρ [96] and van Oorschot and Wiener’s parallel collision search [118] (PCS).

The MDLP involves finding not one hidden scalar, but rather the hidden vector of coefficients in a (bounded) linear combination:

Definition 5.1.1 (MDLP) *Let $\mathcal{G}$ be an abelian group, and fix a list of elements $P_1, \dots, P_n$ in $\mathcal{G}$ and a subset $\mathcal{K}$ of $\mathbb{Z}^n$ (which we will suppose is a product of intervals). The **Multi-***

dimensional Discrete Logarithm Problem (MDLP) is, given $Q = \sum_{i=1}^n [e_i]P_i$ with $(e_1, \dots, e_n) \in \mathcal{K}$, find $(e_1, \dots, e_n)$.

The MDLP is interesting when $\#\mathcal{K}$ is much less than $\#\mathcal{G}$: otherwise, we can reduce to the DLP and solve with e.g. van Oorschot–Wiener. Low-dimensional MDLPs appear in classical DLP-based cryptography: the case $n = 1$ is the Interval-DLP, which we solve with Pollard’s *Kangaroo* algorithm [97], a random-walk-based algorithm which runs in only $O(\sqrt{\#\mathcal{K}})$ group operations—but which is also fundamentally limited to dimension $n = 1$. MDLPs also appear in algorithmic number theory: for example, Gaudry and Schost’s solves 2-dimensional MDLP algorithm [59], which also runs in square-root time $O(\sqrt{\#\mathcal{K}})$ (but with a larger implicit constant). It is a critical component of their record genus-2 point-counting computations [58, 60]. This algorithm was extended to dimension ≤ 4 , but no further, by Galbraith and Ruprai [56, 106]. One of our goals in Chapters 5 and 6 is to go beyond dimension $n = 4$.

5.1.1 From discrete-logarithm-based to group-action-based cryptography

Recall that the most promising candidates for cryptographic group actions are *isogeny-based*, using the class-group actions introduced in cryptography by Couveignes [33] and (independently) Rostovtsev and Stolbunov [105] (see Section 2.5.1). The reference cryptosystem nowadays is CSIDH [20] (see 2.5.2), where $\mathfrak{G}$ is the ideal class group of $\mathbb{Z}[\sqrt{-p}]$ and $\mathcal{X}$ is a set of supersingular elliptic curves over $\mathbb{F}_p$. In this case, $\#\mathcal{X} = \#\mathfrak{G}$ and $\log_2 \#\mathcal{X} \approx (1/2) \log_2 p$, and the action $(\mathfrak{a}, E) \mapsto \mathfrak{a} \star E$ is computed by viewing $\mathfrak{a}$ as an ideal in the endomorphism ring of E (we compute the isogeny from E with corresponding kernel).

Recall that one requirement for isogeny-based group actions (see Section 2.5) is computing $(\mathfrak{a}, E) \mapsto \mathfrak{a} \star E$ efficiently for arbitrary $\mathfrak{a}$: the complexity of computing the isogeny grows with the norm of $\mathfrak{a}$, which is generally exponential.¹ Couveignes suggested decomposing $\mathfrak{a}$ into an equivalent product $\mathfrak{a} \sim \mathfrak{l}_1^{e_1} \cdots \mathfrak{l}_n^{e_n}$ of ideals of small norm. This requires knowing the structure of the class group $\mathfrak{G}$, which is itself a subexponential classical computation [14, 18, 63] (though polynomial-time in the quantum setting, thanks to Shor’s algorithm). The class-group structure has been computed for 512-bit p in [13], but the quantum security of such p is disputed [11, 15, 23], and for larger p the class group is out of reach.

¹Some recent work [38, 41, 92, 93] achieves more efficient general class group actions through clever parametrization and the use of high-dimensional isogenies.

Restricted Effective Group Actions (REGAs) [3] (recall from Section 2.2.1) formalize the situation where the structure of $\mathfrak{G}$ is *not* known. Rather than decomposing $\mathfrak{a}$ into an equivalent product $\mathfrak{l}_1^{e_1} \cdots \mathfrak{l}_n^{e_n}$, we follow Rostovtsev and Stolbunov’s approach: fix (public) primes $\mathfrak{l}_1, \dots, \mathfrak{l}_n$ of very small norm, and then construct ideals $\mathfrak{a} := \prod_{i=1}^n \mathfrak{l}_i^{e_i}$ corresponding to exponent vectors $\mathbf{e} = (e_1, \dots, e_n)$ taken from some private keyspace $\mathcal{K} \subset \mathbb{Z}^n$. When the class group structure is not known, we can only conjecture that the public keyspace $\{\mathfrak{a} = \prod_i \mathfrak{l}_i^{e_i} : (e_1, \dots, e_n) \in \mathcal{K}\}$ is as big as $\mathcal{K}$, and its elements uniformly distributed over $\mathcal{X}$.

The bounded vectorization corresponding to attacking a CSIDH public key, or more generally vectorization in a REGA, is a *Bounded Hidden Shift Problem*:

Definition 5.1.2 (BHSP) *Let $\star : \mathbb{Z}^n \times \mathcal{X} \rightarrow \mathcal{X}$ be a commutative group action of $\mathbb{Z}^n$ on $\mathcal{X}$, and fix a finite subset $\mathcal{K}$ of $\mathbb{Z}^n$. The **Bounded Hidden Shift Problem** (BHSP) is, given P in $\mathcal{X}$ and $Q = (e_1, \dots, e_n) \star P$ with $(e_1, \dots, e_n) \in \mathcal{K}$, find a hidden shift $(e_1, \dots, e_n)$.*

The MDLP is a special case of the BHSP where $\mathcal{X} = \mathcal{G}$: fixing elements $P_1, \dots, P_n$ of $\mathcal{G}$ gives the action $(e_1, \dots, e_n) \star P := \sum_i [e_i]P_i + P$, and then taking $P = 0$ expresses the MDLP as a BHSP. Here, we prefer the language of group actions and BHSPs; translating back to the MDLP context is straightforward. We do this partly for simplicity in presentation; partly because we are looking towards applications in isogeny-based cryptography; and partly because we find that all of the algorithms we consider—including Pollard, van Oorschot–Wiener, Gaudry–Schost, Galbraith–Ruprai, and our own—are more naturally expressed in this framework.

We are particularly interested in the case where

$$\mathcal{K} = \mathcal{K}_{(B_1, \dots, B_n)} := [-B_1, B_1] \times \cdots \times [-B_n, B_n]$$

and, in particular, the case where

$$\mathcal{K} = \mathcal{K}_{B,n} := [-B, B]^n.$$

Remark 5.1.3 *The symmetric box $\mathcal{K}_{(B_1, \dots, B_n)}$ can be replaced with any other rectangular region $\prod_{i=1}^n [L_i, U_i]$ in $\mathbb{Z}^n$ if we replace P with $P' = \mathbf{t} \star P$ where $\mathbf{t} = \frac{1}{2}(L_1 - U_1, \dots, L_n - U_n)$. We will work with boxes centred around the origin because they are ubiquitous in the isogeny-based cryptosystems in our applications, but transposing our results to arbitrary rectangular regions (including those with sides of even length) is straightforward.*

Remark 5.1.4 *The group action in Definition 5.1.2 is an action of $\mathbb{Z}^n$, not $\mathfrak{G}$. This action always factors through the action of $\mathfrak{G}$ on $\mathcal{X}$ (with $(e_1, \dots, e_n)$ acting as $\prod_i \mathfrak{l}_i^{e_i}$ on*

$\mathcal{X}$), but defining the BHSP in terms of an action of $\mathbb{Z}^n$ allows us to abstract away the particular group $\mathfrak{G}$ (whose structure is generally not known anyway).

Remark 5.1.5 *When attacking CSIDH public keys, we do not necessarily need to solve a BHSP problem with shifts in the private keyspace $\mathcal{K}$: we can accept any solution for which the corresponding action can be evaluated efficiently.*

5.1.2 Dimension matters

Many well-known generic classical DLP algorithms, including Pollard ρ and van Oorschot–Wiener, but also Shanks’ Baby-step giant-step [107], generalize immediately to the group action setting (see e.g. [111]), solving general vectorization in $O(\sqrt{\#\mathfrak{G}})$ operations in $\mathfrak{G}$. These are not only the best known classical algorithms for general discrete logarithms but also the best known classical algorithms for *general* vectorization. For long-term security, then, while we need $\#\mathfrak{G}$ (and $\#\mathcal{X}$) large enough to resist subexponential quantum attacks, the private keyspace $\mathcal{K}$ can be taken just large enough to resist exponential square-root classical baby-step giant-step and quantum Grover attacks—and no larger, to minimise the cost of group action computations.

All of these algorithms are based on (parallel) collision search between random walks in $\mathcal{G}$, where the walks are probabilistically constrained to particular subsets of $\mathcal{G}$ derived from $\mathcal{K}$. As we noted above, so far this approach has only been made to work in dimension ≤ 4 . The fundamental reason for this is related to the nature of high-dimensional random walks: collision probabilities for unbounded walks drop quickly to zero, and in higher dimensions the walks are hard to bound effectively. These algorithms therefore cannot be applied in the high dimensions of REGA cryptanalysis: for example, the CSIDH-512 parameter set corresponds to dimension $n = 74$ (and bounds $B_1 = \dots = B_n = 5$).

Chi-Domínguez, Esser, Kunzweiler, and May [26] have recently defined a Parallel Collision Search (PCS) algorithm for vectorization targeting $\mathcal{K} = \{-1, 0, 1\}^n$. Under standard heuristics, given space $M \leq 3^{3n/2}$, their algorithm runs in an expected number of operations

$$\frac{3^{n/4}}{\sqrt{M}} \cdot \sqrt{\#\mathcal{K}} \quad \text{when } \mathcal{K} = \{-1, 0, 1\}^n.$$

They also give improved algorithms for smaller M , and extensions to $\mathcal{K} = \{-m, \dots, m\}$ for small m . While the performance of their approach actually improves as m grows, it is still far from a pure square-root algorithm in low memory: for example, with constant M , the runtime complexity is $O((\#\mathcal{K})^{0.75})$ when $m = 1$, $O((\#\mathcal{K})^{0.629})$ when $m = 2$, and $O((\#\mathcal{K})^{0.618})$ when $m = 3$.

5.1.3 A new high-dimensional algorithm

In this chapter we introduce a family of low-memory algorithms to solve MDLP and BHSP instances. Our algorithms are inspired by older MDLP algorithms: namely, Pollard’s Kangaroo algorithm [97], which is fundamentally limited to dimension $n = 1$, and the Gaudry–Schost algorithm [59], designed for $n = 2$ and extended to $n \leq 4$ by Galbraith and Ruprai [56].

We break the 4-dimensional barrier, introducing significant changes to overcome the high-dimensional effects that obstructed Galbraith and Ruprai. These changes are inspired by a hint from Pollard cited in [56, Appendix A]. In this chapter, we examine what happens when these ideas are put into practice. The resulting algorithms run in expected time $\approx \alpha^n \sqrt{\pi \# \mathcal{K}}$, where α is a function of $\mathcal{K}$ which converges to a small constant as $\# \mathcal{K}$ grows.

5.2 Group actions and group operations

Before going further, we should provide a model for group action computations. We call actions by the unit vectors $\mathbf{e}^{(i)}$ in $\mathbb{Z}^n$ (and their negatives) *elementary shifts*. Of course, in concrete settings different elementary shifts may have wildly different costs, especially in the case of actions by isogeny computations.

We express the complexity of our algorithms in terms of actions by arbitrary elements of $\mathcal{K}$, which we call $\mathcal{K}$ -shifts. We can always compute a $\mathcal{K}$ -shift as a composition of elementary shifts. For example, if $\mathcal{K} = \mathcal{K}_{(B_1, \dots, B_n)}$, then

- any $\mathcal{K}$ -shift can be computed as the composition of at most $\sum_i B_i$ elementary shifts; and
- the expected cost of a uniformly randomly sampled $\mathcal{K}$ -shift is $\sum_{i=1}^n B_i(B_i + 1)/(2B_i + 1)$ elementary shifts.

In some concrete situations, $\mathcal{K}$ -shifts may be computed much more efficiently: for example, if $\mathcal{X}$ is a DLP group $\mathcal{G} = \langle P_1, \dots, P_n \rangle$ with $\mathbb{Z}^n$ acting via $(e_1, \dots, e_n) \star Q := Q + \sum_{i=1}^n [e_i]P_i \in \mathcal{G}$, then $\mathcal{K}$ -shifts cost $O(\sum_{i=1}^n \log(B_i))$ rather than $O(\sum_{i=1}^n B_i)$ elementary shifts.

We will therefore leave the bit-operation cost of computing elementary shifts and $\mathcal{K}$ -shifts aside here; the reader can make appropriate substitutions according to their application context.

5.3 Solving the MDLP in dimensions ≤ 4

We begin with a review of the Gaudry–Schost algorithm [59] and its improvements due to Galbraith and Ruprai [56] for dimensions ≤ 4 . Our analysis follows Ruprai’s thesis [106], which contains more details on virtually all of the results in this section. We will use the language of the MDLP in this section, reflecting the motivations of the original algorithms, but it will be clear that these algorithms can be immediately adapted to the BHSP.

Pollard’s classic Kangaroo method [97] runs in time $O(\sqrt{\#\mathcal{K}})$, but it is intrinsically limited to dimension $n = 1$. While it is also based on collision-finding via random walks, its design and analysis are different to those of the algorithms we will consider below, so we will not discuss it in detail here.

5.3.1 The Gaudry–Schost algorithm

The Gaudry–Schost algorithm uses pseudorandom walks to solve MDLP instances in dimension $n \leq 4$. We are given an element Q of the space $\mathcal{N} = \{\sum_{i=1}^n [a_i]P_i : a_i \in \prod_{i=1}^n [-B_i, B_i]\}$; we want to find the shift vector $\mathbf{x} = (x_1, \dots, x_n)$ such that $\sum_{i=1}^n [x_i]P_i = Q$. We can fix a set of precomputed linear combinations $\sum_{i=1}^n [c_i]P_i$ (corresponding to the Tame kangaroo), each stored with its coefficient vector $(c_1, \dots, c_n)$.

In this algorithm a Tame kangaroo $\mathcal{T}$ and a Wild kangaroo $\mathcal{W}$ jump (i.e., take pseudorandom shift) in the sets

$$\mathcal{N}_T = \mathcal{N} \quad \text{and} \quad \mathcal{N}_W = Q + \mathcal{N},$$

respectively. Each kangaroo $(K, \mathbf{v}, s)$ consists of its footprint K (an element of the group), a shift vector $\mathbf{v}$, and a type s (Tame or Wild). At each jump, $(K, \mathbf{v}, s)$ is replaced with

$$(K, \mathbf{v}, s) \mapsto (K + ([c_1]P_1 + \dots + [c_n]P_n), \mathbf{v} + (c_1, \dots, c_n), s),$$

where the shift vector $(c_1, \dots, c_n)$ is pseudorandomly selected using a hash of K . Every time a kangaroo hits a *distinguished point*, we save the kangaroo’s state $(K, \mathbf{v}, s)$ in a hash table indexed by K . A distinguished point (DP) is any footprint $K \in \mathcal{N}$ satisfying a public predicate $P(K)$ that holds with probability θ (e.g., the first t bits of $H(K)$ are zero, so $\theta = 2^{-t}$). Each kangaroo reports and stores only when $P(K) = 1$; this creates sparse “landmarks” (expected spacing $1/\theta$ steps).

Kangaroos keep on jumping until a Tame–Wild collision is found: that is, a collision in the hash table reveals kangaroos $(K, \mathbf{a}, \text{Tame})$ and $(K, \mathbf{b}, \text{Wild})$ with the same footprint K but opposite types. Then we know $\sum_i [a_i]P_i = Q + \sum_i [b_i]P_i$, and we recover $\mathbf{x} = \mathbf{a} - \mathbf{b}$.

If one of the kangaroos jumps out of the search space, then we restart it from a random point. That point must not have already been visited by another kangaroo of the same type. To ensure this, we check for about $1/\theta$ jumps that the kangaroo has not landed on a distinguished point already visited by another walk of the same type. In the literature, it is usual to force a restart if a walk does not hit a distinguished point after $20/\theta$ jumps.

To calculate the number of jumps needed for this algorithm to succeed, we apply the birthday paradox to the overlap between $\mathcal{N}_W$ and $\mathcal{N}_T$. Consider two lists sampled alternatively and uniformly at random from a set of size M . The expected number of times we need to sample an element to store in each list before a collision between the two lists is $\sqrt{\pi M} + O(1)$ (see [88, 106]). When this algorithm is applied to the MDLP, therefore,

1. the expected number of jumps in the worst case is $(\sqrt{2})^n \sqrt{\pi \# \mathcal{N}}$; and
2. the expected number of jumps in the average case is $((4 - 2\sqrt{2})^n + o(1)) \sqrt{\pi \# \mathcal{N}}$.

(See [59] or [56] for further details and proof.)

Galbraith and Ruprai [56] defined a version of the Gaudry–Schost algorithm with a constant expected running time no matter where Q is situated in $\mathcal{N}$. To do so, they designed a constant overlap between $\mathcal{N}_W$ and $\mathcal{N}_T$. The running time is directly affected by the size of the overlap, the bigger it is, the faster the algorithm runs since Kangaroos take more jumps in the collision zone. A constant overlap removes the difference between the worst and best case that is in the original Gaudry–Schost algorithm. The key idea is to replace $\mathcal{N}_T$ and $\mathcal{N}_W$ with

$$\mathcal{N}_T = \prod_{i=1}^n [-2B_i/3; 2B_i/3], \quad \mathcal{N}_W = Q + \left(\prod_{i=1}^n [-B_i; -B_i/3] \cup \prod_{i=1}^n [B_i/3; B_i] \right),$$

(Figure 5.1 illustrates the two-dimensional case). Note that $\#(\mathcal{N}_T \cap \mathcal{N}_W) \approx \frac{1}{3^n} \# \mathcal{N}$, and the new search space has size $\frac{2^n}{3^n} \# \mathcal{N}$. With this $\mathcal{N}_T$ and $\mathcal{N}_W$, the expected number of jumps before a Tame-Wild collision is $(\sqrt{4/3})^n + o(1)) \sqrt{\pi \# \mathcal{N}}$, *irrespective of $\mathbf{x}$* .

5.3.2 Dimensional limitations

As Galbraith and Ruprai observe in [56], these algorithms are not effective in dimension > 4 with constant storage. Indeed, to achieve constant storage, we must reduce the proportion of distinguished points as the search space grows. In the Gaudry–Schost algorithm, we expect to store approximately $t\theta$ points, where t is the number of group operations. Since

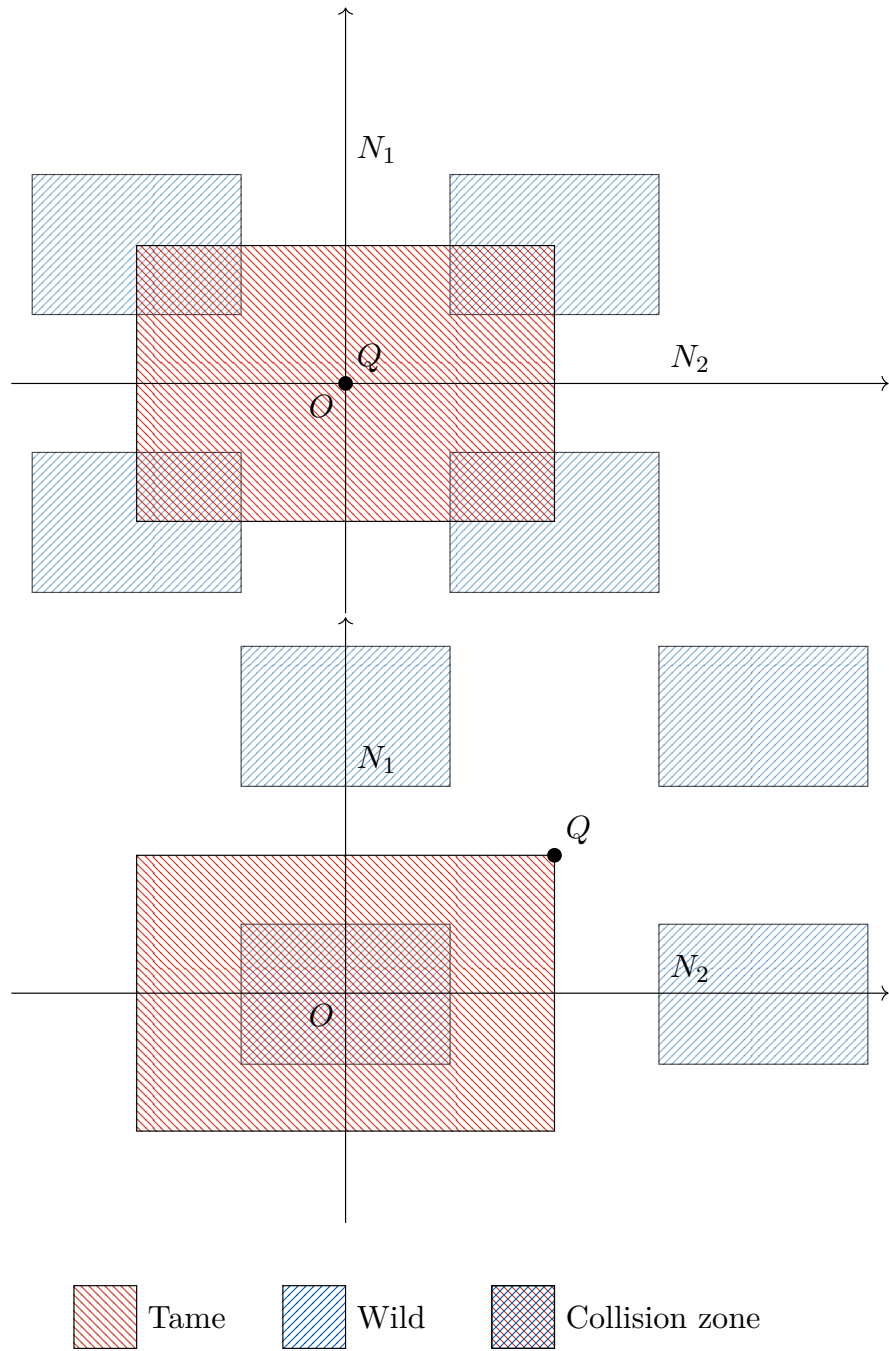


Figure 5.1: Galbraith–Ruprai Wild and Tame kangaroo jumping zones in dimension 2. The volume of the collision zone is constant, irrespective of Q .

$t = O(\sqrt{\#\mathcal{N}})$, low-memory implies $\theta = c/\sqrt{\#\mathcal{N}}$ for some constant $c > 1$, so the mean length of a walk before hitting a distinguished point is $(m\sqrt{\#\mathcal{N}})/c$, where m is the mean jump size. In large dimension, we tend to jump beyond the boundaries B_i (which forces a random restart) before saving the walk to memory and thus the Tame-Wild Birthday paradox approach fails.

Furthermore, as the dimension increases, so does the probability that the starting position of a walk is close to the boundaries—and then we jump out of the search space very quickly. Finally, it is worth noting that the jump spaces in the improved Gaudry–Schost algorithm by Galbraith–Ruprai [56] are difficult to instantiate for small boundaries. Indeed, in this improved version, $\mathcal{N}_W$ and $\mathcal{N}_T$ are reduced to approximately 2/3 of the original size, with constant overlap. Reducing the search space in such a way is not possible for bounds equal to 1.

5.4 Special points

Pollard suggested a method to overcome this boundary issue in the appendix of [56]: declare a subset of points to be “special”, with the idea that a kangaroo starting in a box will hit a special point with about the same probability as it is likely to have left the box. On encountering a special point, it “restarts” from a pseudorandom point back in a box but most importantly it also is pseudorandomly Tame or Wild. This is to counter lost Tame-Wild collisions due to some restarts occurring on the Tame kangaroo path but not the Wild kangaroo path since their search spaces are not the same. The addition of special points to the original Gaudry–Schost algorithm is the focus of Chapters 5 and 6.

When B is small, or when n is large, it is simpler and just as effective to declare *every* point special, and “restart” on every single jump. This is the approach that we investigate in this chapter. Making this assumption gives us a nice first approach to the problem and will give us more confidence in the analysis of the general case in Chapter 6.

Definition 5.4.1 (Jump, Step, Restart) *From now on, a jump means any move of the kangaroo to any location inside the allowed jumping zone. Steps and restarts are special cases of jumps:*

- *A step is a jump obtained by applying an elementary shift. It keeps a kangaroo type untouched.*
- *A restart is a jump that moves the kangaroo to a pseudorandom location in space (still within the allowed jumping zone). The kangaroo type is pseudorandomly*

chosen.

In this chapter, every move we consider is a jump; steps and restarts are included in this terminology.

5.5 Solving hidden shifts by random sampling

Our first step is an idealised BHSP algorithm based on pure random sampling, without considering how collisions will be detected—or the memory that this will require. We will transform this into a working algorithm, and deduce the resulting time and memory complexities, in Section 5.8.

5.5.1 Solving hidden shifts by random sampling

We are given P and $Q = \mathbf{x} \star P$ for some unknown $\mathbf{x}$ in $\mathcal{K}$; our mission is to find $\mathbf{x}$. We begin by fixing a subset of “tame” shifts $\mathcal{S}_{\text{Tame}} \subset \mathbb{Z}^n$ and a subset of “wild” shifts $\mathcal{S}_{\text{Wild}} \subset \mathbb{Z}^n$. For Tame-Wild collisions to be possible, we must choose $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$ such that $\mathbf{x} \in \mathcal{S}_{\text{Wild}} - \mathcal{S}_{\text{Tame}}$. But since $\mathbf{x}$ is unknown, we must ensure this holds for *all* $\mathbf{x}$ in $\mathcal{K}$: that is,

$$\mathcal{K} \subseteq \mathcal{S}_{\text{Wild}} - \mathcal{S}_{\text{Tame}}. \quad (5.1)$$

Now, we generate a sequence of tuples $(X^{(i)}, \mathbf{v}^{(i)}, s^{(i)})$ in $\mathcal{N} \times \mathbb{Z}^n \times \{\text{Tame}, \text{Wild}\}$, where the i -th tuple is constructed as follows:

1. First, we flip a coin to select either Tame or Wild.
2. If the result is Tame, then we sample $\mathbf{v}^{(i)}$ uniformly from $\mathcal{S}_{\text{Tame}}$, compute $X^{(i)} := \mathbf{v}^{(i)} \star P$, and let $s_i := \text{Tame}$;
3. otherwise, we sample $\mathbf{v}^{(i)}$ uniformly from $\mathcal{S}_{\text{Wild}}$, compute $X^{(i)} := \mathbf{v}^{(i)} \star Q$, and let $s_i := \text{Wild}$.
4. Finally, we append $(X^{(i)}, \mathbf{v}^{(i)}, s^{(i)})$ to the sequence.

A *collision* is a pair of samples $(X^{(i)}, \mathbf{v}^{(i)}, s^{(i)})$ and $(X^{(j)}, \mathbf{v}^{(j)}, s^{(j)})$ such that $X^{(i)} = X^{(j)}$. The collision is Tame-Tame resp. Wild-Wild if $s^{(i)} = s^{(j)} = \text{Tame}$ resp. Wild, and Tame-Wild if $s^{(i)} \neq s^{(j)}$.

Tame-Tame and Wild-Wild collisions give us no useful information. However, if we detect a Tame-Wild collision then we can recover the target vector as

$$\mathbf{x} = \begin{cases} \mathbf{v}^{(j)} - \mathbf{v}^{(i)} & \text{if } (s^{(i)}, s^{(j)}) = (\text{Tame}, \text{Wild}), \\ \mathbf{v}^{(i)} - \mathbf{v}^{(j)} & \text{if } (s^{(i)}, s^{(j)}) = (\text{Wild}, \text{Tame}). \end{cases}$$

What we are implicitly defining here is a random walk $(X^{(1)}, X^{(2)}, \dots)$ in the “total space”

$$\begin{aligned} \mathcal{X}_{\text{Total}}(\mathbf{x}) &:= (\mathcal{S}_{\text{Tame}} \star P) \cup (\mathcal{S}_{\text{Wild}} \star Q) \\ &= (\mathcal{S}_{\text{Tame}} \cup (\mathcal{S}_{\text{Wild}} + \mathbf{x})) \star P. \end{aligned}$$

Tame-Wild collisions can only be found in the “intersection space”

$$\begin{aligned} \mathcal{X}_{\text{Int}}(\mathbf{x}) &:= (\mathcal{S}_{\text{Tame}} \star P) \cap (\mathcal{S}_{\text{Wild}} \star Q) \\ &= (\mathcal{S}_{\text{Tame}} \cap (\mathcal{S}_{\text{Wild}} + \mathbf{x})) \star P \end{aligned}$$

—which, like $\mathcal{X}_{\text{Total}}(\mathbf{x})$, is unknown when $\mathbf{x}$ is secret. We would like to ensure $\mathcal{X}_{\text{Int}}(\mathbf{x})$ is *probably* large compared with $\mathcal{X}_{\text{Total}}(\mathbf{x})$, at least for a large proportion of $\mathbf{x}$ in $\mathcal{K}$, to ensure that collisions are relatively easy to find. But in high dimensions, no matter what our choice for $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$, for most $\mathbf{x}$, the intersection $\mathcal{X}_{\text{Int}}(\mathbf{x})$ is condemned to be *much* smaller than $\mathcal{S}_{\text{Tame}}$ or $\mathcal{S}_{\text{Wild}}$.

For the remainder of this section we will look at choices of $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$, and their impact on the unknown $\mathcal{X}_{\text{Int}}(\mathbf{x})$ and thus Tame-Wild collision probabilities. We will return to the random walk itself, and to how it can be effectively replaced with a pseudorandom walk in a low-memory algorithm, in Section 5.8.

5.5.2 The Birthday Paradox

The probability of finding a Tame-Wild collision depends on $\#\mathcal{S}_{\text{Tame}}$, $\#\mathcal{S}_{\text{Wild}}$, and $\#\mathcal{X}_{\text{Int}}(\mathbf{x})$. The probability of finding Tame-Tame and Wild-Wild collisions, on the other hand, depend only on $\#\mathcal{S}_{\text{Tame}}$ and $\#\mathcal{S}_{\text{Wild}}$, respectively. Theorem 5.5.1 gives the expected number of samples before the first collisions are found.

Theorem 5.5.1 *As $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) \rightarrow \infty$, the expected number of samples before the first Tame-Tame resp. Wild-Wild collisions are*

$$\sqrt{2\pi\#\mathcal{S}_{\text{Tame}}} + o(\sqrt{\#\mathcal{S}_{\text{Tame}}}) \quad \text{resp.} \quad \sqrt{2\pi\#\mathcal{S}_{\text{Wild}}} + o(\sqrt{\#\mathcal{S}_{\text{Wild}}}).$$

The expected number of samples before the first Tame-Wild collision is

$$\sqrt{\pi D(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})} + O(E(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}))$$

where

$$D(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) := \frac{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})}$$

and

$$E(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) := D(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) \cdot \frac{\#\mathcal{X}_{\text{Int}}(\mathbf{x})^{1/4}}{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}.$$

Proof. Each sample randomly selects an element of $\mathcal{X}_{\text{Total}}(\mathbf{x})$ as either a random element of $\mathcal{S}_{\text{Tame}} \star P$ (a Tame sample) with probability $1/2$ or a random element of $\mathcal{S}_{\text{Wild}} \star Q$ (a Wild sample) with probability $1/2$. The first Tame-Tame collision is the first collision in the subsequence of samples in $\mathcal{S}_{\text{Tame}} \star P$: the standard birthday paradox says this occurs after an expected $\sqrt{\pi/2 \#\mathcal{S}_{\text{Tame}}} + O(\sqrt{\#\mathcal{S}_{\text{Tame}}})$ Tame samples, each of which requires an expected 2 samples to produce, which yields the first result. The same argument gives the expected number of samples before the first Wild-Wild collision.

For Tame-Wild collisions, look at the subsequence of random samples that land in $\mathcal{X}_{\text{Int}}(\mathbf{x})$. The probability that a Tame resp. Wild sample lands in $\mathcal{X}_{\text{Int}}(\mathbf{x})$ is $\#\mathcal{X}_{\text{Int}}(\mathbf{x})/\#\mathcal{S}_{\text{Tame}}$ resp. $\#\mathcal{X}_{\text{Int}}(\mathbf{x})/\#\mathcal{S}_{\text{Wild}}$. Given that a sample is in $\mathcal{X}_{\text{Int}}(\mathbf{x})$, it is Tame with probability

$$q_{\text{Tame}} = \frac{\#\mathcal{X}_{\text{Int}}(\mathbf{x})/\#\mathcal{S}_{\text{Tame}}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})/\#\mathcal{S}_{\text{Tame}} + \#\mathcal{X}_{\text{Int}}(\mathbf{x})/\#\mathcal{S}_{\text{Wild}}} = \frac{\#\mathcal{S}_{\text{Wild}}}{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}$$

and similarly, the sample is Wild with probability

$$q_{\text{Wild}} = \frac{\#\mathcal{S}_{\text{Tame}}}{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}.$$

The samples are distributed uniformly over $\mathcal{X}_{\text{Int}}(\mathbf{x})$, so applying the 2-ball case of [57, Theorem 1] (with the urns being the points in $\mathcal{X}_{\text{Int}}(\mathbf{x})$) tells us that the expected number of samples *in the subsequence* before the first Tame-Wild collision in $\mathcal{X}_{\text{Int}}(\mathbf{x})$ is

$$\sqrt{\frac{\pi \cdot \#\mathcal{X}_{\text{Int}}(\mathbf{x})}{4q_{\text{Tame}}q_{\text{Wild}}}} + O(\#\mathcal{X}_{\text{Int}}(\mathbf{x})^{1/4})$$

which is

$$\frac{1}{2} \cdot \frac{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}{\sqrt{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}} \cdot \sqrt{\pi \#\mathcal{X}_{\text{Int}}(\mathbf{x})} + O(\#\mathcal{X}_{\text{Int}}(\mathbf{x})^{1/4}).$$

On average, each of these samples in the subsequence costs

$$q_{\text{Tame}} \frac{\#\mathcal{S}_{\text{Tame}}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})} + q_{\text{Wild}} \frac{\#\mathcal{S}_{\text{Wild}}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})} = \frac{2}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})} \cdot \frac{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}$$

“real” samples (in $\mathcal{X}_{\text{Total}}(\mathbf{x})$) to produce, so the expected number of samples before a Tame-Wild collision is found is

$$\frac{1}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})} \cdot \frac{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}} \left[\frac{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}{\sqrt{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}} \cdot \sqrt{\pi \#\mathcal{X}_{\text{Int}}(\mathbf{x})} + O(\#\mathcal{X}_{\text{Int}}(\mathbf{x})^{1/4}) \right]$$

which is

$$\sqrt{\pi} \sqrt{\frac{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})}} + O\left(\frac{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})} \cdot \frac{\#\mathcal{X}_{\text{Int}}(\mathbf{x})^{1/4}}{\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}}}\right),$$

or $2\sqrt{\pi D(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})} + O(E(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}))$, as claimed. $\square$

Remark 5.5.2 *It is tempting to sample Tame and Wild with different probabilities, especially if $\#\mathcal{S}_{\text{Tame}}$ and $\#\mathcal{S}_{\text{Wild}}$ are significantly unbalanced. Suppose a sample is Tame with probability f and Wild with probability $1 - f$. Repeating the analysis of Theorem 5.5.1, the total number of required samples is multiplied by $(f - f^2)^{-n/2}$; but this factor is minimised when $f = 1/2$. Sampling Tame and Wild with equal probability is therefore the optimal approach, regardless of $\#\mathcal{S}_{\text{Tame}}$ and $\#\mathcal{S}_{\text{Wild}}$.*

5.5.3 Comparing with the ideal birthday bound

It is helpful to compare the result of Theorem 5.5.1 with the classic “ideal” birthday bound $\sqrt{\pi \#\mathcal{K}}$ for collisions in $\mathcal{K}_{B,n}$. This also gives a good comparison with baby-step giant-step analogues.

Definition 5.5.3 *For all $\mathbf{x}$ in $\mathcal{K}$, we define*

$$\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) := \left(\frac{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}}{\#\mathcal{K} \cdot \#\mathcal{X}_{\text{Int}}(\mathbf{x})} \right)^{1/(2n)}.$$

Using this notation, we can re-state the Tame-Wild statement of Theorem 5.5.1 as follows: as $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) \rightarrow \infty$, the expected number of samples before the first Tame-Wild collision is

$$\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n \sqrt{\pi \#\mathcal{K}} + O(E(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})).$$

Remark 5.5.4 *The fundamental quantity here is $\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n$, which measures the difference between the expected performance of this process and an ideal birthday paradox in $\mathcal{K}$. In most applications, the derived quantity $\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})$ can be found to simplify and be independent of the dimension n .*

Remark 5.5.5 If $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$ are chosen such that $\mathbf{x}$ is not in $\mathcal{S}_{\text{Tame}} - \mathcal{S}_{\text{Wild}}$, then $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) = 0$, and the quantity $\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})$ is infinite—which reflects the fact that a collision will never be found.

The expected number of samples before the first Tame-Wild collision varies with the size of the intersection $\mathcal{X}_{\text{Int}}(\mathbf{x})$. The mean number of samples as $\mathbf{x}$ varies over $\mathcal{K}$ is determined by the expected value of $\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n$.

Definition 5.5.6 We define

$$\bar{\alpha}(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}) := (\mathbb{E}(\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n))^{1/n}.$$

We emphasize that $\bar{\alpha}$ is not the expected value of α (which is both harder to calculate, and much less useful to us).

5.6 Tame and Wild shift sets

In view of Theorem 5.5.1, we want to choose shift sets $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$ such that the expected value of $(\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}) / \#\mathcal{X}_{\text{Int}}(\mathbf{x})$ is as small as possible, and that the expected $\#\mathcal{X}_{\text{Int}}(\mathbf{x})$ is sufficiently large for the birthday paradox to apply (and to ensure that the error term is relatively small). Indeed, if $\#\mathcal{X}_{\text{Int}}(\mathbf{x})$ is as small as $O(1)$ then the expected number of samples becomes not $O(\sqrt{\#\mathcal{S}_{\text{Tame}} \cdot \#\mathcal{S}_{\text{Wild}}})$ but $O(\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}})$, which can be strictly greater (by the inequality of the geometric and arithmetic means).

Our shift sets will be products of intervals, so for simplicity we reduce immediately to the case

$$\mathcal{K} = \mathcal{K}_{B,n} := [-B, B]^n.$$

The results for $\mathcal{K}_{B,n}$ below can easily be extended and combined to define useful shift sets for more general keyspaces $\mathcal{K}_{(B_1, \dots, B_m), (n_1, \dots, n_m)} := \prod_{i=1}^m \mathcal{K}_{B_i, n_i}$ using standard properties of the Cartesian product of finite sets.

The simplest choice to ensure $\mathcal{K}_{B,n} \subseteq \mathcal{S}_{\text{Tame}} - \mathcal{S}_{\text{Wild}}$, while maintaining a reasonably large $\mathcal{X}_{\text{Int}}(\mathbf{x})$, is to take $\mathcal{S}_{\text{Tame}} = \mathcal{S}_{\text{Wild}} = \mathcal{K}_{B,n}$. Then

$$\mathcal{X}_{\text{Int}}(\mathbf{x}) = \prod_{i=1}^n ([-B, B] \cap [x_i - B, x_i + B]),$$

so $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) = \prod_{i=1}^n (2B + 1 - |x_i|)$ and thus

$$\alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{x}) = \left(\frac{\#\mathcal{K}_{B,n}}{\#\mathcal{X}_{\text{Int}}(\mathbf{x})} \right)^{1/2n} = \prod_{i=1}^n \sqrt{\frac{2B + 1}{2B + 1 - |x_i|}}^{1/n}. \quad (5.2)$$

We begin with some easy bounds on $\alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{x})$: by definition

$$1 \leq \alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{x}) \leq \alpha_{\max}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) := \sqrt{2 - 1/(B+1)} < \sqrt{2} \approx 1.41421.$$

These bounds are tight: $\alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{0}) = 1$ (and $\mathbf{0}$ is the only such vector) while $\alpha_{\max}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) = \alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; (\pm B, \dots, \pm B))$. The upper bound tends to $\sqrt{2}$ as $B \rightarrow \infty$, but is significantly smaller for small B : for example, $\alpha_{\max}(\mathcal{K}_{1,n}, \mathcal{K}_{1,n}) = \sqrt{3/2} \approx 1.22474487$.

Proposition 5.6.1 *With the above notation:*

$$\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) = \frac{1}{\sqrt{2B+1}} \left(\frac{1}{\sqrt{2B+1}} + \frac{2}{\sqrt{2B}} + \dots + \frac{2}{\sqrt{B+1}} \right).$$

Proof. Let $k_j := \#\{1 \leq i \leq n : |x_i| = j\}$ for $0 \leq j \leq B$; note that $\sum_{j=0}^B k_j = n$. Grouping factors in the product in (5.2) by value of $|x_i|$ gives

$$\alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{x})^n = (2B+1)^{n/2} \prod_{j=0}^B \left(\frac{1}{\sqrt{2B+1-j}} \right)^{k_j}.$$

There are $(2B+1)^n$ different $\mathbf{x}$, and $\binom{n}{k_0, \dots, k_B} 1^{k_0} \cdot 2^{k_1} \dots 2^{k_B}$ different $\mathbf{x}$ with a given $(k_0, \dots, k_B)$: the factors of 2 correspond to choices of sign on the nonzero x_i . We therefore have

$$\begin{aligned} \bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{x})^n &= \mathbb{E}(\alpha(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}; \mathbf{x})^n) \\ &= \frac{1}{(2B+1)^n} \sum_{(k_0, \dots, k_B)} \binom{n}{k_0, \dots, k_B} 2^{k_1 + \dots + k_B} (2B+1)^{n/2} \prod_{j=0}^B \left(\frac{1}{\sqrt{2B+1-j}} \right)^{k_j} \\ &= \frac{1}{(2B+1)^{n/2}} \left(\frac{1}{\sqrt{2B+1}} + \sum_{j=1}^B \frac{2}{\sqrt{B+j}} \right)^n, \end{aligned}$$

and taking n -th roots yields the result. $\square$

We can also give some easy upper and lower bounds on $\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n})$. First, note that the function $B \mapsto \bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n})$ is strictly increasing on $\mathbb{Z}_{>0}$, so

$$\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) \geq \bar{\alpha}(\mathcal{K}_{1,n}, \mathcal{K}_{1,n}) = \frac{1}{3} + \sqrt{\frac{2}{3}} \approx 1.14983. \quad (5.3)$$

Also, for integers $1 < L \leq U$ we have $2(\sqrt{U+1} - \sqrt{L}) \leq \sum_{j=L}^U 1/\sqrt{j} < 2(\sqrt{U} - \sqrt{L-1})$, with $L = B+1$ and $U = 2B$, which gives a lower bound

$$\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) \geq 4 - 2\sqrt{2} \sqrt{1 + \frac{1}{2B+1}} + \frac{1}{2B+1} \quad (5.4)$$

(which is actually weaker than the bound of (5.3) for $B \leq 8$) and an upper bound

$$\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) < (4 - 2\sqrt{2})\sqrt{1 - \frac{1}{2B+1}} + \frac{1}{2B+1}. \quad (5.5)$$

The bounds of (5.4) and (5.5) converge to a common limit:

$$\lim_{B \rightarrow \infty} \bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) = 4 - 2\sqrt{2} \approx 1.17157. \quad (5.6)$$

We see in Table 5.1 that $\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n})$ approaches its $B \rightarrow \infty$ limit quickly.

5.7 “Two-thirds” shift sets

We will now consider a variant designed to make $\#\mathcal{X}_{\text{Int}}(\mathbf{x})$ relatively large, and *constant*, by Galbraith and Ruprai’s Tame and Wild sets from [56]. This gives better results in many cases.

The essential idea can already be given in dimension one. Suppose $\mathcal{K} = [-B, B]$ for some $B > 1$, and let $\mathcal{S}_{\text{Tame}} = [-2B/3, 2B/3]$ and $\mathcal{S}_{\text{Wild}} = [-B, -B/3] \cup [B/3, B]$ (with some choice of rounding). Then $\#\mathcal{S}_{\text{Tame}} \approx \#\mathcal{S}_{\text{Wild}} \approx 2/3\#\mathcal{K}_{B,n}$ and, most interestingly, $\#(\mathcal{S}_{\text{Wild}} \cap (\mathbf{x} + \mathcal{S}_{\text{Tame}})) \approx 1/3\#\mathcal{K}_{B,n}$ for all $\mathbf{x}$. More precise statements depend on rounding, especially when B is small—which is the case in our applications. The following three propositions propose a choice of Tame and Wild shift sets depending on $B \bmod 3$.

Proposition 5.7.1 *Suppose $B \equiv 0 \pmod{3}$. Set $g := \frac{1}{3}B$, and let*

$$\mathcal{S}_{\text{Tame}} = [-2g, 2g]^n \quad \text{and} \quad \mathcal{S}_{\text{Wild}} = ([-B, -g-1] \cup [g+1, B])^n.$$

Then for all $\mathbf{x}$ in $\mathcal{K}_{B,n}$, we have

$$\#\mathcal{S}_{\text{Tame}} = (4g+1)^n, \quad \#\mathcal{S}_{\text{Wild}} = (4g)^n, \quad \#\mathcal{X}_{\text{Int}}(\mathbf{x}) = (2g)^n,$$

and thus

$$\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) = \sqrt{\frac{8g+2}{6g+1}} = \sqrt{\frac{4}{3} + \frac{2}{3(2B+1)}} \quad \text{for all } \mathbf{x} \in \mathcal{K}_{B,n}.$$

Proof. Note that $2B+1 = 6g+1$. It suffices to check the formula for $\#\mathcal{X}_{\text{Int}}(\mathbf{x})$ in the one-dimensional case. There

$$\mathcal{S}_{\text{Tame}} \cap (x + \mathcal{S}_{\text{Wild}}) = \begin{cases} [x+g+1, x+3g] & \text{if } -B \leq x < -g, \\ [-2g, x-g-1] \cup [x+g+1, 2g] & \text{if } -g \leq x \leq g, \\ [x-3g, x-g-1] & \text{if } g < x \leq B. \end{cases}$$

In each case, $\#(\mathcal{S}_{\text{Tame}} \cap (x + \mathcal{S}_{\text{Wild}})) = 2g$, as claimed. $\square$

Proposition 5.7.2 Suppose $B \equiv 1 \pmod{3}$ and $B > 1$. Set $g := \frac{1}{3}(B - 1)$, and let

$$\mathcal{S}_{\text{Tame}} = [-2g - 1, 2g + 1]^n \quad \text{and} \quad \mathcal{S}_{\text{Wild}} = ([-B, -g - 2] \cup [g + 2, B])^n.$$

Then for all $\mathbf{x}$ in $\mathcal{K}_{B,n}$, we have

$$\#\mathcal{S}_{\text{Tame}} = (4g + 3)^n, \quad \#\mathcal{S}_{\text{Wild}} = (4g)^n, \quad \#\mathcal{X}_{\text{Int}}(\mathbf{x}) = (2g)^n,$$

and thus

$$\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) = \sqrt{\frac{8g + 6}{6g + 3}} = \sqrt{\frac{4}{3} + \frac{2}{2B + 1}} \quad \text{for all } \mathbf{x} \in \mathcal{K}_{B,n}.$$

Proof. Same as for Proposition 5.7. □

Remark 5.7.3 Proposition 5.7.2 cannot be used when $B = 1$, because $\mathcal{S}_{\text{Wild}} = \emptyset$.

Proposition 5.7.4 Suppose $B \equiv 2 \pmod{3}$. Set $g := \frac{1}{3}(B + 1)$, and let

$$\mathcal{S}_{\text{Tame}} = [-2g + 1, 2g - 1]^n \quad \text{and} \quad \mathcal{S}_{\text{Wild}} = ([-B, -g] \cup [g, B])^n.$$

Then for all $\mathbf{x}$ in $\mathcal{K}_{B,n}$, we have

$$\#\mathcal{S}_{\text{Tame}} = (4g - 1)^n, \quad \#\mathcal{S}_{\text{Wild}} = (4g)^n, \quad \#\mathcal{X}_{\text{Int}}(\mathbf{x}) = (2g)^n,$$

and thus

$$\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) = \sqrt{\frac{8g - 2}{6g - 1}} = \sqrt{\frac{4}{3} - \frac{2}{3(2B + 1)}} \quad \text{for all } \mathbf{x} \in \mathcal{K}_{B,n}.$$

Proof. Same as for Proposition 5.7. □

Remark 5.7.5 In each case, the error term in Theorem 5.5.1 evaluates to

$$E(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) = \frac{1}{2} \left(\frac{2}{3^{1/4}} \right)^n \cdot \#\mathcal{K}_{B,n}^{1/4} \quad \text{for all } \mathbf{x} \in \mathcal{K}_{B,n}$$

which is dominated by the main term, $\sqrt{D(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})} \approx (\sqrt{4/3})^n \sqrt{\#\mathcal{K}_{B,n}}$ even for moderate B and n .

Looking at α for the $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$ of Propositions 5.7, 5.7.2, and 5.7.4, we have

$$\lim_{B \rightarrow \infty} \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x}) = \sqrt{4/3} \approx 1.154700538.$$

Table 5.1: Values of $\alpha_{\max}$ and $\bar{\alpha}$

B	$\mathcal{S}_{\text{Tame}} = \mathcal{S}_{\text{Wild}} = \mathcal{K}_{B,n}$ (Section 5.6)	Two-thirds (Section 5.7)	
	$\alpha_{\max}$	$\bar{\alpha}$	α (constant)
1	1.22474487	1.14982991	—
2	1.29099445	1.16361137	1.09544512
3	1.32287566	1.16749002	1.19522861
4	1.34164079	1.16909761	1.24721913
5	1.35400640	1.16991402	1.12815215
6	1.36277029	1.17038440	1.17669681
7	1.36930639	1.17067983	1.21106014
8	1.37436854	1.17087741	1.13759292
9	1.37840487	1.17101601	1.16979530
10	1.38169856	1.17111696	1.19522861
11	1.38443731	1.17119277	1.14208048
12	1.38675049	1.17125113	1.16619038
13	1.38873015	1.17129701	1.18634203
14	1.39044357	1.17133374	1.14470294
15	1.39194109	1.17136359	1.16397539
16	1.39326109	1.17138818	1.18065211
∞	$\sqrt{2}$ ≈ 1.41421356	$4 - 2\sqrt{2}$ ≈ 1.17157288	$\sqrt{4/3}$ $\approx \mathbf{1.15470054}$

(Since $\#\mathcal{X}_{\text{Int}}(\mathbf{x})$ is constant as $\mathbf{x}$ varies, so is $\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})$; and thus $\lim_{B \rightarrow \infty} \bar{\alpha} = \sqrt{4/3}$ too.) This is a clear improvement over the expected $4 - 2\sqrt{2}$ for the naïve shift spaces above when B is large.

Figure 5.2 gives us a good understanding of the importance of this optimization by Galbraith and Ruprai. Most especially for larger bounds B , $\bar{\alpha}$ is almost the same without a bad worst case compared to naïve shifts. However, in the rest of our work we make the choice to work with naïve shifts (section 5.6). We have two reasons for this. The first one is that we are particularly interested in very small $B = 1, 2$ or 3 . Two-thirds shift sets are not possible for $B = 1$, this prevents us from doing a proper analysis on CSIDH parameter sets which can have very low bounds and, in fact, $B = 1$ in CSIDH-1024. Furthermore, the analysis is easier to carry out for naïve shift sets. As such, for now, we restrict ourselves to not use two-thirds shift sets. However, two-thirds shift sets is a very valuable optimization that needs to be adapted to our algorithm, we leave it as future work.

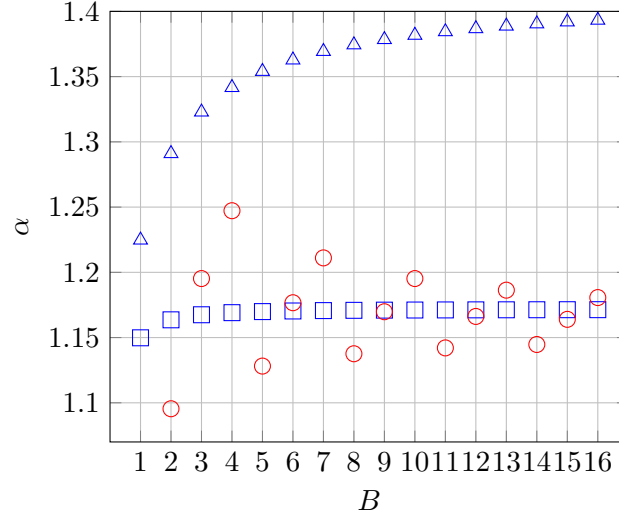


Figure 5.2: Values of α for increasing small B for the naive shift set ($\square$ for $\bar{\alpha}(B)$ and $\triangle$ for $\alpha_{\max}(B)$) and the “two-thirds” shift set ($\circ$ for $\alpha(B)$). As $B \rightarrow \infty$, for the naive shift set $\alpha_{\max}(B) \rightarrow \sqrt{2} \approx 1.4142$ and $\bar{\alpha}(B) \rightarrow 4 - 2\sqrt{2} \approx 1.1716$, while for the two-thirds shift set $\alpha(B) \rightarrow \sqrt{4/3} \approx 1.1547$.

5.8 Low-memory algorithms

The memory complexity for solving hidden shifts in $\mathcal{K}$ by random sampling is the same as the number of group operations. This is, on average, bigger than $\sqrt{\#\mathcal{K}}$ —but if we could accept square-root time and space, then we would just use baby-step giant-step. To reduce the memory cost, we will use pseudorandom walks and distinguished points, like Gaudry–Schost’s kangaroos (however our kangaroos hop differently).

5.8.1 Design

For simplicity, we use only naive shifts below (so $\#\text{Tame} = \#\text{Wild} = \#\mathcal{K}$); we leave the analysis of “two-thirds” shifts for later work.

Deterministic Pseudorandom Walks

The distinctive feature of our algorithm is that **kangaroos deterministically shapeshift between Tame and Wild states**. This addresses a fundamental limitation in previous approaches where distinct, unchanging kangaroo types implied different border conditions. Restarting is not a limiting factor any more. In our algorithm, all kangaroos follow the same deterministic pseudorandom walk mechanism and *randomly switch types*, eliminating the need to manage separate search spaces for Tame and Wild. However, later

on, when we will want to introduce small steps with elementary-shifts, we will deal with the separate search spaces for **Tame** and **Wild**. For now, kangaroos need not take small steps to stay in their respective spaces: every kangaroo deterministically restarts at each jump.

When kangaroos collide, they now continue on the same path without failure without probability of not detecting a collision. Let

$$h_{\text{Tame}} : (\mathbb{Z}^n \star P) \rightarrow \mathcal{S}_{\text{Tame}} \quad \text{resp.} \quad h_{\text{Wild}} : (\mathbb{Z}^n \star P) \rightarrow \mathcal{S}_{\text{Wild}}$$

be hash functions generating shifts for **Tame** resp. **Wild** moves, and let

$$h_s : (\mathbb{Z}^n \star P) \rightarrow \{0, 1\}$$

be a pseudorandom function selecting the kangaroo's type at each jump. This yields the deterministic pseudorandom transition function

$$\begin{aligned} \sigma : \mathcal{X}_{\text{Total}}(\mathbf{x}) &\longrightarrow \mathcal{X}_{\text{Total}}(\mathbf{x}) \\ X &\longmapsto \begin{cases} h_{\text{Tame}}(X) \star P & \text{if } h_s(X) = 0, \\ h_{\text{Wild}}(X) \star Q & \text{if } h_s(X) = 1. \end{cases} \end{aligned}$$

Ignoring distinguished points for now, each jump $X_{i-1} \mapsto X_i = \sigma(X_{i-1})$ proceeds by computing $s_i = h_s(X_{i-1})$ and $\mathbf{v}_i = h_{\text{Tame}}(X_{i-1})$ (resp. $h_{\text{Wild}}(X_{i-1})$) if $s_i = 0$ (resp. $s_i = 1$).

Distinguished Points

The main selling-point of parallel collision search algorithms is their low memory requirements, which are typically achieved using a set of *distinguished points* [59, 96]. Briefly: during a kangaroo walk, a triple $(X_i, \mathbf{v}_i, s_i)$ is only stored if X_i is a distinguished point in $\mathcal{X}_{\text{Total}}(\mathbf{x})$.

Let θ denote the probability that a random point in $\mathcal{X}_{\text{Total}}(\mathbf{x})$ is distinguished; the expected number of jumps before hitting a distinguished point is then $\approx 1/\theta$. Because walks are deterministic, kangaroos that collide continue on the same path, and the collision is detected when the same distinguished point is reached by two different walks. If a collision occurs, then it takes $\approx 1/\theta$ further jumps before it is detected. Generally, we must retrace the walks from their last distinguished point to pinpoint the exact collision.

In previous algorithms, kangaroo walks were pseudorandom in movement but *not* in type: **Tame** kangaroos never go wild, and **Wild** kangaroos are never tamed. This rigid

typing creates a problem in dimensions $n > 4$, colliding kangaroos may step outside of the search space before reaching a distinguished point, thus restarting and losing the collision. Our approach resolves this issue. All kangaroos use the same deterministic pseudorandom transition function, so the artificial distinction between Tame and Wild disappears: any kangaroo can replay the exact same walk as another, regardless of its current type. This guarantees that once a collision occurs, it will propagate along identical paths and be caught as soon as a distinguished point is reached. We will see that this change fundamentally improves the behaviour of kangaroo methods in high dimensions.

Cycles and cycle detection

Introducing distinguished points to gain a better space-time trade-off comes with a potential risk: the algorithm may fall into *cycling*. Consider a kangaroo trail K . Its last visited distinguished position was stored as $(X_d, \mathbf{v}_d, s_d)$. Now suppose $X_i = X_j$ for $i > j > d$: that is, K collides with its own path. This collision should be detected when K hits a new distinguished point, but there is the possibility that X_k is never distinguished for all $j \leq k \leq i$. Then, K ends up in an endless loop. We adopt the usual strategy in literature to avoid this: we force a random restart of a kangaroo if it has seen no distinguished points after $20/\theta$ jumps. The probability of a forced restart on a non-cycling kangaroo is then $(1 - \theta)^{20/\theta} \approx e^{-20}$.

We must be careful to set θ such that we do not always cycle in between distinguished points. For low-memory, we fix $\theta = c/\sqrt{\min(\#\mathcal{S}_{\text{Tame}}, \#\mathcal{S}_{\text{Wild}})}$. In our current method, we always have $\min(\#\mathcal{S}_{\text{Tame}}, \#\mathcal{S}_{\text{Wild}}) = \#\mathcal{S}_{\text{Wild}}$ so $\theta = c/\sqrt{\#\mathcal{S}_{\text{Wild}}}$.

5.8.2 Fake collisions

There are Tame-Tame, Wild-Wild, and Tame-Wild collisions, but only Tame-Wild collisions solve our problem. In the meantime, our algorithm must deal with many *fake collisions* (i.e., Tame-Tame or Wild-Wild collisions).

Collision recovery

In previous kangaroo methods, and in the “ideal” unlimited-memory algorithm of Section 5.5, we know if a collision is Tame-Tame or Wild-Wild or Tame-Wild as soon as we detect it (immediately) at a distinguished point, because kangaroos cannot change their type. This is no longer the case for our algorithm with distinguished points: if a collision happens, then we must retrieve its type to know if we have found a Tame-Wild collision.

Suppose we have detected a collision between kangaroos K_1 and K_2 . To retrieve the original collision, we apply the idea in van Oorschot–Wiener: rewind the two collided kangaroos to their previous distinguished points, stored in memory. For each kangaroo, at a distinguished point, we save the number of jumps n that have been done since the last encountered distinguished point. Suppose K_1 and K_2 have taken n_1 and n_2 jumps, respectively, before reaching the same distinguished point; without loss of generality, $n_1 > n_2$. We then “replay” K_1 for $n_1 - n_2$ jumps, then we replay both kangaroos step by step until we find the original collision. This requires an average of $1.5/\theta$ jumps [118].

For efficiency, in the case of a fake collision, after collision recovery we store the state of the kangaroo that collided with a previous path at the true collision point and not at the distinguished point.

Lost jumps

As in van Oorschot–Wiener PCS, we must take *lost jumps* into account. Whenever “fake” collisions occur, the added cost is not only due to collision recovery: after colliding, two kangaroos’ paths merge into one, so all of the computational effort of one of the two kangaroos can be considered lost. As the length of a trail between two distinguished points is $1/\theta$ on average, we consider each “fake” collision to cost $1/\theta$ lost jumps. Lost jumps have no impact on memory requirements, as nothing is stored during this process.

5.8.3 Algorithms layout

We keep a shared distinguished points table $H : X \mapsto (\mathbf{v}, s, j)$ where j is the number of jumps since the last visited distinguished point. Collisions are checked only at distinguished points. When two kangaroos meet at the same X , they deterministically follow the same subsequent path, so no detection is lost. Pseudo-code can be found in Algorithm 16 and 17.

5.8.4 Complexity statements

The number of jumps taken in our algorithm is higher than the number of ideal random samples in Section 5.5. First, we want the number of points visited by our algorithm to equal the number of samples in the ideal case.

Lemma 5.8.1 (Inverse Expected Occupancy) *Let $N_0 \in [0, M)$ be the desired expected number of non-empty bins after throwing N balls uniformly at random into M*

Algorithm 16 Worker (runs on each of the m parallel kangaroos)

Input: θ ; base P ; target Q

Output: FOUND($\mathbf{k}$) or CONTINUE

// State: $(X, \mathbf{v}, s) \in \mathcal{N} \times \mathbb{Z}^n \times \{\text{Tame}, \text{Wild}\}$

```

103 if Is_distinguished( $\theta, X$ ) then
104   if  $X \in H$  then
105      $(\mathbf{v}', s', j') \leftarrow H[X]$ 
106      $(\text{status}, \mathbf{k}, X_{\text{coll}}^-, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-) \leftarrow \text{CollisionRecover}((X, \mathbf{v}, s, j), (X, \mathbf{v}', s', j'))$ 
107     if  $\text{status} \neq \text{BAD}$  then
108       return FOUND( $\mathbf{k}$ )
109      $H[X_{\text{coll}}^-] \leftarrow (\mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-, j_{\text{coll}}^-)$ ; // Bad collision: save the state one jump
110     before the recovered collision
111 else
112    $H[X] \leftarrow (\mathbf{v}, s, j)$ 
113 if  $h_s(X) = 0$  then  $s \leftarrow \text{Tame}$ ,  $\mathbf{v} \leftarrow h_{\text{Tame}}(X)$ ,  $X \leftarrow \mathbf{v} \star P$ 
114 else  $s \leftarrow \text{Wild}$ ,  $\mathbf{v} \leftarrow h_{\text{Wild}}(X)$ ,  $X \leftarrow \mathbf{v} \star Q$ 
115 if Is_cycling( $\theta, X, \mathbf{v}, s$ ) then  $(X, \mathbf{v}, s) \leftarrow \text{RandomRestart}(\mathcal{N})$ 
116 return CONTINUE

```

Algorithm 17 CollisionRecover($(X, \mathbf{v}, s, j), (X, \mathbf{v}', s', j')$)

Input: Two DP records with identical footprint X

Output: $(\text{status}, \mathbf{k}, X_{\text{coll}}^-, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-)$

// Deterministically locate the *first* collision along the two walks and the caller's predecessor

```

114  $(X_{\text{coll}}^-, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-, \mathbf{v}'_{\text{coll}}, s'_{\text{coll}}, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-) \leftarrow \text{LocateFirstCollision}((X, \mathbf{v}, s, j), (X, \mathbf{v}', s', j'))$ 
115 if  $s_{\text{coll}} = s'_{\text{coll}}$  then
116   return (BAD,  $\perp$ ,  $X_{\text{coll}}^-, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-$ )
117 if  $s_{\text{coll}} = \text{Wild}$  then
118    $\mathbf{k} \leftarrow \mathbf{v}_{\text{coll}} - \mathbf{v}'_{\text{coll}}$ 
119 else
120    $\mathbf{k} \leftarrow \mathbf{v}'_{\text{coll}} - \mathbf{v}_{\text{coll}}$ 
121 return (GOOD,  $\mathbf{k}, X_{\text{coll}}^-, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-$ )

```

bins. The number of balls required to reach this occupancy is

$$N = -M \cdot \log \left(1 - \frac{N_0}{M} \right).$$

Moreover, if $N_0 = \gamma \cdot \sqrt{M}$ for some constant $\gamma > 0$, then for small $\frac{\gamma}{M}$, the number of balls required admits the asymptotic expansion:

$$N \approx \gamma \cdot \sqrt{M} + \frac{\gamma^2}{2}.$$

Proof. After throwing N balls uniformly at random into M bins, the expected number of empty bins is $M \cdot \left(1 - \frac{1}{M}\right)^N \approx M \cdot e^{-N/M}$ (see [78, Chapter 5]), so the expected number of non-empty bins is

$$N_0 = M - M \cdot e^{-N/M}.$$

Solving for N gives

$$N = -M \cdot \log \left(1 - \frac{N_0}{M} \right).$$

If $N_0 = \gamma \cdot \sqrt{M}$ for some constant $\gamma > 0$, then

$$N = -M \cdot \log \left(1 - \frac{\gamma}{\sqrt{M}} \right).$$

When M is large compared to γ , the Taylor expansion gives

$$N = -M \cdot \left(-\frac{\gamma}{\sqrt{M}} - \frac{\gamma^2}{2M} - O\left(\frac{1}{M}\right) \right) \approx \gamma \cdot \sqrt{M} + \frac{\gamma^2}{2},$$

as claimed. $\square$

Lemma 5.8.2 (Expected Number of Bins with Collisions) *If N balls are thrown uniformly at random into M bins, then the expected number of bins containing at least two balls is $\approx N - M \left(1 - e^{-N/M}\right)$. Moreover, if $N = \gamma \cdot \sqrt{M}$ for some constant $\gamma > 0$, then for small $\frac{\gamma}{M}$, the expected number of bins with collisions is $\approx \frac{\gamma^2}{2}$.*

Proof. Follows from Lemma 5.8.1. $\square$

Our algorithms fits in this framework where N is the total number of visited points, $M = \#\text{Tame}$ or $M = \#\text{Wild}$ and $N_0 = \frac{1}{2}\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n \sqrt{\pi \#\mathcal{K}}$ the ideal random number of samples in Tame (or Wild).

A large number of “fake” Tame-Tame or Wild-Wild collisions occur before we finally obtain a Tame-Wild collision. We want to investigate the expected number of fake collisions, to calculate the total amount of work spent on collision recovery and lost jumps.

Proposition 5.8.3 *The average number of Tame-Tame or Wild-Wild collisions is*

$$\frac{\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n} \pi}{4}.$$

Proof. We use Lemma 5.8.2 for $\gamma = \frac{\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n \sqrt{\pi}}{2}$. The number of total collisions is γ^2 as we count collisions in both Tame and Wild. $\square$

We can now estimate the overall complexity of our algorithm.

Theorem 5.8.4 *As $\#\mathcal{X}_{\text{int}}(\mathbf{x}) \rightarrow \infty$, if $\theta = c/\sqrt{\#\mathcal{K}}$, then the expected number of jumps (restarts) before the first Tame-Wild collision is*

$$\begin{aligned} & \left(\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n + \frac{5\sqrt{\pi}}{8c} \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n} \right) \sqrt{\pi \#\mathcal{K}} \\ & + \frac{\pi}{4} \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n} + O(E(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})). \end{aligned} \quad (5.7)$$

Proof. The expected number of jumps before the first Tame-Wild collision is the sum of

1. the expected number of jumps required to visit enough points in the ideal random sampling regime, and
2. the additional cost due to collision recovery and lost jumps.

The ideal number of jumps is approximately

$$\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n \cdot \sqrt{\pi \cdot \#\mathcal{K}},$$

with half of the samples expected to be Tame and half Wild. Apply Lemma 5.8.1 to both $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$, with M corresponding to each respective space and

$$N_0 = \frac{\sqrt{\pi}}{2} \cdot \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n \cdot \sqrt{M}.$$

The number of jumps required to visit enough distinct points is then

$$2\gamma\sqrt{\#\mathcal{K}} + \gamma^2 = \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n \cdot \sqrt{\pi \cdot \#\mathcal{K}} + \frac{\pi}{4} \cdot \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n}.$$

The overhead from collision recovery and lost jumps is an extra $\frac{5}{2\theta}$ times the number of collisions, which is

$$\frac{5\pi}{8c} \cdot \alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n} \cdot \sqrt{\#\mathcal{K}}$$

by Proposition 5.8.3 and the fact that $\theta = \frac{c}{\sqrt{\#\mathcal{K}}}$. Summing both contributions yields the claimed result. $\square$

Corollary 5.8.5 (Parallelisation) *Under the assumptions of Theorem 5.8.4, if m processors run m kangaroos in parallel, then the expected time until the first Tame-Wild collision is the result of Theorem 5.8.4 multiplied by $\frac{t}{m}$. Here t is the time required for one jump.*

Theorem 5.8.6 *As $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) \rightarrow \infty$, the expected number of memory units used before the first Tame-Wild collision is on average about*

$$c\sqrt{\pi}\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n + \frac{\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n}\pi}{4}.$$

Proof. We multiply the number of jumps needed to reach the number of points necessary in the ideal case by the probability $\theta = \frac{c}{\sqrt{\#\mathcal{S}_{\text{Wild}}}} = \frac{c}{\sqrt{\#\mathcal{S}_{\text{Tame}}}} = \frac{c}{\sqrt{\#\mathcal{K}}}$. Then, we add the number of fake collisions, each leading to an extra element in memory. No extra memory is required for the jumps taken during collision recovery. $\square$

5.9 An instance of van Oorschot–Wiener

Kangaroo-based algorithms fall under the general framework of van Oorschot and Wiener [118] for low-memory parallel collision search. However, the added bounds restrictions in the case of MDLP make important changes in the analysis. Tame and Wild kangaroos do not have the same search space and thus the same borders. This comes from the fact that the Wild search space is shifted by a point Q . The analysis from [118] needs to be adapted to take this difference into account. In our design for this chapter, we decided to simplify our algorithm: each jump is a restart. Our approach is to remove the possibility of stepping out of bounds. Making this assumption allows to more easily reconsider our work as an instance of van Oorschot–Wiener algorithm. We will calculate once more the complexity analysis of our algorithm by adapting the analysis done by van Oorschot–Wiener. This aims to convince our readers of the correctness of the analysis conveyed in the previous section 5.8.4 as in Chapter 6, we will have no choice but to use the same kind of reasoning as in 5.8.4.

In their work, van Oorschot and Wiener present two different regimes:

1. “Finding a small number of collisions”,
2. “Finding a large number of collisions”.

However, in the case of a large number of collisions, they actually assume that we are looking for only one “golden” collision. This is quite restrictive. As such, even though this

work should intuitively clearly be described as an instance of “Finding a large number of collisions”, our analysis is an adaptation of the “small number of collisions” case.

5.9.1 Finding a small number of collisions

In their work [118, section 4.1], van Oorschot and Wiener consider a pseudorandom function $f : S \rightarrow S$. We note $N = \#S$. They proceed to do a pseudorandom walk with f using distinguished points. Let q be the probability that a collision is useful (in our case that refers to Wild-Tame collisions). In [118], the probability of no useful collision being found after k jumps is

$$\left(1 - \frac{q}{N}\right) \left(1 - \frac{2q}{N}\right) \cdots \left(1 - \frac{(k-1)q}{N}\right) \approx e^{-\frac{k^2 q}{2N}}. \quad (5.8)$$

We consider that we have m different processors. The work to be done by each processor before finding a useful collision is $\sqrt{\frac{\pi N}{2q}}/m$ jumps. Once a collision has occurred, some work has to be done to locate that collision. The expected time for this process is $1.5/\theta$ iterations as described in the previous section. Besides, every time a collision occurs, an additional $1/\theta$ points are to be produced before it is detected. It is expected that $1/q$ fake collisions (Tame-Tame or Wild-Wild in our case) will occur before we find a useful one. In the analysis from [118], this added work of detecting and locating these collisions is overlooked because their number is small enough that all other processors go on working and it doesn't affect the complexity. However, in our case, their number is larger and thus it should be taken into account. In summary, the run-time stated in [118] to detect the first useful collision is

$$\left(\left(\sqrt{\frac{\pi N}{2q}} + \frac{2.5}{q\theta} \right) / m + \frac{2.5}{\theta} \right) t \quad (5.9)$$

where t is the time required for an iteration of f .

Remark 5.9.1 *In the case of a golden collision search, van Oorschot and Wiener suggest using several versions of the function f should be used. This is to avoid the case where the golden collision is stuck in a cycle. In our case, many collisions are useful, not just one. So this is not necessary.*

5.9.2 Expected runtime for our instance

We now apply this adapted analysis to our instance. Here, q is defined as $q = \#(\mathcal{S}_{\text{Wild}} \cap \mathcal{S}_{\text{Tame}}) / (\#\mathcal{S}_{\text{Tame}} + \#\mathcal{S}_{\text{Wild}} + \#(\mathcal{S}_{\text{Tame}} \cap \mathcal{S}_{\text{Wild}})) = 1/(2\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n} + 1)$.

Theorem 5.9.2 *As $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) \rightarrow \infty$, if $\theta = c/\sqrt{\#\mathcal{K}}$, then the expected number of jumps before the first Tame-Wild collision is*

$$\left(\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^n + \frac{5\alpha(\mathcal{S}_{\text{Tame}}, \mathcal{S}_{\text{Wild}}; \mathbf{x})^{2n}}{\sqrt{\pi}c} \right) \frac{\sqrt{\pi} \#\mathcal{K}}{m} + \frac{2.5\sqrt{\#\mathcal{K}}}{c}.$$

Proof. Plug the definition of q above into Equation (5.9). We say that $\theta = c/\sqrt{\#\mathcal{K}}$ and simplify. $\square$

We observe that the complexity given by analysis as an instance of the van Oorschot–Wiener PCS and the complexity given in Section 5.8.4 are very similar and mainly change in constant coefficients.

5.10 Experimental evaluation

We empirically evaluate the performance of our low-memory hidden shift algorithm across various group sizes $\mathcal{K}_{B,n} = [-B, B]^n$ with $c = 1$, focusing particularly on how the observed number of jumps relates to the theoretical complexity

$$\text{Jumps (Restarts)} \approx f(B, n)^n \sqrt{\pi \cdot \#\mathcal{K}_{B,n}} \quad \text{where} \quad f(B, n) := \left(\bar{\alpha}(B)^n + \frac{5\sqrt{\pi}}{8} \bar{\alpha}(B)^{2n} \right)^{1/n}$$

and theoretical space complexity

$$\text{Memory} \approx g(B, n)^n \quad \text{where} \quad g(B, n) := \left(\sqrt{\pi} \bar{\alpha}(B)^n + \frac{\pi}{4} \bar{\alpha}(B)^{2n} \right)^{1/n}.$$

(For values of $\bar{\alpha}(B)$ for increasing B , we refer the reader to Table 5.1.)

The real numbers of jumps and memory units in our experiments yield corresponding values $\hat{f}(B, n)$ and $\hat{g}(B, n)$ which we compare to $f(B, n)$ and $g(B, n)$.

Implementation details. Experiments were executed on a Slurm-managed cluster, on nodes with dual-socket **AMD EPYC 9754 128-Core** processors (2×128 physical cores, SMT2: 256 cores / 512 threads), **1007 GiB RAM**, under **AlmaLinux 9.4** (kernel 5.14.0-427.13.1.el9_4.x86_64).

All experiments were implemented in Python (Python 3.12.9) with **multiprocessing**. Each state x is labelled by

$$L(x) = \text{int}(\text{SHA3-256}(\text{UTF-8}(x))),$$

i.e., we hash the UTF-8 encoding of the exponent vector with SHA3-256. For every instance, the secret key is chosen at random. We chose to set the distinguished points

probability as

$$\theta = \frac{c}{\sqrt{\#\mathcal{K}_{B,n}}}, \quad c = 1.$$

A point is *distinguished* iff the lower 16 bits of $L(x)$ are at most $\lfloor \theta 2^{16} \rfloor$ (so the DP rate $\approx \theta$). The pseudorandom walk uses Python’s Mersenne–Twister (`random.Random`) seeded with $L(x)$: each coordinate of the jump vector is sampled independently and uniformly using `randrange(-B, B)`. This is how we implemented h_{Tame} and h_{Wild} . We then sample a branch bit with `rand.randint(0,1)` (seeded with $L(x)$), and take the branch “Wild” if $b = 1$ and “Tame” if $b = 0$. Each time we increment our jump counter.

5.10.1 Ternary Case: $B = 1$

The ternary case $B = 1$ corresponds to $\mathcal{K}_{B,n} = [-1, 1]^n$. This extreme case is of particular interest because of its special structure, and its application to CSIDH. Also, because of the ternary bounds, any restart is an elementary-shift which is perfect to look into the framework studied in this chapter. Tables 5.2 and 5.3 present our results. The time and space complexities both substantially exceed the theoretical estimates. However, the evolution of $f(1, n)$ and $g(1, n)$ shows that the data supports the hypothesis of modelling the complexity with an exponential factor f^n (resp. g^n) in front of $\sqrt{\pi \mathcal{K}_{1,n}}$ for large n .

n	B	θ	$\mathbb{E}[\text{jumps}]$	$\text{SD}[\text{jumps}]$	Theo. $\mathbb{E}[\text{jumps}]$	$\hat{f}$	f	$\hat{f}/f$
2	1	0.33	64	77	21	3.48	2.02	1.72
3	1	0.19	142	200	45	2.49	1.70	1.46
4	1	0.11	500	863	96	2.37	1.57	1.51
5	1	0.06	2 369	5 926	209	2.44	1.50	1.63
6	1	0.04	9 581	53 419	458	2.42	1.46	1.66
7	1	0.02	64 904	451 129	1 009	2.59	1.43	1.81
8	1	0.01	343 247	2 522 619	2 237	2.64	1.41	1.87
9	1	0.01	1 059 248	6 899 852	4 982	2.53	1.40	1.81
10	1	0.004	4 997 868	37 842 070	11 134	2.55	1.38	1.85

Table 5.2: Empirical and theoretical jump complexity (with sample standard deviation), values of f , and the ratio $\hat{f}/f$ over 1000 instances.

To give a better understanding of these results, we plot the time complexity on a log scale, as well as $\hat{f}(1, n)$ and $\hat{g}(1, n)$. Figure 5.3 displays the evolution of $\log \mathbb{E}[\text{jumps}]$ as a function of the dimension n , for fixed $B = 1$. On a logarithmic scale, exponential growth translates into linear behaviour, so the theoretical predictions $f^n \sqrt{\pi \mathcal{K}_{1,n}}$ should be straight lines. The trend (Figure 5.4) is consistent with the expectation of an exponential factor, especially for larger n . However, even though the linear behaviour in log scale is

n	B	θ	$\mathbb{E}[\text{memory}]$	$\text{SD}[\text{memory}]$	Theo. $\mathbb{E}[\text{memory}]$	$\hat{g}$	g	$\hat{g}/g$
2	1	0.333	4.22	2.52	3.72	2.05	1.93	1.06
3	1	0.192	9.24	5.63	4.51	2.10	1.65	1.27
4	1	0.111	11.98	8.92	5.50	1.86	1.53	1.22
5	1	0.0642	18.68	14.27	6.74	1.80	1.46	1.23
6	1	0.0370	26.56	22.52	8.30	1.73	1.43	1.21
7	1	0.0214	40.66	38.23	10.27	1.70	1.39	1.22
8	1	0.0123	56.96	55.02	12.77	1.66	1.37	1.21
9	1	0.00713	79.17	91.57	15.95	1.65	1.36	1.21
10	1	0.00412	105.64	135.38	20.02	1.59	1.35	1.18

Table 5.3: Empirical and theoretical memory complexity (with sample standard deviation), values of g , and the ratio $\hat{g}/g$ over 1000 instances.

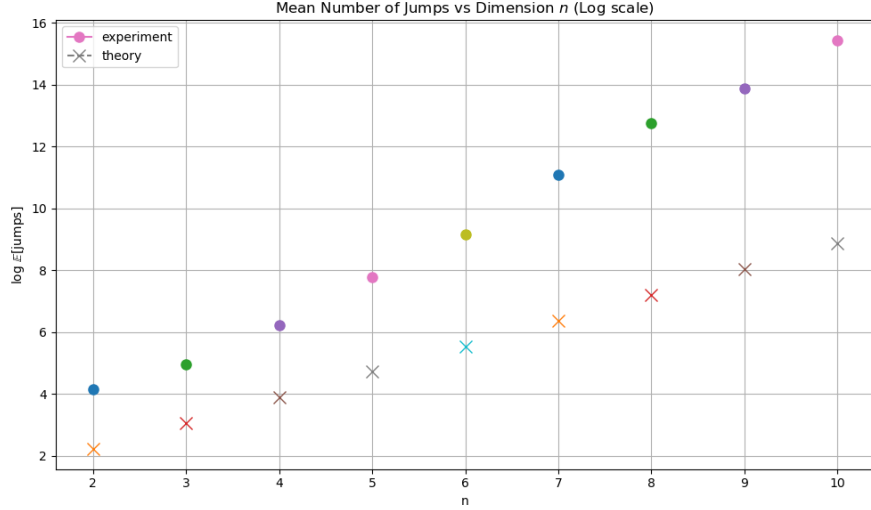


Figure 5.3: Logarithmic plot of the mean number of jumps over 1000 instances. Experimental values (dots) are compared to the theoretical prediction (crosses).

confirmed, the value for f is not the one predicted by our theory. The ratio $\hat{f}(1, n)/f(1, n)$ suggests we are missing a multiplicative factor which seems to stabilize from $n = 7$ around 1.84. As this factor scales exponentially, it is non-negligible.

Similarly, we examine the memory footprint, modelled by $\mathbb{E}[\text{memory}] \approx g(1, n)^n$. By taking the n -th root of the empirical memory values, we estimate $\hat{g}(1, n) = \mathbb{E}[\text{memory}]^{1/n}$, shown in Figure 5.5.

Here again, the experimental values converge to a constant. However, as for the number of jumps, this constant do not match with theory and is underestimated. Once again, $\hat{g}(1, n)/g(1, n)$ suggests that we are missing a constant factor. We will consider possible causes for the discrepancy between theory and practice in Section 5.10.3.

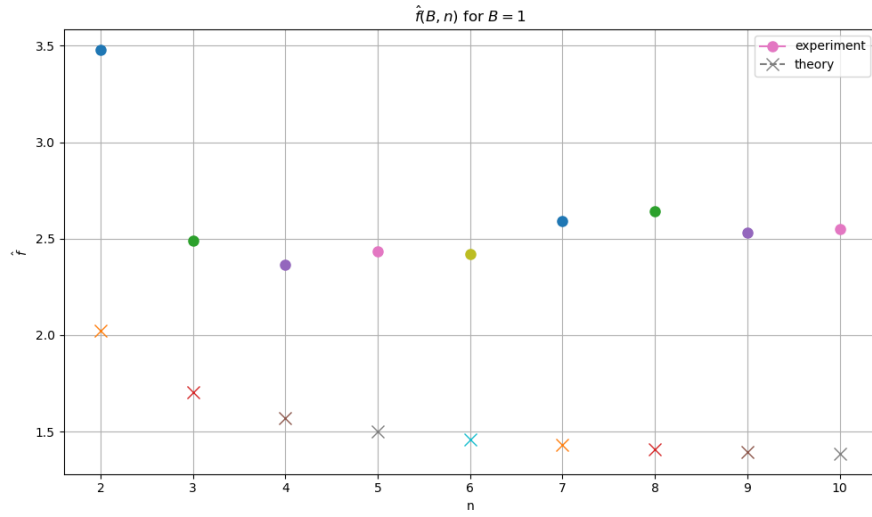


Figure 5.4: Empirical and theoretical values of $\hat{f}(1, n)$ over 1000 instances.

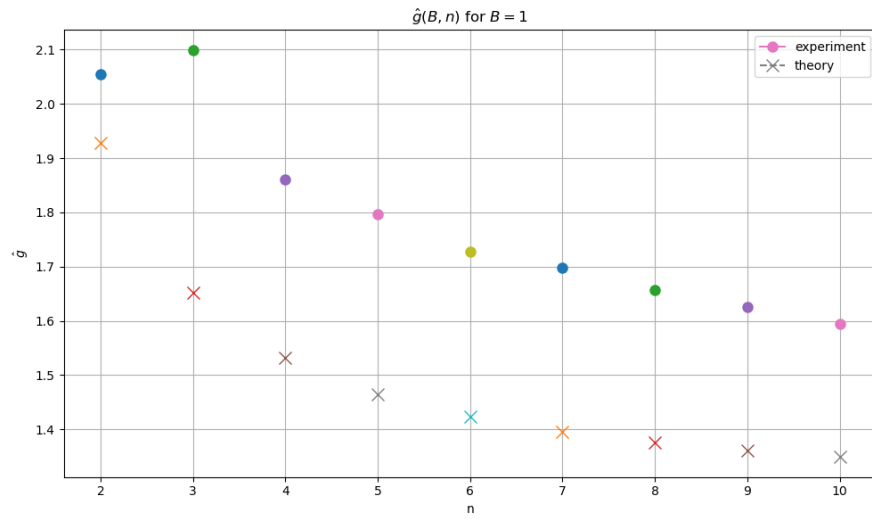


Figure 5.5: Empirical and theoretical values of $\hat{g}(1, n)$ over 1000 instances.

5.10.2 Breaking the dimension barrier for $n > 4$

We would like to stress the main result from both Chapters 5 and 6: breaking the dimension barrier of $n = 4$ for low-memory algorithm in the trend of Gaudry–Schost with asymptotic time complexity that scales as $\sqrt{\pi \mathcal{K}_{B,n}}$ up to a multiplicative exponential prefactor $f(B)^n$, whereas the memory complexity is purely exponential $g(B)^n$.

We do experiments for non-ternary keyspaces for $(B, n) = (4, 5); (4, 6); (5, 5)$. We would like in further work to provide more experiments results but for now we restrict ourselves to these sets as it takes a long time to gather over 1000 instances for higher B and n . Tables 5.4 and 5.5 present the results. For the interest of comparison, we added some values from $B = 1$. It gives us a better understanding of the impact of higher dimensions and different bounds given by $f(B, n)$.

n	B	θ	$\mathbb{E}[\text{jumps}]$	$\text{SD}[\text{jumps}]$	Theo. $\mathbb{E}[\text{jumps}]$	$\hat{f}$	f	$\hat{f}/f$
5	1	6.42×10^{-2}	2 369	5 926	209	3.48	1.50	1.63
5	4	4.12×10^{-3}	293 542	2 331 868	3 675	3.69	1.54	2.40
5	5	2.49×10^{-3}	364 825	3 633 747	6 106	3.48	1.54	2.26
5	6	1.64×10^{-3}	585 338	4 160 523	9 303	3.52	1.54	2.29
6	1	3.70×10^{-2}	9 581	53 419	458	2.42	1.46	1.66
6	4	1.37×10^{-3}	1 769 499	13 137 680	14 493	3.33	1.50	2.22
10	1	4.12×10^{-3}	4 997 868	37 842 070	11 134	2.55	1.38	1.85

Table 5.4: Empirical and theoretical results for jump complexity (with sample SD) and function f for various B, n over 1000 instances, with the ratio $\hat{f}/f$.

n	B	θ	$\mathbb{E}[\text{memory}]$	$\text{SD}[\text{memory}]$	Theo. $\mathbb{E}[\text{memory}]$	$\hat{g}$	g	$\hat{g}/g$
5	1	6.42×10^{-2}	18.68	14.27	6.74	1.80	1.46	1.23
5	4	4.12×10^{-3}	36.21	36.05	7.61	2.05	1.50	1.37
5	5	2.49×10^{-3}	42.32	43.71	7.66	2.12	1.50	1.41
5	6	1.64×10^{-3}	42.80	46.30	7.68	2.12	1.50	1.41
6	1	3.70×10^{-2}	26.56	22.52	8.30	1.73	1.43	1.21
6	4	1.37×10^{-3}	49.94	56.62	9.64	1.92	1.46	1.32
10	1	4.12×10^{-3}	105.64	135.38	20.02	1.59	1.35	1.18

Table 5.5: Empirical and theoretical results for memory complexity (with sample SD) and function g for various B, n over 1000 instances, with the ratio $\hat{g}/g$.

These results suggest once again the underestimation in $f(B, n)$ and $g(B, n)$. This impacts importantly the expected number of jumps and memory in higher dimensions as both $\hat{f}(B, n)/f(B, n)$ and $\hat{g}(B, n)/g(B, n)$ scale exponentially. However, we are able to confirm that the values for $f(B, n)$ and $g(B, n)$ should be converging to a constant

for a fixed n and high bounds B . This suggests that our complexities are at least in the right form even if the calculated constants need important revisions. The impact of the dimension n is clearly visible when comparing the parameter sets for $(n, B) = (10, 1)$ and $(5, 4)$. The cardinality of the search space is the same for both however the number of jumps and memory is much higher. When we compare the ratios here with the one for ternary bounds, it would appear we might be missing a B -dependant factor in f .

5.10.3 Possible sources of the constant gap in practice

Our experiments consistently show that the empirical factors $\hat{f}(B, n)$ and $\hat{g}(B, n)$ are larger than their theoretical counterparts $f(B, n)$ and $g(B, n)$. Below we discuss plausible causes for this. This discussion is unfortunately limited by the small number of experimental results. A proper analysis would require experiments in higher dimensions, with larger bounds, and also with many more random runs. The exponential complexity of our algorithm (and thus large computations) have been a limiting factor for us as well as some technical difficulties to run very long time jobs with our facilities.

(i) Small- B effects. One hypothesis is that very small bounds B distort the result. For a fixed n , the ratio $\hat{f}(B)/f(B)$ actually grows as B increases (see Table 5.4, $n = 5$), which argues that we might be missing a term depending on B . The analysis of $f(B, n)$ may be missing second-order terms that are non-negligible for the range of B we test. However, the ratio $\hat{g}(B)/g(B)$ does not seem too impacted, as such, we might want to investigate if we miss a term dependant on B in the calculations of the number of lost steps.

(ii) Experimental samples. We average over 1000 instances per setting. For small n , many trials inevitably hit the *same* few secret vector $\mathbf{x}$. As the pseudorandom function for walks is fixed across trials, repeated inputs can induce strong correlations between walk lengths, undermining the law of large numbers. Note that very small n should be solved by brute force; we retain them only to visualize trends. On the other hand, for large n , 1000 trials may undersample $\mathcal{K}_{B,n}$.

(iii) Sensitivity to the parameter θ . The theoretical constants assume $\theta = \frac{c}{\sqrt{\mathcal{K}_{B,n}}}$. In the experiments we chose to set $c = 1$. Maybe we have overlooked how sensitive to θ the algorithm is. Taking such a small θ may have led to some error with important collision recovery or lost steps. More importantly, we may be facing many more than expected infinite cycles which loops without ever meeting a distinguished point. This

could especially be the case for small θ like what we have chosen. It would be insightful to try out slightly larger choices for c . For example, if testing on ternary bounds, one could try $c = 6$ (maximum authorized value for $n = 2$).

(iv) High standard deviation. When looking at tables 5.2; 5.3; 5.4 and 5.5, we observe a very high standard deviation. It is usually bigger than the jump or memory counts themselves. This is due to the big difference between the best case and worst case of our algorithm. Recall from Section 5.6 that in the exponential prefactor, we have $\alpha_{\min} = 1$ and $\alpha_{\max} = \sqrt{\frac{3}{2}} \approx 1.225$ when $B = 1$. It could be that this phenomena is introducing noise which makes our analysis less accurate. This could be fixed for $B > 1$ by adapting our algorithm with the trick from Galbraith and Ruprai [56] using *two-thirds* shift sets (see Section 5.7). We won't do this in this thesis but it should be explored in future work.

(v) PRF, hashing. We have only tried one choice of PRF and hashing. The walks may have deviated from ideal pseudorandom behaviour; this could bias runtimes upward and help explain the larger exponential constant. Testing alternatives would mean more experimental work which implies large computations.

5.11 Comparisons with existing methods

In this section, we give an overview and comparison with other methods to attack the BHSP Problem.

5.11.1 Parallel Collision Search (PCS) without bounds

When looking at the general hidden shift problem, PCS appears as the state-of-the-art method. As such, the van Oorschot–Wiener method [118] is the fastest known low-memory algorithm to find a collision for a function $f : S \rightarrow S$. PCS solves BHSP instances using random walks over the whole space S being acted on without any consideration of bounds. Distinguished points allow to reduce memory requirements. Van Oorschot and Wiener use $\theta = c\sqrt{W/\#S}$ (where W represents the available memory). The time complexity of the algorithm follows the birthday bound and is in $O(\sqrt{\#S})$. The optimum θ varies with the cost of processors and memory.

In our case, the complexity of our algorithm is expected to be something of the form $\alpha^n \sqrt{\pi N}$. It can be seen as another form of the van Oorschot–Wiener method but which we have analysed in a very different setting. The crucial difference here is that N is the

size of the set $\mathcal{K} \star S$ where $\mathcal{K}$ is a subset of $\mathbb{Z}^n$ constrained under certain bounds. Here, we have added α^n due to the fact that not all jumps are performed in the intersection of Tame and Wild spaces. Depending on the context, N can be significantly smaller than $\#S$ and thus render our algorithm more efficient. For example, Cheon introduced the *auxiliary-input discrete logarithm problem* [25]: besides g and g^x in a group of order q , one is also given g^{x^m} for some $m \mid (q \pm 1)$. This auxiliary relation lets us search over smaller, bounded exponents rather than the whole group, so the effective search set is $N \approx q/m \ll q$.

5.11.2 REGA-oriented PCS

We now compare our algorithm with the PCS algorithm for REGAs from [26]. This algorithm solves BHSP instances where $\mathcal{K}_{B,n} = [-B, B]^n$ but using a different PCS instantiation. Their work focuses on ternary keyspaces ($B = 1$), it generalizes to bigger bounds but in practice the analysis has been conducted only for $B \leq 3$ and needs to be done for each B individually. The time complexity of their algorithm is expressed as $\#\mathcal{K}_{B,n}^r = (2B + 1)^{nr}$ for some constant r depending on the bound B and on the memory requirements. r reduces as the available memory increases (with a lower limit on r of 0.5). We can rewrite the asymptotic time complexity of our algorithm in the same way with memory $\bar{\alpha}(B)^{2n}$. Let's compare our results for small bounds $B = 1, 2, 3$. In table 5.6, we rewrite the complexities of both algorithm in the form $\#\mathcal{K}_{B,n}^r = (2B + 1)^{nr}$, give the values for r and make the comparison for the same memory requirements (which are set by our algorithm).

B	r		Memory	
	This work	[26]	This work	[26]
1	0.754	0.56	1.32^n	1.32^n
2	0.688	0.548	1.35^n	1.35^n
3	0.659	0.54	1.36^n	1.36^n

Table 5.6: Comparison of theoretical complexity (this work vs. [26]). We derived the complexity of [26] using their figure 5.b.

It seems that, for large n , [26] offer better results. Although we managed to break the $n = 4$ dimension barrier for Gaudry–Schost like algorithms, it seems we still suffer from high dimensions in the performance. In particular, for CSIDH with $B = 1$, it looks like [26] is a more promising approach. For larger B (e.g. $B \geq 4$), the analysis in [26] is speculative; it may be that our algorithm is more promising in that setting, but a proper

comparison will require more analysis of [26]. On the other hand, our algorithm analysis is realized for any bound B .

Now, a comparison between our work and the work from [26] points towards some common results w.r.t. the complexity form of low-memory algorithm on the BHSP in high dimensions. Both techniques find a result expressed as $O(\alpha^n \sqrt{\mathcal{K}_{B,n}})$. We conjecture that these sorts of low-memory bounded PCS algorithms have exponential factors like this due to the high-dimensional geometry of the keyspace, given that two different approaches have ended up with substantial exponential factors.

5.11.3 Meet-in-the-middle

Along with PCS, a meet-in-the-middle attack is believed to be one of the best classical attacks on group-action-based cryptosystems. However, it is argued in [7] that the van Oorschot–Wiener method [118] is a better approach when it comes down to large instances of isogeny-based group-action cryptosystems. The asymptotical time complexity of such an attack is also in $O(\sqrt{\#S})$. As such, the arguments are the same as for PCS, our attack allows to attack a space of potentially smaller size N . However note that we get a better time-memory trade-off by using distinguished points rather than meet-in-the-middle method.

5.12 Classical security of CSIDH

PCS on CSIDH (see Section 2.5.2) works in time $O(\sqrt{\#S}) = O(p^{1/4})$ where p is a prime defining the base field for supersingular curves (S). This set has size approximately $\#S = \sqrt{p}$. We want to compare $\sqrt{\#S}$ to $\alpha^n \sqrt{\pi \# \mathcal{K}_{B,n}}$ where $\mathcal{K}_{B,n}$ is the private keyspace, i.e., exponent vectors of length n and bound B . Note that p must be taken large enough to defend against subexponential attacks with Kuperberg’s algorithm (see Section 2.1.1). On the other hand, $\# \mathcal{K}_{B,n}$ only needs to be large enough to prevent square-root attacks, and so in practice $\#S$ may be much larger than $\# \mathcal{K}_{B,n}$.

Let B the bounds on the exponents. The CSIDH private key space $\mathcal{K}_{B,n}$ is chosen such that $\# \mathcal{K}_{B,n} \approx 2^{2\lambda}$ i.e., $(2B + 1)^n \approx 2^{2\lambda}$ where λ is the classical security parameter. This is to protect against classical attacks like ours using PCS on the restricted space. This condition is quite probably stricter than it needs to be. Our work provides a better understanding of the security assumptions for classical attacks on restricted group action which can lead to looser conditions.

Our algorithm is not going to be more efficient on average than classical PCS when the bounded search space spans the entire graph or has similar size. This might be the case for

$$\#S \leq \#\mathcal{K}_{B,n}. \quad (5.10)$$

However future CSIDH parameter sets probably would not be in this impractical regime (because p has to be larger to defend against subexponential quantum attacks). Some choices could be safe and faster in the case where

$$\#\mathcal{K}_{B,n} \ll \#S. \quad (5.11)$$

We want to investigate the condition given in (5.11) to get tighter classical security estimates and smaller bounds in CSIDH for the same security target and thus achieve a more efficient instantiation. Mostly, we argue on the choices of B and n . Note that condition (5.10) is the one given in the specification of CSIDH [20]. This condition is aimed at protecting the scheme against attacks on the BHSP. Our algorithm and the one from [26] have shown that there is also an exponential factor α^n in such PCS algorithms. The size of the keyspace $\mathcal{K}_{B,n}$ could be reduced to gain a substantial speedup in CSIDH (and more generally, isogeny-based group-action) performance.

We consider the two following CSIDH instantiations, they both aim at 128-bit security.

Example 5.12.1 *Let's first consider CSIDH-512. In this case, the parameters are set with a 512-bit prime p , $n = 74$ and $B = 5$. Those parameters have been chosen to satisfy equation (5.10) with minimal B .*

In this situation, PCS on the whole space runs with time complexity around 2^{128} group actions (square root of the class group size). Our algorithm runs on average with $(1.17^{74} + 1.37^{74} \frac{5\sqrt{\pi}}{8}) 11^{37} \sqrt{\pi} \approx 2^{163}$ actions. As expected, this work does not give any advantages in this case because condition (5.10) is met.

The bounds here could be lower without having a security breach by our algorithm. If we set $B = 3$ instead of $B = 5$, our algorithm runs in a time complexity of around 2^{139} which still respects 128-bit security which does not depend on B for classic PCS. However, the attack from [26] would run in around 2^{112} actions which is a breach in security. It seems that $B = 4$ might be the best pick here but the analysis in [26] does not cover $B \geq 4$ so we can only refer to the complexity of our work.

Instead of lowering the bounds, one could aim at a lower n taking advantage of the exponential factor in our algorithm and the one from [26]. If we chose $n = 60$ we get an attack using about 2^{237} actions by our algorithm ($B = 5$ is not studied in [26]). As such

it would seem that such smaller choice of n would lead to a speed-up in CSIDH. This can be easily achieved by setting the last 14 dimensions in the key vector to be always set to 0.

Example 5.12.2 We revisit the second parameter proposal for CSIDH-1024: a 1024-bit prime p , $n = 130$, and $B = 2$. In this setting, $(2B+1)^n = 5^{130} \approx 2^{302}$, so Condition (5.10) is not satisfied, whereas Condition (5.11) is. For such a large prime p , classical bounded-case attacks are probably acceptable, and thus enforcing Condition (5.10) was deemed unnecessary by the authors.

A parallel collision search (PCS) over the full class group costs about 2^{256} group actions, whereas our algorithm runs in $\left(1.16^{130} + 1.35^{130} \frac{5\sqrt{\pi}}{8}\right) 5^{65} \sqrt{\pi} \approx 2^{208}$ actions.

Our algorithm outperforms a classical PCS on the full class group however it does not break the 128-bit security. As such, even if the choice not to meet Condition (5.10) was already made in the design of CSIDH-1024, this could be even more efficient while being secure under PCS attacks on the bounded keyspace.

We first investigate $B = 1$. Our algorithm runs in $\left(1.15^{130} + 1.32^{130} \frac{5\sqrt{\pi}}{8}\right) 3^{65} \sqrt{\pi} \approx 2^{157}$ actions. As for the algorithm from [26], it runs in 2^{115} -actions. So, even though this choice of parameter is safe against our algorithm, it is still an invalid choice of parameters for 128-bit security.

Now, let's investigate smaller dimensions. Consider $n = 102$, in that case, [26] runs in 2^{129} actions and our algorithm in 2^{165} actions. Such a smaller choice of n should lead to a significant speed-up.

Remark 5.12.3 (CTIDH keyspaces) CTIDH [4] uses a completely different restricted private keyspace to define a large set of efficient constant-time group actions. Fix an integer $1 \leq b \leq n$, and choose pairs of positive integers $(C_1, T_1), \dots, (C_b, T_b)$ such that $C_1 + \dots + C_b = n$ and $\prod_{i=1}^b \binom{C_i + T_i - 1}{T_i} := N \approx 2^\lambda$. Let $S_i := \sum_{j=1}^i C_j$ for $1 \leq i \leq b$ (so in particular, $S_0 = 0$). Now C_i represents the size of the i -th “batch”, which comprises the elementary actions with indices $S_i + 1 \leq j \leq S_i + C_i$; and T_i is the total number of elementary actions from the i -th batch to be contributed to a key. More specifically,

$$\mathcal{K} = \left\{ (x_1, \dots, x_n) \in \mathbb{Z}^d \mid \sum_{j=1}^{C_i} |x_{S_i+j}| = T_i \text{ for all } 1 \leq i \leq b \right\}.$$

Defining optimal shift sets to attack this keyspace is an interesting problem for future work.

Chapter 6

Kangaroos and special points

Let's recall the setting used in the previous chapter. Let G be an abelian group and $\{B_i\}_1^n$ the bounds in the n -dimensional MDLP. *Tame kangaroos* jump in $\mathcal{S}_{\text{Tame}} \star P$ and *Wild kangaroos* jump in $\mathcal{S}_{\text{Wild}} \star Q$ for some secret Q with exponent vector $\mathbf{x}$ bounded by $\{B_i\}_1^n$. For simplicity, we consider the case where $\mathcal{S}_{\text{Tame}} = \mathcal{S}_{\text{Wild}} = \mathcal{K}_{B,n}$, the key space. Any points in the search space $\mathcal{X}_{\text{Total}}(\mathbf{x})$ has a probability $\theta = c/\sqrt{\#\mathcal{K}_{B,n}}$ to be a *distinguished point*. A footprint and exponent vector of a kangaroo is only saved into memory when it hits a distinguished point.

If we return to the ideas behind the 1- and 2-dimensional algorithms, then we could consider designing pseudorandom walks that move by a series of pseudorandomly-chosen but small, and easily computed, shifts.

When B is small, random elements of $\mathcal{K}_{B,n}$ tend to have many coordinates on the boundary, so random elementary shifts have a high probability of taking a walk out of $\mathcal{K}_{B,n} \star P$ or $\mathcal{K}_{B,n} \star Q$. Once we have left these boxes around P and Q , further random elementary shifts are unlikely to bring us back. For example, when $B = 1$, we expect a random $R = \mathbf{r} \star P$ in $\mathcal{K}_{B,n} \star P$ to have two-thirds of the coordinates of $\mathbf{r}$ equal to ± 1 . Thus, applying an elementary shift $\mathbf{e}$ to R moves R out of $\mathcal{K}_{B,n} \star P$ with probability $1/3$ (there is probability $2/3$ that the elementary shift is in a coordinate i where $r_i = \pm 1$, and then the shift moves that coordinate beyond ± 1 or back to 0 with probability $1/2$). A further random elementary shift $\mathbf{e}'$ only has a $1/(2n)$ chance of bringing us back into $\mathcal{K}_{B,n} \star P$ (this only happens when $\mathbf{e}' = -\mathbf{e}$). Of course, one might get lucky and walk out

of $\mathcal{K}_{B,n} \star P$ and into $\mathcal{K}_{B,n} \star Q$, but this is less likely—and then the next few elementary steps will probably take us out of $\mathcal{K}_{B,n} \star Q$.

When B is very small, then, or when n is large, there seems little to be gained by using pseudo-random walks of elementary shifts. In general, when B is small and n is large, it is better to simply restart at every single jump in the pseudo-random walk, as we have done in Chapter 5. Each of these restarts costs $O(B \cdot n)$ elementary shifts, but at least it ensures that the walk always stays within the desired search space.

When B is much larger, though, small shifts become interesting. We investigate this general framework from now on as it could bring some substantial improvements for large B . Besides, taking elementary steps is one of the original ideas behind kangaroos. In Chapter 5, our simplification turns the use of kangaroos just the same as any pseudo-random function. Kangaroos taking elementary shifts should be more efficient and practical pseudorandom functions.

This chapter analyses how one could extend our algorithm so that it does not restart at every jump. We aim at a practical improvement to reduce restarts and have more steps. We will describe the new design of our algorithm and its theoretical complexity analysis. This chapter does not include experimental results due to lack of time however the main contribution here is its design and analysis.

We introduce the concept of special points. Recall that this idea is mentioned in the appendix of the work from Galbraith and Ruprai [56] coming from a private communication with Pollard [56, Appendix A]. Whenever a kangaroo lands on a special point, it will restart. A point has probability β to be special. In the previous chapter we considered every point to be special hence restarting at every jump.

Note that when hitting a special point, if it is not a distinguished point as well then we do not save any information of the current state of the kangaroo. From Chapter 5, we have a new way of deterministically restarting pseudorandomly as either a Wild or Tame kangaroo. In this way, a collision can still be detected after a restart.

6.1 An adequate new search space

When a kangaroo steps out of boundaries, it restarts. However, it is important to note that this kind of restart is made in the same way as when hitting a special point and yet is quite different. The difference relies in the fact that boundaries for a Wild and a Tame kangaroo are *not* the same (except when $\mathbf{x} = \mathbf{0}$). As such, even if a Tame and Wild kangaroo are following the same path after a collision, if this path goes over either the Tame or Wild kangaroo boundaries then only one of the two kangaroos restarts.

This issue leads to lost collisions in a similar way as previous works which is exactly what causes dimensional limitations. However, compared to previous algorithms, if a kangaroo is getting close to the borders but hits a special point before, it is sent back pseudorandomly in the space. This gives us the intuition that a cleverly chosen β is crucial for solving this issue.

Our solution to this problem is to remove completely any borders. This adds a reasonable assumption: We suppose that it is possible to calculate Tame and Wild kangaroos a little outside of their original jumping spaces.

One could think that this idea is opposite to the main interest of this work i.e., using the restricted key space to recover Q rather than doing a parallel collision search over the whole group. But even if we allow kangaroos to step outside of $\mathcal{X}_{\text{Total}}$, we will always restart kangaroos in their original jumping spaces using $\mathcal{S}_{\text{Tame}}$ and $\mathcal{S}_{\text{Wild}}$ *often enough*.

6.1.1 New bounds and search space

Even if in practice we remove any bounds and the stepping zone of kangaroos is the whole group, we still want a framework to analyse our results. The answer to this problem is already given by special points. In reality, whenever a kangaroo cross borders, it will take on average $1/\beta$ steps before it hits a special point. When this happens, the kangaroo is sent back in the original jump spaces. So one can see that even if there are no harsh bounds set, we have on average some imaginary bounds that kangaroos rarely cross.

Let's assume we have a fixed β . With this in mind, we look for the smallest bounds of a new search space for which the overall analysis represents what happens in practice. We want to adjust the boundaries of our new search space so that when a kangaroo steps out of boundaries, it had taken about $1/\beta$ steps on average and should land on a special point. Recall that the expected number of steps until a one-dimension simple walk reaches $-a$ or a is a^2 . As such, it takes about $1/\beta$ steps on average to reach $-1/\sqrt{\beta}$ or $1/\sqrt{\beta}$. We set new shift spaces with bounds extended by $\frac{1-\sqrt{\beta}}{\sqrt{\beta}}$ (we do not count the last step on a special point as it is a restart).

$$\mathcal{S}'_{\text{Tame}} = \mathcal{S}'_{\text{Wild}} = \left[-B - \frac{1-\sqrt{\beta}}{\sqrt{\beta}}; B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} \right]_{i=1}^n \cap \mathbb{Z}^n.$$

Recall that $\#\mathcal{K}_{B,n} = (2B+1)^n$, we define $\#\mathcal{S}'_{\text{Tame}} = \#\mathcal{S}'_{\text{Wild}} = (2(B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}}) + 1)^n$.

The remaining question is how to choose β to begin with. It should be a trade-off between having a smaller search space as β gets higher and the cost of restarting. Stepping in the space correspond to small operations as we only perform elementary

shifts. However, when restarting we calculate a point in G corresponding to any integer tuple in the key space $\mathcal{K}_{B,n}$. A restart is usually much more costly than a normal step. In this regard, it is better to have β being quite small. Thus, one should take into account this trade-off depending on the situation to fix β . For example, if we wish to reduce the search space as much as possible, an intuitive choice for β would be $\beta = 1$. However, in that case, a kangaroo would restart at every jump (it is what we studied in the previous chapter).

We decide to add a constraint on β . If β is set too small, a kangaroo can step very far from the original jumping zones. We would like this to remain somewhat *reasonable*, as such we set

$$\frac{1 - \sqrt{\beta}}{\sqrt{\beta}} \leq B \quad \text{i.e.,} \quad \beta \geq \frac{1}{(B+1)^2}$$

6.2 Parallel algorithm layout

A kangaroo state is $(X^{(i)}, \mathbf{v}^{(i)}, s^{(i)}) \in \mathcal{N} \times \mathbb{Z}^n \times \{\text{Tame}, \text{Wild}\}$. We distinguish two cases at step i based on $X^{(i-1)}$.

(S) *Special jump (restart)*. If $X^{(i-1)}$ is a special point, we reseed deterministically from $X^{(i-1)}$ into $\mathcal{X}_{\text{Total}}(\mathbf{x})$ in using three hashes. This is the same as what was done in Chapter 5.

$$\begin{aligned} s^{(i)} &\leftarrow (h_s(X^{(i-1)}) = 0 ? \text{Tame} : \text{Wild}), \\ \mathbf{v}^{(i)} &\leftarrow \begin{cases} h_{\text{Tame}}(X^{(i-1)}) & \text{if } s^{(i)} = 0, \\ h_{\text{Wild}}(X^{(i-1)}) & \text{if } s^{(i)} = 1 \end{cases}, \\ X^{(i)} &\leftarrow \begin{cases} \mathbf{v}^{(i)} \star P & \text{if } s^{(i)} = 0, \\ \mathbf{v}^{(i)} \star Q & \text{if } s^{(i)} = 1. \end{cases} \end{aligned}$$

(N) *Normal jump (step)*. If $X^{(i-1)}$ is not special, the kangaroo kind stays fixed and we take a small step given by a elementary shift hash $h_{\pm}: (\mathbb{Z}^n \star P) \rightarrow \{-1, 0, 1\}^n$:

$$\begin{aligned} s^{(i)} &\leftarrow s^{(i-1)}, \\ \delta^{(i)} &\leftarrow h_{\pm}(X^{(i-1)}), \\ \mathbf{v}^{(i)} &\leftarrow \mathbf{v}^{(i-1)} + \delta^{(i)}, \\ X^{(i)} &\leftarrow X^{(i-1)} + \delta^{(i)} \star P. \end{aligned}$$

Thus h_s is *only* consulted at special points to (re)choose the kind; during normal steps, s is unchanged and the walk advances with $\pm 1/0$ increments.

We keep a shared distinguished points table $H : X \mapsto (\mathbf{v}, s, j)$ where j is the number of jumps since the last visited distinguished point. Collisions are checked only at distinguished points. When two kangaroos meet at the same X , they deterministically follow the same subsequent path, so no detection is lost. Collision recovery uses Algorithm 17.

Algorithm 18 Worker (runs on each of the m parallel kangaroos)

Input: θ, β ; base P ; target Q

Output: FOUND($\mathbf{k}$) or CONTINUE

// State: $(X, \mathbf{v}, s) \in \mathcal{N} \times \mathbb{Z}^n \times \{\text{Tame}, \text{Wild}\}$

```

122 if Is_distinguished( $\theta, X$ ) then
123   if  $X \in H$  then
124      $(\mathbf{v}', s', j') \leftarrow H[X]$ 
125      $(\text{status}, \mathbf{k}, X_{\text{coll}}^-, \mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-) \leftarrow \text{CollisionRecover}((X, \mathbf{v}, s, j), (X, \mathbf{v}', s', j'))$ 
126     if  $\text{status} \neq \text{BAD}$  then
127       return FOUND( $\mathbf{k}$ )
128      $H[X_{\text{coll}}^-] \leftarrow (\mathbf{v}_{\text{coll}}^-, s_{\text{coll}}^-, j_{\text{coll}}^-)$ ; // Bad collision: save the state one jump
129     before the recovered collision
130   else
131      $H[X] \leftarrow (\mathbf{v}, s, j)$ 
132 if Is_special( $\beta, X$ ) then
133   if  $h_s(X) = 0$  then  $s \leftarrow \text{Tame}$ ,  $\mathbf{v} \leftarrow h_{\text{Tame}}(X)$ ,  $X \leftarrow \mathbf{v} \star P$ 
134   else  $s \leftarrow \text{Wild}$ ,  $\mathbf{v} \leftarrow h_{\text{Wild}}(X)$ ,  $X \leftarrow \mathbf{v} \star Q$ 
135   return CONTINUE
136  $\delta \leftarrow h_{\pm}(X)$ ,  $\mathbf{v} \leftarrow \mathbf{v} + \delta$ ,  $X \leftarrow X + \delta \star P$ 
137 if Is_cycling( $\theta, X, \mathbf{v}, s$ ) then  $(X, \mathbf{v}, s) \leftarrow \text{RandomRestart}(\mathcal{N})$ 
138 return CONTINUE

```

6.3 Theoretical results

We are searching for some $\mathbf{x}$ in the input keyspace

$$\mathcal{K}_{B,n} := [-B, B]^n$$

where B is a positive integer.

6.3.1 The birthday bound

First we need to derive the birthday bound in this new framework with special points. A lot of this analysis has been done in Chapter 5 Section 5.5 and so, in this chapter, we

only need to recalculate α . We will only consider naive shift sets. We adapt the results in section 5.6 to our new framework. Keep in mind that we have a difference between the restart spaces and jumping spaces.

We have,

$$\begin{aligned}\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x}) &:= \left(\frac{\#\mathcal{S}'_{\text{Tame}} \cdot \#\mathcal{S}'_{\text{Wild}}}{\#\mathcal{K}_{B,n} \cdot \#\mathcal{X}_{\text{Int}}(\mathbf{x})'} \right)^{1/(2n)} \\ &= \prod_{i=1}^n \left(\frac{2 \left(B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} \right) + 1}{\sqrt{2B+1} \sqrt{2 \left(B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} \right) + 1 - |x_i|}} \right)^{1/n}.\end{aligned}$$

We begin with some bounds. Recall that $\frac{1-\sqrt{\beta}}{\sqrt{\beta}} \leq B$. Note that the minimum is for $\alpha(\mathcal{K}_{B',n}, \mathcal{K}_{B',n}; \mathbf{0}) = \sqrt{1 + \frac{2(1-\sqrt{\beta})}{\sqrt{\beta}(2B+1)}}$ (and $\mathbf{0}$ is the only such vector) while $\alpha_{\max}(\mathcal{K}_{B',n}, \mathcal{K}_{B',n}) = \alpha(\mathcal{K}_{B',n}, \mathcal{K}_{B',n}; (\pm B, \dots, \pm B))$. This just means that when the secret vector $\mathbf{x}$ is at the origin of $\mathcal{K}_{B,n}$, we have a minimum and the maximum is when $\mathbf{x}$ is at the edge of $\mathcal{K}_{B,n}$. This is due to the different size of the overlap. The upper bound tends to $2\sqrt{2}$ for $B \rightarrow \infty$, but is smaller for small B .

$$\begin{aligned}\sqrt{1 + \frac{2(1-\sqrt{\beta})}{\sqrt{\beta}(2B+1)}} &\leq \alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x}) \leq \alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}) \\ &\leq \frac{4B+1}{\sqrt{(2B+1)(B+1)}} \\ &\leq \frac{4(B+\frac{1}{2})-1}{\sqrt{2}(B+\frac{1}{2})}.\end{aligned}$$

Proposition 6.3.1 *With the above notation:*

$$\bar{\alpha}(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}) = \frac{2 \left(B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} \right) + 1}{(2B+1)^{3/2}} \left[\frac{1}{\sqrt{2 \left(B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} \right) + 1}} + \frac{2}{\sqrt{2B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}}}} \right. \\ \left. + \dots + \frac{2}{\sqrt{B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} + 1}} \right].$$

Proof. The proof follows the same logic than for Proposition 5.6.1. □

Remark 6.3.2 Note that when $\beta = 1$, the formula in Proposition 6.3.1 is the same as in Proposition 5.6.1 in Chapter 5 where our kangaroos were always restarting (which corresponds to $\beta = 1$ in this chapter).

We can also give some easy upper and lower bounds on $\bar{\alpha}(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}})$. First, note that the function $\beta \mapsto \bar{\alpha}(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}})$ is strictly decreasing on $\mathbb{Z}_{>0}$. As such, we have the same lower bound as in Chapter 5 which corresponds to $\beta = 1$. We give again the lower bounds (5.3) and (5.4).

$$\begin{aligned}\bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) &\geq \bar{\alpha}(\mathcal{K}_{1,n}, \mathcal{K}_{1,n})_{\beta=1} = \frac{1}{3} + \sqrt{\frac{2}{3}} \approx 1.14983, \\ \bar{\alpha}(\mathcal{K}_{B,n}, \mathcal{K}_{B,n}) &\geq 4 - 2\sqrt{2} \sqrt{1 + \frac{1}{2B+1}} + \frac{1}{2B+1}.\end{aligned}$$

Since $\beta \geq \frac{1}{(B+1)^2}$ and $\bar{\alpha}$ decreases in β , we have the upper bound

$$\bar{\alpha}(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}) \leq \frac{4B+1}{(2B+1)^{3/2}} \left[\frac{1}{\sqrt{4B+1}} + \frac{2}{\sqrt{3B}} + \cdots + \frac{2}{\sqrt{2B+1}} \right].$$

This upper bound in B increases on $\mathbb{Z}_{>0}$. It is less than and asymptotically tends to $4(\sqrt{6} - 2) \approx 1.798$ when $B \rightarrow \infty$.

6.3.2 Complexity statement

Our reasoning to derive complexity formulas follows the same logic as in Chapter 5, section 5.8.4. We introduce the ratio between the size of the key space and the size of the jump space,

$$\begin{aligned}\rho(B, \beta) &:= \left(\frac{\#\mathcal{K}_{B,n}}{\#\mathcal{S}'_{\text{Tame}}} \right)^{1/n} = \left(\frac{\#\mathcal{K}_{B,n}}{\#\mathcal{S}'_{\text{Wild}}} \right)^{1/n} \approx \frac{2B+1}{2 \left(B + \frac{1-\sqrt{\beta}}{\sqrt{\beta}} \right) + 1} \\ &= 1 - \frac{2(1-\sqrt{\beta})}{(2B-1)\sqrt{\beta} + 2} \leq 1.\end{aligned}$$

Proposition 6.3.3 Without considering lost jumps and collision recovery effort, as $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) \rightarrow \infty$, the number of jumps required to visit enough distinct points is

$$\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^n \cdot \sqrt{\pi \cdot \#\mathcal{K}_{B,n}} + \frac{\pi}{4} \cdot \alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^{2n} \rho(B, \beta)^n.$$

Proof. We apply lemma 5.8.1 with $\gamma = \frac{\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^n}{2} \sqrt{\pi \rho(B, \beta)^n}$, $M = \#\mathcal{S}'_{\text{Tame}} = \#\mathcal{S}'_{\text{Wild}}$ and $N_0 = \frac{\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^n}{2} \sqrt{\pi \#\mathcal{K}_{B,n}}$. $\square$

Table 6.1: Values of $\alpha_{\max}(B, \beta)$ and $\bar{\alpha}(B, \beta)$ for several $\beta \in (1/(B+1)^2, 1)$.

B	$\beta = 0.3$		$\beta = 0.5$		$\beta = 0.8$	
	$\alpha_{\max}$	$\bar{\alpha}$	$\alpha_{\max}$	$\bar{\alpha}$	$\alpha_{\max}$	$\bar{\alpha}$
2	1.3792	1.3806	1.3322	1.2764	1.3017	1.1967
3	1.3755	1.3288	1.3466	1.2506	1.3288	1.1916
4	1.3767	1.2974	1.3570	1.2348	1.3454	1.1881
5	1.3790	1.2764	1.3648	1.2243	1.3566	1.1856
6	1.3815	1.2614	1.3708	1.2167	1.3647	1.1837
7	1.3839	1.2501	1.3755	1.2110	1.3708	1.1823
8	1.3861	1.2414	1.3793	1.2066	1.3755	1.1812
9	1.3880	1.2344	1.3824	1.2031	1.3793	1.1802
10	1.3897	1.2286	1.3850	1.2002	1.3825	1.1795
11	1.3912	1.2239	1.3872	1.1978	1.3851	1.1788
12	1.3925	1.2198	1.3891	1.1958	1.3873	1.1783
13	1.3938	1.2164	1.3908	1.1941	1.3892	1.1778
14	1.3948	1.2134	1.3922	1.1926	1.3909	1.1774
15	1.3958	1.2108	1.3935	1.1913	1.3923	1.1771
16	1.3967	1.2085	1.3947	1.1901	1.3936	1.1767
∞	$\sqrt{2}$ (≈ 1.4142)	$4 - 2\sqrt{2}$ (≈ 1.1716)	$\sqrt{2}$ (≈ 1.4142)	$4 - 2\sqrt{2}$ (≈ 1.1716)	$\sqrt{2}$ (≈ 1.4142)	$4 - 2\sqrt{2}$ (≈ 1.1716)

Proposition 6.3.4 *The average number of Tame-Tame or Wild-Wild collisions is*

$$\frac{\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^{2n} \pi}{4} \rho(B, \beta)^n.$$

Proof. We apply lemma 5.8.2 with $\gamma = \frac{\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^n}{2} \sqrt{\pi \rho(B, \beta)^n}$ for both Tame-Tame and Wild-Wild collisions. $\square$

We can now derive our complexity statements.

Theorem 6.3.5 *As $\#\mathcal{X}_{\text{Int}}(\mathbf{x}) \rightarrow \infty$, if $\theta = c/\sqrt{\#\mathcal{K}_{B,n}}$, the expected number of jumps (steps + restarts) before the first Tame-Wild collision is*

$$\begin{aligned} \mathcal{C}(B, \beta; \mathbf{x}) := & \left(\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^n + \frac{5\sqrt{\pi}}{8c} \alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^{2n} \rho(B, \beta)^n \right) \sqrt{\pi \#\mathcal{K}_{B,n}} \\ & + \frac{\pi}{4} \alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^{2n} \rho(B, \beta)^n + O(E(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})). \end{aligned} \quad (6.1)$$

In particular, if one in $1/\beta$ jumps is a restart in expectation (i.e., a jump is a restart with probability β and a step otherwise), then the expected count decompose as:

$$\text{Restarts} = \beta \mathcal{C}(B, \beta; \mathbf{x}), \quad \text{Steps} = (1 - \beta) \mathcal{C}(B, \beta; \mathbf{x}).$$

Proof. We add the cost of collision recovery and lost jumps to the number of jumps required to visit enough distinct points as stated in Proposition 6.3.3. The cost in managing fake collisions and lost steps is $\frac{5}{2\theta}$ multiplied by the number of Tame-Tame and Wild-Wild collisions given by Proposition 6.3.4. $\square$

Corollary 6.3.6 (Parallelisation) *Under the assumptions of Theorem 6.3.5, if m processors run m kangaroos in parallel, then the expected time until the first Tame-Wild collision is the result of Theorem 6.3.5 multiplied by $\frac{\beta t_R + (1-\beta)t_S}{m}$. Here t_R is the time required for one restart and t_S is the time required for one step.*

Remark 6.3.7 *Note that in Chapter 5, we have $\beta = 1$. Thus, the time complexity is multiplied by $\frac{t_R}{m}$.*

Theorem 6.3.8 *As $\#\mathcal{X}_{\text{int}}(\mathbf{x}) \rightarrow \infty$, the expected number of memory units used before the first Tame-Wild collision is on average about*

$$c\sqrt{\pi}\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^n + \frac{\alpha(\mathcal{S}'_{\text{Tame}}, \mathcal{S}'_{\text{Wild}}; \mathbf{x})^{2n}\pi}{4}\rho(B, \beta)^n.$$

Proof. We multiply the number of jumps needed to reach the number of points necessary in the ideal case by the probability $\theta = \frac{c}{\sqrt{\#\mathcal{K}_{B,n}}}$. Then we add the number of collisions, each leading to one more memory unit being used. No extra memory is required for the jumps taken during collision recovery. $\square$

6.4 Discussion

We discuss some properties of the time and memory complexities of our algorithm.

6.4.1 Asymptotic regimes

Write $\bar{\alpha} = \bar{\alpha}(B, \beta)$ and $\rho = \rho(B, \beta)$ for brevity. For fixed $\beta \in (0, 1]$ and $B \rightarrow \infty$, one has $\bar{\alpha} \rightarrow 4 - 2\sqrt{2} =: a_0 \approx 1.1716$ and $\rho \rightarrow 1$, hence $\mathcal{C} \sim (a_0^n + \frac{5\sqrt{\pi}}{8c} a_0^{2n})\sqrt{\pi \#\mathcal{K}_{B,n}}$.

Under the constraint $\beta \geq 1/(B+1)^2$, consider the “edge case” $\beta = 1/(B+1)^2$. $\bar{\alpha}$ attains the envelope $U(B) = \frac{4B+1}{(2B+1)^{3/2}} \left(\frac{1}{\sqrt{4B+1}} + \frac{2}{\sqrt{3B}} + \cdots + \frac{2}{\sqrt{2B+1}} \right)$ with $\lim_{B \rightarrow \infty} U(B) = 4(\sqrt{6} - 2) =: a_1 \approx 1.798$ and $\rho = \frac{2B+1}{4B+1} \rightarrow \frac{1}{2}$; thus for $B \rightarrow \infty$, $\mathcal{C} \sim (a_1^n + \frac{5\sqrt{\pi}}{8c} a_1^{2n} 2^{-n})\sqrt{\pi \#\mathcal{K}_{B,n}}$.

6.4.2 Dimension n and the role of ρ

In the time complexity, the exponential factor depends on $\bar{\alpha}^n$ and $(\bar{\alpha}^2 \rho)^n$. Since $\bar{\alpha} > 1$ (typically $\bar{\alpha} \in [4 - 2\sqrt{2}, 4(\sqrt{6} - 2)] \approx [1.17, 1.80]$) and $\rho \in [\frac{2B+1}{4B+1}, 1] \subset [1/2, 1]$, both factors grow exponentially in n . In particular, even at the most favourable edge case for β , $\rho \approx \frac{1}{2}$ for very large B and $\bar{\alpha}^2 \rho \approx (1.80)^2 \cdot \frac{1}{2} \approx 1.62$ at the envelope. So the second term is at best mildly attenuated by ρ^n and often still increases with n . Thus, as for the different versions of the Gaudry–Schost algorithm, higher dimension markedly amplifies the constant in front of $\sqrt{\#\mathcal{K}_{B,n}}$. However, in the case of versions of the Gaudry–Schost algorithm, $n > 4$ is not possible and so this impact for higher dimensions is not noticeable.

6.4.3 Impact of β

Note that we have an illegal value in our analysis for β , that is $\beta = 0$. This would mean infinite borders for our search space, and so basically a PCS attack on the whole group. Choosing $\beta = 0$ defeats the point of our algorithm. However, this should never be the case since we have set the hypothesis $\beta > \frac{1}{(B+1)^2}$.

When looking at Theorem 6.3.5, in terms of jumps, the optimal choice for β is 1 as it minimizes $\bar{\alpha}$ and ρ . However, that is if one do not care about the bigger cost of a restart over a step. This is consistent with our results from Chapter 5 which investigates this exact subcase. However, the whole point of special points and this generalized version of the algorithm is to reduce the number of restarts.

6.4.4 Memory

The memory requirements here are the same as discussed in Chapter 5. The choice of θ has to be a trade-off between memory and the number of group operations. In our context, we would like to have a low-memory algorithm. The total number of memory units required in the algorithm is of about $t\theta$ where t is the number of jumps, recall that this is why we chose $\theta = \frac{c}{\sqrt{\#\mathcal{K}_{B,n}}}$ for some constant c . In the leading term of the time complexity, the only c -dependence appears as a factor proportional to $1/c$, i.e.,

$$\left(\bar{\alpha}^n + \frac{5\sqrt{\pi}}{8c} \bar{\alpha}^{2n} \rho^n\right) \sqrt{\pi \#K}.$$

Increasing c strictly decreases the leading constant. Subject to the regime constraints on $\theta = c/\sqrt{\#K}$ (c is a constant and $\theta \leq 1$), one should choose c as large as possible to minimize the overall time complexity. This seems reasonable as intuitively, more distinguished points should decrease the number of lost jumps and jumps needed for

collision recovery. Of course, as a downside, this would impact the memory complexity and make it worse.

6.4.5 Future work

We would like to have a proof-of-concept of the generalized kangaroo algorithm with special points. It would be important to confront our theoretical results with experimental ones as was done in Chapter 5. The parameter space here is large: on top of n and B , one can also vary β (and even θ). Since each configuration is an exponential-time computation, a thorough experimental evaluation would be computationally heavy.

To go further, we should consider the optimisation obtained by choosing two-thirds shifts in the manner of Galbraith and Ruprai [56]. This means rederiving complexity statements and more precisely the values of α .

Bibliography

- [1] M. Ajtai. Generating hard instances of lattice problems (extended abstract). STOC '96, New York, NY, USA, 1996. ACM. [doi:10.1145/237814.237838](https://doi.org/10.1145/237814.237838).
- [2] M. Ajtai. Generating hard instances of the short basis problem. In J. Wiedermann, P. van Emde Boas, and M. Nielsen, editors, *Automata, Languages and Programming*, pages 1–9, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [3] N. Alamati, L. De Feo, H. Montgomery, and S. Patranabis. Cryptographic group actions and applications. In S. Moriai and H. Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 411–439. Springer, 2020. [doi:10.1007/978-3-030-64834-3_14](https://doi.org/10.1007/978-3-030-64834-3_14).
- [4] G. Banegas, D. J. Bernstein, F. Campos, T. Chou, T. Lange, M. Meyer, B. Smith, and J. Sotáková. CTIDH: faster constant-time CSIDH. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):351–387, 2021. URL: <https://doi.org/10.46586/tches.v2021.i4.351-387>, [doi:10.46586/TCHES.V2021.I4.351-387](https://doi.org/10.46586/TCHES.V2021.I4.351-387).
- [5] G. Banegas, K. Carrier, A. Chailloux, A. Couvreur, T. Debris-Alazard, P. Gaborit, P. Karpman, J. Loyer, R. Niederhagen, N. Sendrier, B. Smith, and J.-P. Tillich. WAVE – submission to NIST post-quantum signatures project, 2023. <https://github.com/wavesign/wave>.
- [6] A. Basso, P. Dartois, L. D. Feo, A. Leroux, L. Maino, G. Pope, D. Robert, and B. Wesolowski. Sqisign2d–west. In K.-M. Chung and Y. Sasaki, editors, *Advances in*

Cryptology – ASIACRYPT 2024, pages 339–370, Singapore, 2025. Springer Nature Singapore.

- [7] E. Bellini, J. Chavez-Saab, J.-J. Chi-Domínguez, A. Esser, S. Ionica, L. Rivera-Zamarripa, F. Rodríguez-Henríquez, M. Trimoska, and F. Zweydinger. Parallel isogeny path finding with limited memory. In *Progress in Cryptology – INDOCRYPT 2022: 23rd International Conference on Cryptology in India, Kolkata, India, December 11–14, 2022, Proceedings*, pages 294–316, Berlin, Heidelberg, 2022. Springer-Verlag. doi:10.1007/978-3-031-22912-1_13.
- [8] D. J. Bernstein. RSA signatures and Rabin–Williams signatures: the state of the art. 2008. URL: <https://cr.yp.to/signs/rwsota-20080131.pdf>.
- [9] D. J. Bernstein, L. De Feo, A. Leroux, and B. Smith. Faster computation of isogenies of large prime degree. In S. D. Galbraith, editor, *Proceedings of the Fourteenth Algorithmic Number Theory Symposium*, pages 39–55. Mathematics Sciences Publishers, 2020. <https://eprint.iacr.org/2020/341>.
- [10] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe. The SPHINCS+ signature framework. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS ’19*, page 2129–2146. Association for Computing Machinery, 2019. doi:10.1145/3319535.3363229.
- [11] D. J. Bernstein, T. Lange, C. Martindale, and L. Panny. Quantum circuits for the CSIDH: optimizing quantum evaluation of isogenies. In Y. Ishai and V. Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part II*, volume 11477 of *Lecture Notes in Computer Science*, pages 409–441. Springer, 2019. doi:10.1007/978-3-030-17656-3_15.
- [12] D. J. Bernstein and J. P. Sorenson. Modular exponentiation via the explicit chinese remainder theorem. *Math. Comput.*, 76(257):443–454, 2007. doi:10.1090/S0025-5718-06-01849-7.
- [13] W. Beullens, T. Kleinjung, and F. Vercauteren. CSI-FiSh: Efficient isogeny based signatures through class group computations. In S. D. Galbraith and S. Moriai, editors, *Advances in Cryptology – ASIACRYPT 2019 – 25th International Conference on the Theory and Application of Cryptology and Information Security, Kobe, Japan, December 8-12, 2019, Proceedings, Part I*, volume

- 11921 of *Lecture Notes in Computer Science*, pages 227–247. Springer, 2019. doi:[10.1007/978-3-030-34578-5_9](https://doi.org/10.1007/978-3-030-34578-5_9).
- [14] J.-F. Biasse, M. J. Jacobson Jr., and A. K. Silvester. Security estimates for quadratic field based cryptosystems. In R. Steinfeld and P. Hawkes, editors, *Information Security and Privacy – ACISP 2010*, volume 6168 of *Lecture Notes in Computer Science*, pages 233–247. Springer, 2010. https://doi.org/10.1007/978-3-642-14081-5_15.
 - [15] X. Bonnetain and A. Schrottenloher. Quantum security analysis of CSIDH. In A. Canteaut and Y. Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 493–522. Springer, 2020. doi:[10.1007/978-3-030-45724-2_17](https://doi.org/10.1007/978-3-030-45724-2_17).
 - [16] J. W. Bos, T. Kleinjung, and D. Page. Efficient modular multiplication. Cryptology ePrint Archive, Paper 2021/1151, 2021. URL: <https://eprint.iacr.org/2021/1151>.
 - [17] C. Boschini, D. Fiore, E. Pagnin, L. Torresetti, and A. Visconti. Progressive and efficient verification for digital signatures: extensions and experimental results. *J. Cryptogr. Eng.*, 14(3):551–575, 2024. URL: <https://doi.org/10.1007/s13389-024-00358-0>, doi:[10.1007/S13389-024-00358-0](https://doi.org/10.1007/S13389-024-00358-0).
 - [18] J. Buchmann. A subexponential algorithm for the determination of class groups and regulators of algebraic number fields. In C. Goldstein, editor, *Séminaire de Théorie des Nombres (Paris), 1988–89*, volume 91 of *Progress in Mathematics*, pages 27–41. Birkhäuser Boston, 1990.
 - [19] W. Castryck and T. Decru. An efficient key recovery attack on SIDH. In C. Hazay and M. Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 423–447. Springer, 2023. doi:[10.1007/978-3-031-30589-4_15](https://doi.org/10.1007/978-3-031-30589-4_15).
 - [20] W. Castryck, T. Lange, C. Martindale, L. Panny, and J. Renes. CSIDH: an efficient post-quantum commutative group action. In T. Peyrin and S. D. Galbraith, editors,

- Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part III*, volume 11274 of *Lecture Notes in Computer Science*, pages 395–427. Springer, 2018. doi:[10.1007/978-3-030-03332-3_15](https://doi.org/10.1007/978-3-030-03332-3_15).
- [21] W. Castryck, L. Panny, and F. Vercauteren. Rational isogenies from irrational endomorphisms. In A. Canteaut and Y. Ishai, editors, *Advances in Cryptology - EUROCRYPT 2020 - 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10-14, 2020, Proceedings, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 523–548. Springer, 2020. doi:[10.1007/978-3-030-45724-2_18](https://doi.org/10.1007/978-3-030-45724-2_18).
 - [22] D. X. Charles, K. E. Lauter, and E. Z. Goren. Cryptographic hash functions from expander graphs. *Journal of Cryptology*, 22(1):93–113, Jan. 2009. doi:[10.1007/s00145-007-9002-x](https://doi.org/10.1007/s00145-007-9002-x).
 - [23] J. Chávez-Saab, J. Chi-Domínguez, S. Jaques, and F. Rodríguez-Henríquez. The SQALE of CSIDH: sublinear Vélu quantum-resistant isogeny action with low exponents. *J. Cryptogr. Eng.*, 12(3):349–368, 2022. doi:[10.1007/S13389-021-00271-W](https://doi.org/10.1007/S13389-021-00271-W).
 - [24] Y. Chen, N. Genise, and P. Mukherjee. Approximate trapdoors for lattices and smaller hash-and-sign signatures. In *Advances in Cryptology – ASIACRYPT 2019, Proceedings, Part III*, volume 11923 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2019. doi:[10.1007/978-3-030-34618-8_1](https://doi.org/10.1007/978-3-030-34618-8_1).
 - [25] J. H. Cheon. Security analysis of the strong diffie-hellman problem. In S. Vaudenay, editor, *Advances in Cryptology - EUROCRYPT 2006*, pages 1–11, Berlin, Heidelberg, 2006. Springer Berlin Heidelberg.
 - [26] J. Chi-Domínguez, A. Esser, S. Kunzweiler, and A. May. Low memory attacks on small key CSIDH. In M. Tibouchi and X. Wang, editors, *Applied Cryptography and Network Security - 21st International Conference, ACNS 2023, Kyoto, Japan, June 19-22, 2023, Proceedings, Part II*, volume 13906 of *Lecture Notes in Computer Science*, pages 276–304. Springer, 2023. doi:[10.1007/978-3-031-33491-7_11](https://doi.org/10.1007/978-3-031-33491-7_11).
 - [27] J. Chi-Domínguez, A. Esser, S. Kunzweiler, and A. May. Low memory attacks on small key CSIDH. In M. Tibouchi and X. Wang, editors, *Applied Cryptography*

- and Network Security – ACNS 2023, Proceedings, Part II, volume 13906 of *Lecture Notes in Computer Science*, pages 276–304, Cham, 2023. Springer. doi:10.1007/978-3-031-33491-7_11.
- [28] J. Chi-Domínguez and F. Rodríguez-Henríquez. Optimal strategies for CSIDH. *Adv. Math. Commun.*, 16(2):383–411, 2022. doi:10.3934/amc.2020116.
 - [29] K. Conrad. Separable extensions. <https://kconrad.math.uconn.edu/blurbs/galoistheory/separable1.pdf>, 2023. Expository notes, University of Connecticut. Accessed 2025-10-04.
 - [30] A. Costache, B. Feigon, K. Lauter, M. Massierer, and A. Puskás. Ramanujan graphs in cryptography. In J. S. Balakrishnan, A. Folsom, M. Lalin, and M. Manes, editors, *Research Directions in Number Theory*, pages 1–40, Cham, 2019. Springer International Publishing.
 - [31] C. Costello and H. Hisil. A simple and compact algorithm for SIDH with arbitrary degree isogenies. In T. Takagi and T. Peyrin, editors, *Advances in Cryptology – ASIACRYPT 2017 – 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part II*, volume 10625 of *Lecture Notes in Computer Science*, pages 303–329. Springer, 2017. doi:10.1007/978-3-319-70697-9_11.
 - [32] C. Costello and B. Smith. Montgomery curves and their arithmetic. *Journal of Cryptographic Engineering*, Mar 2017. doi:10.1007/s13389-017-0157-6.
 - [33] J.-M. Couveignes. Hard homogeneous spaces. Cryptology ePrint Archive, Paper 2006/291, 2006. <https://eprint.iacr.org/2006/291>. URL: <https://eprint.iacr.org/2006/291>.
 - [34] J.-M. Couveignes and F. Morain. Schoof’s algorithm and isogeny cycles. In L. M. Adleman and M.-D. Huang, editors, *Algorithmic Number Theory*, pages 43–58, Berlin, Heidelberg, 1994. Springer Berlin Heidelberg.
 - [35] D. Cozzo and N. P. Smart. Sashimi: Cutting up csi-fish secret keys to produce an actively secure distributed signing protocol. In J. Ding and J.-P. Tillich, editors, *Post-Quantum Cryptography – PQCrypto 2020*, volume 12100 of *Lecture Notes in Computer Science*, pages 169–186, Cham, 2020. Springer. doi:10.1007/978-3-030-44223-1_10.

- [36] C. Cremers, S. Düzl , R. Fiedler, M. Fischlin, and C. Janson. Buffing signature schemes beyond unforgeability and the case of post-quantum signatures. In *IEEE Symposium on Security and Privacy*, pages 1696–1714, 2021.
- [37] J. Daemen and V. Rijmen. AES proposal: Rijndael. Technical report, National Institute of Standards and Technology, 1999. URL: <https://csrc.nist.gov/archive/aes/rijndael/Rijndael-ammended.pdf>.
- [38] P. Dartois, J. K. Eriksen, T. B. Fouotsa, A. H. L. Merdy, R. Invernizzi, D. Robert, R. Rueger, F. Vercauteren, and B. Wesolowski. PEGASIS: Practical effective class group action using 4-dimensional isogenies. Cryptology ePrint Archive, Paper 2025/401, 2025. URL: <https://eprint.iacr.org/2025/401>.
- [39] P. Dartois, A. Leroux, D. Robert, and B. Wesolowski. Sqisignhd: New dimensions in cryptography. In M. Joye and G. Leander, editors, *Advances in Cryptology – EUROCRYPT 2024*, pages 3–32, Cham, 2024. Springer Nature Switzerland.
- [40] L. De Feo. Exploring isogeny graphs, 2018. Habilitation   diriger des recherches (HDR).
- [41] L. De Feo, T. B. Fouotsa, P. Kutas, A. Leroux, S.-P. Merz, L. Panny, and B. Wesolowski. Scallop: Scaling the CSI-FiSh. In A. Boldyreva and V. Kolesnikov, editors, *Public-Key Cryptography – PKC 2023, Proceedings, Part I*, volume 13940 of *Lecture Notes in Computer Science*, pages 345–375, Cham, 2023. Springer. doi:10.1007/978-3-031-31368-4_13.
- [42] L. De Feo and S. D. Galbraith. Seasign: Compact isogeny signatures from class group actions. In Y. Ishai and V. Rijmen, editors, *Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part III*, volume 11478 of *Lecture Notes in Computer Science*, pages 759–789. Springer, 2019. doi:10.1007/978-3-030-17659-4_26.
- [43] L. De Feo, J. Kieffer, and B. Smith. Towards practical key exchange from ordinary isogeny graphs. In T. Peyrin and S. D. Galbraith, editors, *Advances in Cryptology - ASIACRYPT 2018 - 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2-6, 2018, Proceedings, Part III*, volume 11274 of *Lecture Notes in Computer Science*, pages 365–394. Springer, 2018. doi:10.1007/978-3-030-03332-3_14.

- [44] L. De Feo, D. Kohel, A. Leroux, C. Petit, and B. Wesolowski. SQISign: Compact post-quantum signatures from quaternions and isogenies. In S. Moriai and H. Wang, editors, *Advances in Cryptology - ASIACRYPT 2020 - 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7-11, 2020, Proceedings, Part I*, volume 12491 of *Lecture Notes in Computer Science*, pages 64–93. Springer, 2020. doi:[10.1007/978-3-030-64837-4_3](https://doi.org/10.1007/978-3-030-64837-4_3).
- [45] L. De Feo, A. Leroux, P. Longa, and B. Wesolowski. New algorithms for the Deuring correspondence - towards practical and secure SQISign signatures. In C. Hazay and M. Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 659–690. Springer, 2023. doi:[10.1007/978-3-031-30589-4_23](https://doi.org/10.1007/978-3-031-30589-4_23).
- [46] T. Debris-Alazard, N. Sendrier, and J. Tillich. Wave: A new family of trapdoor one-way preimage sampleable functions based on codes. In *Advances in Cryptology - ASIACRYPT 2019, Proceedings, Part I*, volume 11921 of *Lecture Notes in Computer Science*, pages 21–51. Springer, 2019. doi:[10.1007/978-3-030-34578-5_2](https://doi.org/10.1007/978-3-030-34578-5_2).
- [47] W. Diffie and M. Hellman. New directions in cryptography. In *IEEE Transactions on Information Theory*, volume IT-22(6), pages 644–654, 1976. URL: <https://ee.stanford.edu/~hellman/publications/24.pdf>.
- [48] D. S. Dummit and R. M. Foote. *Abstract algebra*. Wiley, New York, 3rd ed edition, 2004.
- [49] P. Dusart. Explicit estimates of some functions over primes. *Ramanujan Journal*, 45:227–251, 2018. doi:[10.1007/s11139-016-9839-4](https://doi.org/10.1007/s11139-016-9839-4).
- [50] J. K. Eriksen, L. Panny, J. Sotáková, and M. Veroni. Deuring for the people: Supersingular elliptic curves with prescribed endomorphism ring in general characteristic. Cryptology ePrint Archive, Paper 2023/106, 2023. URL: <https://eprint.iacr.org/2023/106>.
- [51] T. Espitau, G. Niot, C. Sun, and M. Tibouchi. SQUIRRELS: Square unstructured integer euclidean lattice signature. NIST PQC Additional Digital Signatures submission, Specification v1.0, 2023. Accessed 2025-10-04. URL: <https://www.squirrels-pqc.org/squirrels-spec-v1.0.pdf>.

- [52] R. P. Feynman. Simulating physics with computers. *International Journal of Theoretical Physics*, 21(6-7):467–488, 1982. doi:[10.1007/BF02650179](https://doi.org/10.1007/BF02650179).
- [53] M. Fischlin. Progressive verification: The case of message authentication. In *INDOCRYPT 2003*, volume 2904 of *Lecture Notes in Computer Science*, pages 416–429, Berlin, Heidelberg, 2003. Springer. doi:[10.1007/978-3-540-24582-7_31](https://doi.org/10.1007/978-3-540-24582-7_31).
- [54] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang. Falcon: Fast-fourier lattice-based compact signatures over NTRU – submission to NIST post-quantum cryptography project, 2018. <https://falcon-sign.info/>.
- [55] S. Galbraith, L. Panny, B. Smith, and F. Vercauteren. Quantum equivalence of the DLP and CDHP for group actions. *CoRR*, abs/1812.09116, 2018. URL: <http://arxiv.org/abs/1812.09116>, arXiv:1812.09116.
- [56] S. Galbraith and R. S. Ruprai. An improvement to the Gaudry–Schost algorithm for multidimensional discrete logarithm problems. In M. G. Parker, editor, *Cryptography and Coding*, volume 5921 of *Lecture Notes in Computer Science*, pages 368–382, Berlin, Heidelberg, 2009. Springer. doi:[10.1007/978-3-642-10868-6_22](https://doi.org/10.1007/978-3-642-10868-6_22).
- [57] S. D. Galbraith and M. Holmes. A non-uniform birthday problem with applications to discrete logarithms. *Discrete Applied Mathematics*, 160(10):1547–1560, 2012. doi:<https://doi.org/10.1016/j.dam.2012.02.019>.
- [58] P. Gaudry and É. Schost. Construction of secure random curves of genus 2 over prime fields. In C. Cachin and J. Camenisch, editors, *Advances in Cryptology - EUROCRYPT 2004, International Conference on the Theory and Applications of Cryptographic Techniques, Interlaken, Switzerland, May 2-6, 2004, Proceedings*, volume 3027 of *Lecture Notes in Computer Science*, pages 239–256. Springer, 2004. doi:[10.1007/978-3-540-24676-3_15](https://doi.org/10.1007/978-3-540-24676-3_15).
- [59] P. Gaudry and É. Schost. A low-memory parallel version of Matsuo, Chao, and Tsujii’s algorithm. In D. Buell, editor, *Algorithmic Number Theory*, volume 3076 of *Lecture Notes in Computer Science*, pages 208–222, Berlin, Heidelberg, 2004. Springer. doi:[10.1007/978-3-540-24847-7_15](https://doi.org/10.1007/978-3-540-24847-7_15).
- [60] P. Gaudry and É. Schost. Genus 2 point counting over prime fields. *J. Symb. Comput.*, 47(4):368–400, 2012. URL: <https://doi.org/10.1016/j.jsc.2011.09.003>, doi:[10.1016/J.JSC.2011.09.003](https://doi.org/10.1016/J.JSC.2011.09.003).

- [61] C. Gentry, C. Peikert, and V. Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In R. E. Ladner and C. Dwork, editors, *Proceedings of the 40th Annual ACM Symposium on Theory of Computing (STOC '08)*, STOC '08, pages 197–206, Victoria, BC, Canada, 2008. ACM. doi:[10.1145/1374376.1374407](https://doi.org/10.1145/1374376.1374407).
- [62] A. Guillevic and F. Morain. Discrete Logarithms. In N. E. Mrabet and M. Joye, editors, *Guide to pairing-based cryptography*, page 42. CRC Press - Taylor and Francis Group, Dec. 2016. URL: <https://inria.hal.science/hal-01420485>.
- [63] J. L. Hafner and K. S. McCurley. A rigorous subexponential algorithm for computation of class groups. *Journal of the American Mathematical Society*, 2(4):837–850, 1989.
- [64] A. Hülsing, L. Rausch, and J. Buchmann. Optimal parameters for XMSS-MT. In *Security Engineering and Intelligence Informatics*, volume 8128 of *Lecture Notes in Computer Science*, pages 194–208, Berlin, Heidelberg, 2013. Springer. doi:[10.1007/978-3-642-40588-4_14](https://doi.org/10.1007/978-3-642-40588-4_14).
- [65] A. Hülsing and J. Rijneveld. XMSS reference implementation, 2024. <https://github.com/XMSS/xmss-reference>.
- [66] D. Jao, S. D. Miller, and R. Venkatesan. Expander graphs based on GRH with an application to elliptic curve cryptography. *Journal of Number Theory*, 129(6):1491–1504, 2009. doi:<https://doi.org/10.1016/j.jnt.2008.11.006>.
- [67] D. R. Kohel. *Endomorphism rings of elliptic curves over finite fields*. PhD thesis, University of California at Berkeley, 1996. <http://iml.univ-mrs.fr/~kohel/pub/thesis.pdf>.
- [68] G. Kuperberg. A subexponential-time quantum algorithm for the dihedral hidden subgroup problem. *SIAM Journal of Computing*, 35(1):170–188, 2005. arXiv:[quant-ph/0302112](https://arxiv.org/abs/quant-ph/0302112).
- [69] G. Kuperberg. Another Subexponential-time Quantum Algorithm for the Dihedral Hidden Subgroup Problem. In S. Severini and F. Brandao, editors, *8th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2013)*, volume 22 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 20–34, Dagstuhl, Germany, 2013. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik.

URL: <http://drops.dagstuhl.de/opus/volltexte/2013/4321>, doi:10.4230/LIPIcs.TQC.2013.20.

- [70] D. V. Le, M. Kelkar, and A. Kate. Flexible signatures: Making authentication suitable for real-time environments. In K. Sako, S. Schneider, and P. Y. A. Ryan, editors, *Computer Security – ESORICS 2019*, volume 11735 of *Lecture Notes in Computer Science*, pages 173–193, Cham, 2019. Springer. doi:10.1007/978-3-030-29959-0_9.
- [71] A. K. Lenstra, H. W. Lenstra, and L. Lovász. Factoring polynomials with rational coefficients. *Mathematische Annalen*, 261(4):515–534, 1982. doi:10.1007/BF01457454.
- [72] H. W. Lenstra Jr. Factoring integers with elliptic curves. *Annals of Mathematics*, 126(3):649–673, Nov. 1987. URL: <https://wstein.org/edu/124/lenstra/lenstra.pdf>, doi:10.2307/1971363.
- [73] A. Leroux. *Quaternion algebras and isogeny-based cryptography*. PhD thesis, École Polytechnique, Palaiseau, France, 2022. URL: https://www.lix.polytechnique.fr/Labo/Antonin.LEROUX/manuscrit_these.pdf.
- [74] J. López and R. Dahab. Fast multiplication on elliptic curves over $\text{GF}(2^m)$ without precomputation. In Ç. K. Koç and C. Paar, editors, *Cryptographic Hardware and Embedded Systems – CHES 1999*, volume 1717 of *Lecture Notes in Computer Science*, pages 316–327, Berlin, Heidelberg, 1999. Springer. URL: https://link.springer.com/chapter/10.1007/3-540-48059-5_27, doi:10.1007/3-540-48059-5_27.
- [75] L. Maino, C. Martindale, L. Panny, G. Pope, and B. Wesolowski. A direct key recovery attack on SIDH. In C. Hazay and M. Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 448–471. Springer, 2023. doi:10.1007/978-3-031-30589-4_16.
- [76] M. Meyer, F. Campos, and S. Reith. On lions and elligators: An efficient constant-time implementation of CSIDH. In J. Ding and R. Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019, Chongqing, China, May 8-10, 2019 Revised Selected Papers*, volume 11505 of *Lecture Notes in Computer Science*, pages 307–325. Springer, 2019. doi:10.1007/978-3-030-25510-7_17.

- [77] D. Micciancio and C. Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 700–718. Springer, 2012. doi:[10.1007/978-3-642-29011-4_41](https://doi.org/10.1007/978-3-642-29011-4_41).
- [78] M. Mitzenmacher and E. Upfal. *Probability and Computing: Randomized Algorithms and Probabilistic Analysis*. Cambridge University Press, 2005.
- [79] H. Montgomery and M. Zhandry. Full quantum equivalence of group action dlog and cdh, and more. In S. Agrawal and D. Lin, editors, *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part I*, volume 13791 of *Lecture Notes in Computer Science*, pages 3–32. Springer, 2022. doi:[10.1007/978-3-031-22963-3_1](https://doi.org/10.1007/978-3-031-22963-3_1).
- [80] P. L. Montgomery. Speeding the Pollard and elliptic curve methods of factorization. *Mathematics of computation*, 48(177):243–264, 1987.
- [81] G. L. Mullen and D. Panario. *Handbook of Finite Fields*. Chapman Hall/CRC, 1st edition, 2013.
- [82] K. Nakagawa, H. Onuki, W. Castryck, M. Chen, R. Invernizzi, G. Lorenzon, and F. Vercauteren. Sqsign2d-east: A new signature scheme using 2-dimensional isogenies. In K.-M. Chung and Y. Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 272–303, Singapore, 2025. Springer Nature Singapore.
- [83] National Bureau of Standards. Data encryption standard (DES). FIPS Publication 46, U.S. Department of Commerce, Jan. 1977. URL: <https://csrc.nist.gov/files/pubs/fips/46/final/docs/nbs.fips.46.pdf>.
- [84] National Institute of Standards and Technology. Data encryption standard (DES) and triple data encryption algorithm (TDEA). FIPS Publication 46-3, U.S. Department of Commerce, Oct. 1999. URL: <https://csrc.nist.gov/files/pubs/fips/46-3/final/docs/fips46-3.pdf>.
- [85] National Institute of Standards and Technology. Stateless hash-based digital signature standard. Technical Report Federal Information Processing Standards Publications (FIPS PUBS) 205, NIST, 8 2004. doi:[10.6028/NIST.FIPS.205](https://doi.org/10.6028/NIST.FIPS.205).

- [86] J. Neukirch. *Algebraic Number Theory*, volume 322 of *Grundlehren der mathematischen Wissenschaften*. Springer, Berlin, Heidelberg, 1999. URL: <https://web.math.ucsb.edu/~agboola/teaching/2021/fall/225A/neukirch.pdf>, doi: [10.1007/978-3-662-03983-0](https://doi.org/10.1007/978-3-662-03983-0).
- [87] P. Q. Nguyen and I. E. Shparlinski. Counting co-cyclic lattices. *SIAM Journal on Discrete Mathematics*, 30(3):1358–1370, 2016. doi:[10.1137/15M103950X](https://doi.org/10.1137/15M103950X).
- [88] K. Nishimura and M. Sibuya. Probability to meet in the middle. *J. Cryptol.*, 2(1):13–22, jan 1990. doi:[10.1007/BF02252867](https://doi.org/10.1007/BF02252867).
- [89] N. I. of Standards and Technology. Announcing request for nominations for public-key post-quantum cryptographic algorithms, 2016. <https://csrc.nist.gov/projects/post-quantum-cryptography>.
- [90] K. Okeya, H. Kurumatani, and K. Sakurai. Elliptic curves with the Montgomery-form and their cryptographic applications. In *Public Key Cryptography, Third International Workshop on Practice and Theory in Public Key Cryptography, PKC 2000, Melbourne, Victoria, Australia, January 18-20, 2000, Proceedings*, volume 1751 of *Lecture Notes in Computer Science*, pages 238–257. Springer, 2000. doi: [10.1007/978-3-540-46588-1_17](https://doi.org/10.1007/978-3-540-46588-1_17).
- [91] H. Onuki, Y. Aikawa, T. Yamazaki, and T. Takagi. A constant-time algorithm of CSIDH keeping two points. *IEICE Trans. Fundam. Electron. Commun. Comput. Sci.*, 103-A(10):1174–1182, 2020. doi:[10.1587/transfun.2019DMP0008](https://doi.org/10.1587/transfun.2019DMP0008).
- [92] A. Page and D. Robert. Introducing clapoti(s): Evaluating the isogeny class group action in polynomial time. Cryptology ePrint Archive, Paper 2023/1766, 2023. URL: <https://eprint.iacr.org/2023/1766>.
- [93] L. Panny, C. Petit, and M. Stopar. KLaPoTi: An asymptotically efficient isogeny group action from 2-dimensional isogenies. Cryptology ePrint Archive, Paper 2024/1844, 2024. URL: <https://eprint.iacr.org/2024/1844>.
- [94] C. Peikert. Public-key cryptosystems from the worst-case shortest vector problem: extended abstract. In *Proceedings of the Forty-First Annual ACM Symposium on Theory of Computing, STOC '09*, page 333–342, New York, NY, USA, 2009. Association for Computing Machinery. doi:[10.1145/1536414.1536461](https://doi.org/10.1145/1536414.1536461).
- [95] C. Peikert. A decade of lattice cryptography. Cryptology ePrint Archive, Paper 2015/939, 2015. URL: <https://eprint.iacr.org/2015/939>.

- [96] J. M. Pollard. Monte Carlo methods for index computation $(\bmod p)$. *Mathematics of Computation*, 32(143):918–924, 1978.
- [97] J. M. Pollard. Kangaroos, Monopoly and discrete logarithms. *J. Cryptology*, 13:437–447, 2000. doi:[10.1007/s001450010010](https://doi.org/10.1007/s001450010010).
- [98] C. Pomerance, J. Selfridge, and S. S. Wagstaff, Jr. The pseudoprimes to $25 \cdot 10^9$. *Mathematics of Computation*, 35(151):1003–1026, 7 1980.
- [99] T. Pornin and J. P. Stern. Digital signatures do not guarantee exclusive ownership. In *ACNS 2005*, volume 3531 of *LNCS*, pages 138–150, 2005. doi:[10.1007/11496137_10](https://doi.org/10.1007/11496137_10).
- [100] M. O. Rabin. Digitalized signatures and public-key functions as intractable as factorization. Technical Report 212, MIT Laboratory for Computer Science, 1979. URL: http://ncstrl.mit.edu/Dienst/UI/2.0/Describe/ncstrl.mit_lcs/MIT/LCS/TR-212.
- [101] O. Regev. On lattices, learning with errors, random linear codes, and cryptography. In *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*, STOC '05, pages 84–93, New York, NY, USA, 2005. Association for Computing Machinery. doi:[10.1145/1060590.1060603](https://doi.org/10.1145/1060590.1060603).
- [102] J. Renes. Computing isogenies between montgomery curves using the action of $(0, 0)$. In T. Lange and R. Steinwandt, editors, *Post-Quantum Cryptography (PQCrypto 2018)*, volume 10786 of *Lecture Notes in Computer Science*, pages 229–247. Springer, Cham, 2018. doi:[10.1007/978-3-319-79063-3_11](https://doi.org/10.1007/978-3-319-79063-3_11).
- [103] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Commun. ACM*, 21(2):120–126, feb 1978. doi:[10.1145/359340.359342](https://doi.org/10.1145/359340.359342).
- [104] D. Robert. Breaking SIDH in polynomial time. In C. Hazay and M. Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part V*, volume 14008 of *Lecture Notes in Computer Science*, pages 472–503. Springer, 2023. doi:[10.1007/978-3-031-30589-4_17](https://doi.org/10.1007/978-3-031-30589-4_17).
- [105] A. Rostovtsev and A. Stolbunov. Public-key cryptosystem based on isogenies. Cryptology ePrint Archive, Report 2006/145, 2006. <https://eprint.iacr.org/2006/145>.

- [106] R. S. Ruprai. *Improvements to the Gaudry–Schost algorithm for multidimensional discrete logarithm problems and applications*. Ph.d. thesis, Royal Holloway University of London, 2009.
- [107] D. Shanks. Class number, a theory of factorization and genera. *Proc. Symp. PureMath.*, 20:415–440, 1971.
- [108] P. W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Foundations of Computer Science, 1994 Proceedings., 35th Annual Symposium on*, pages 124–134. IEEE, 1994.
- [109] P. W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. SFCS ’94. IEEE Computer Society, 1994. doi:10.1109/SFCS.1994.365700.
- [110] J. H. Silverman. *The Arithmetic of Elliptic Curves*, volume 106 of *Graduate Texts in Mathematics*. Springer, New York, 2nd edition, 2009. doi:10.1007/978-0-387-09494-6.
- [111] B. Smith. Pre- and post-quantum diffie-hellman from groups, actions, and isogenies. In L. Budaghyan and F. Rodríguez-Henríquez, editors, *Arithmetic of Finite Fields - 7th International Workshop, WAIFI 2018, Bergen, Norway, June 14-16, 2018, Revised Selected Papers*, volume 11321 of *Lecture Notes in Computer Science*, pages 3–40. Springer, 2018. doi:10.1007/978-3-030-05153-2_1.
- [112] SPHINCS+ Team. SPHINCS+ reference implementation, 2024. <https://github.com/sphincs/sphincsplus>.
- [113] Squirrels team. Squirrels reference implementation, 2024. Package from <https://squirrels-pqc.org/> with sha256sum c4fd2a961f177a2f49970ca1758bde1a676a2fd7b3e62ea0e0c0593955902ab7.
- [114] A. Stolbunov. Reductionist security arguments for public-key cryptographic schemes based on group action. *Norsk informasjonssikkerhetskonferanse (NISK)*, pages 97–109, 2009.
- [115] A. Stolbunov. Constructing public-key cryptographic schemes based on class group action on a set of isogenous elliptic curves. *Advances in Mathematics of Communications*, 4(2):215–235, 2010. URL: </article/id/e8001706-6615-4b24-b499-8ea9d348dabb>, doi:10.3934/amc.2010.4.215.

- [116] A. R. Taleb and D. Vergnaud. Speeding-up verification of digital signatures. *Journal of Computer and System Sciences*, 116:22–39, 2021. URL: <https://www.sciencedirect.com/science/article/pii/S0022000020300854>, doi:<https://doi.org/10.1016/j.jcss.2020.08.005>.
- [117] J. H. van Lint. *Introduction to Coding Theory*, volume 86 of *Graduate Texts in Mathematics*. Springer-Verlag Berlin Heidelberg, 3 edition, 1999. doi:[10.1007/978-3-642-58575-3](https://doi.org/10.1007/978-3-642-58575-3).
- [118] P. C. van Oorschot and M. J. Wiener. Parallel collision search with cryptanalytic applications. *Journal of Cryptology*, 12(1):1–28, jan 1999. doi:[10.1007/PL00003816](https://doi.org/10.1007/PL00003816).
- [119] J. Vlu. Isognies entre courbes elliptiques. *Comptes rendus hebdomadaires des sances de l’Acadmie des sciences, Srie A*, 273:238–241, 7 1971. URL: <https://gallica.bnf.fr/ark:/12148/cb34416987n/date>.
- [120] J. Voight. *Quaternion Algebras*, volume 288 of *Graduate Texts in Mathematics*. Springer, Cham, 2021. doi:[10.1007/978-3-030-56694-4](https://doi.org/10.1007/978-3-030-56694-4).
- [121] S. S. Wagstaff. Is there a shortage of primes for cryptography? *International Journal of Network Security*, 3:296–299, 2006.
- [122] S. S. Wagstaff, Jr. Pseudoprimes and a generalization of Artin’s conjecture. *Acta Arithmetica*, 41:141–150, 1982.
- [123] W. C. Waterhouse. Abelian varieties over finite fields. *Annales scientifiques de l’cole Normale Suprieure*, 2(4):521–560, 1969. URL: <http://eudml.org/doc/81852>.
- [124] B. Wesolowski. The supersingular isogeny path and endomorphism ring problems are equivalent. In *2021 IEEE 62nd Annual Symposium on Foundations of Computer Science (FOCS)*, pages 1100–1111. IEEE, 2022. doi:[10.1109/FOCS52979.2021.00109](https://doi.org/10.1109/FOCS52979.2021.00109).
- [125] B. Wesolowski. Random walks in number-theoretic cryptology, Aug. 2024. Habilitation  diriger des recherches (HDR), Manuscript presented on 29 August 2024.
- [126] H. C. Williams. A modification of the RSA public-key encryption procedure. *IEEE Transactions on Information Theory*, 26:726–729, 1980.

- [127] P. Zimmermann and B. Dodson. 20 years of ECM. In F. Hess, S. Pauli, and M. Pohst, editors, *Algorithmic Number Theory – 7th International Symposium, ANTS-VII, Berlin, Germany, July 23–28, 2006, Proceedings*, volume 4076 of *Lecture Notes in Computer Science*, pages 525–542, Berlin, Heidelberg, 2006. Springer. [doi:10.1007/11792086_37](https://doi.org/10.1007/11792086_37).

Titre : Concevoir des systèmes cryptographiques post-quantiques efficaces, pratiques et sûrs

Mots clés : Cryptographie post-quantique, Réseaux, Isogénies, Action de groupe

Résumé : Cette thèse explore des stratégies d'optimisation pour la cryptographie post-quantique. Par systèmes cryptographiques post-quantiques, nous entendons des schémas classiques (non quantiques) dont la sécurité est destinée à résister à des attaques provenant d'ordinateurs quantiques.

Premièrement, nous étudions des systèmes cryptographiques dans le cadre GPV avec des réseaux ou des codes non structurés qui souffrent généralement de clés de grande taille. Nous traitons ce problème avec une méthode permettant de remplacer les grandes clés publiques dans les signatures de type GPV par de plus petites clés de vérification privées. Cela réduit significativement le stockage et le temps d'exécution du vérificateur tout en préservant la sécurité. Appliquée aux schémas à signatures courtes Wave et Squirrels, notre méthode compresse les clés de Squirrels-I de 665 kB à 20,7 kB et les clés de Wave822 de 3,5 MB à 207,97 kB. Cela est particulièrement utile lorsque de nombreuses vérifications sont effectuées avec la même clé (par exemple, les mises à jour logicielles, les certificats racine).

Deuxièmement, nous présentons quelques accélérations pour le calcul d'isogénies. L'une

réorganise l'évaluation des formules de Vélú pour accélérer l'évaluation des ℓ -isogénies pour de nombreux nombres premiers ℓ , ce qui constitue un calcul de base dans de nombreux systèmes cryptographiques à base d'isogénies. Une autre vise les isogénies rationnelles dont les noyaux sont engendrés par des éléments irrationnels. Nos techniques accélèrent les schémas à base d'isogénies tels que CSIDH, CSI-FiSh et CRS.

Enfin, nous proposons des algorithmes classiques de recherche parallèle de collisions pour les logarithmes discrets restreints multidimensionnels. La cryptographie basée sur les actions de groupes, et plus spécifiquement les *Restricted Effective Group Actions* (REGA), est profondément liée à ces problèmes. Une méthode existante pour résoudre les logarithmes discrets restreints multidimensionnels est l'algorithme de Gaudry–Schost. Cependant, il ne s'applique qu'à de faibles dimensions, ce qui ne correspond pas aux actions de groupes à base d'isogénies actuelles. Nous étendons l'algorithme de Gaudry–Schost à des dimensions plus élevées, offrant une vision plus approfondie des menaces classiques pour les échanges de clés REGA.

Title : Designing Efficient, Practical and Secure Post-quantum Cryptosystems

Keywords : Post-quantum cryptography, Lattices, Isogenies, Group action

Abstract : This thesis explores optimisation strategies for post-quantum cryptography. By post-quantum cryptosystems, we mean classical (non-quantum) schemes whose security is intended to resist attacks from quantum computers.

First, we study cryptosystems instantiating the GPV framework with unstructured lattices or codes which typically suffer from large keys. We address this with a method to replace large public keys in GPV-style signatures with smaller private verification keys. This significantly reduces verifier storage and runtime while preserving security. Applied to the conservative, short-signature schemes Wave and Squirrels, our method compresses Squirrels-I keys from 665 kB to 20.7 kB and Wave822 keys from 3.5 MB to 207.97 kB. This is especially useful when many verifications are performed using the same key (e.g., software updates, root certificates).

Second, we present some speed-ups for isogeny

computation. One reorganises the evaluation of Vélú's formulæ to speed up ℓ -isogeny evaluation for many primes ℓ , which is a basic computation in many isogeny-based cryptosystems. Another targets rational isogenies whose kernels are generated by irrational elements. Our techniques speed up isogeny-based schemes such as CSIDH, CSI-FiSh and CRS. Finally, we propose classical parallel collision search algorithms for multi-dimensional restricted discrete logarithms and hidden shifts. Group-action-based cryptography and, more specifically, Restricted Effective Group Actions (REGA) are deeply linked to these problems. One existing method for solving multi-dimensional restricted discrete logarithms is the Gaudry–Schost algorithm. However, it only applies to low dimensions which do not fit current isogeny-based group actions. We extend the Gaudry–Schost algorithm to higher dimensions, offering a deeper view of classical threats to REGA key exchanges.